

基于交会定位与覆盖搜索的 无线电干扰源清除策略

摘要

无线电干扰源的现场排查需要兼顾搜索范围、定位精度和清除时间。针对源数量、位置、接收半径及发射方向未知的情况，本文利用带误差的示向度建立位置可行域，将区域扫描与局部清除统一安排，形成机器狗的自动搜索策略。任务时间按移动、检测、换频和清除费用累计，在完成全部源清除的条件下比较不同策略。

针对问题一，将示向误差转化为正向楔形的半平面约束，通过可行性判别、边界交点枚举和顶点距离计算得到交会区域直径。构造边长 36 米的等边三角形算例，其最小包围圆半径为 20.785 米，说明以区域直径为直径的圆并不总能覆盖区域。进一步以最小包围圆判断一次清除的可行性，计算半径取 19.9 米，为设备 20 米清除范围预留余量。

针对问题二，分析交会夹角、目标距离与定位误差的关系。利用首次接收信息，得到三个半径 1000 米圆盘的交集作为第二检测点的安全候选区域，再根据第二次测向后的采样最坏包围半径评价 42 个候选点。算例中，横向候选点的包围半径为 65.426 米，共线对照为 375.231 米；增加 29.844 秒的移动与检测费用，明显改善了距离方向的定位效果。

针对问题三，以原点和正六边形环的六个点组成扫描骨架。环半径取 1200 米时，全域最坏覆盖距离为 968.90 米，较最小接收半径保留 31.10 米余量。将未访问的扫描点和已发现源共同排序，每次执行首项任务后重新规划。局部定位最多进行 6 次测向，随后直接清除或转入矩形光学覆盖；逐频道记录扫描和清除状态，确定任务停止时机。在 refined 基础上加入多初解路线组合、原地补测、全向负观测更新及安全接近，构成增强策略 field。

针对问题四，采用原点、940 米内环和 1870 米错相外环组成的 25 点骨架。通过 36 个闭三角形剖分及辅助点 $h = g + 5n$ ，证明任意源位置和发射方向都能被至少一个检测点接收；计入 1 米位置误差后，仍有 4 米前向余量且到源距离小于 999 米。信号中断后采用反射收缩的回退测向，并保留光学覆盖。结合有限任务数、检测次数和行程估计，得到总虚拟时间小于 100 小时的上界。

每题 1000 个同世界合成案例中，field 相对 refined 的平均单源时间分别降低 3.40% 和 6.23%，两题主组均完成全清。六次正式测试均采用 field，问题三、四分别成功清除 40 个和 37 个源，三局等权平均时间分别为 236.616 和 525.993 秒/源，均按频道完成条件结束。结果说明，覆盖布点与交会定位能够衔接全域搜索和近距清除，联合调度则有助于降低完整任务的移动费用。

关键词：交会定位；最小包围圆；全域覆盖；任务排序；定向干扰源；机器狗搜索

1 问题重述

1.1 问题背景

无线电干扰源会影响正常通信，快速确定其位置并完成清除是频谱管理中的重要任务。区域排查通常先通过测向设备获得目标的大致方向，再从不同位置继续检测，逐步缩小范围，最后进行近距离确认。机器狗可以代替人员进入危险区域，但它不能预先知道各个源的位置，需要根据检测结果不断调整行动。

题目将目标区域确定为半径 1800 米的圆域，圆心为坐标原点，东、北分别为 x 轴和 y 轴正向。源总数为 10 至 16，分布在 20 个频道中，每个频道至多对应一个源。源的接收半径为 1000 至 1500 米，示向度误差不超过 1° 。机器狗从原点出发，速度为 5 米/秒，初始检测频道为 1。进入源 20 米范围后可完成光学定位和清除 [1]。

问题三中的源全部全向发射；问题四中含有定向源，机器狗只有位于发射前方且距离不超过接收半径时才能收到信号。因此，定向源的搜索还需考虑检测点相对发射方向的位置。

1.2 需要解决的问题

根据题目要求，本文依次研究以下四个问题：

1. 由多个检测点及其示向度建立交会定位区域，计算区域直径，并判断直径圆的覆盖能力。
2. 利用全向源的首次测向结果，确定第二检测点的候选区域和选取方法。
3. 为全向源设计自动搜索、定位与清除策略，在所有源清除的条件下缩短任务时间。
4. 将策略扩展到定向源，处理背向无信号和圆域边界情形，完成模拟器测试并统计结果。

2 问题分析

2.1 问题一的分析

示向度存在误差，单次测量给出的是一个角度范围。将这个范围表示为以检测点为顶点的楔形，多个楔形的公共部分就是源的可能位置区域。它可能为空、无界，也可能为有界多边形，需要先分类，再求直径。

对于有界凸多边形，最远点对可在顶点中寻找，故直径计算可以化为有限个顶点间的距离比较。但区域直径与覆盖半径是两个不同的量，本文通过三角形算例分析二者的差别，并用最小包围圆连接后续清除任务。

2.2 问题二的分析

两次测向接近平行时，交会区域沿径向延伸，难以判断目标距离；增大交会夹角可以缩短该方向上的误差范围。不过，横向移动也会增加时间，并可能超出源的接收范

围。为此，先根据首次接收信息求出第二点的安全区域，再比较其中不同候选点的定位效果。

2.3 问题三的分析

源数量未知，使得停止条件成为搜索策略的一部分。仅清除已经发现的源并不足以结束任务，还要排除未扫描频道中存在其他源的可能。本文先设置有限个检测点，使其接收圆盘覆盖整个目标圆域，再为每个频道保留扫描记录。

在访问检测点的过程中，新发现的源会形成定位任务。若逐个源定位后返回原检测点，容易产生往返行程。因此，将未访问的扫描点与已发现源放入同一任务集合，每次只选择下一项任务，执行后重新排序。

2.4 问题四的分析

定向源的无信号有两种主要原因：距离过远，或机器狗位于发射背面。圆周上朝外发射的源尤其难以被圆内检测点发现。问题四需重新设计扫描点集，使每个可能源在任意发射方向上都有前方检测点。

本文采用错相双环布点，用三角形剖分建立方向覆盖条件。发现源后，仍使用问题一的交会区域；局部失去信号时向最近一次有效测点回退，测向次数达到上限后转为光学覆盖。

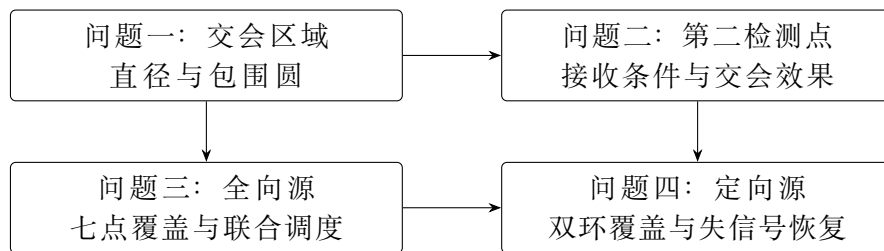


图 1: 四个问题的求解关系

3 模型假设

结合题目条件，模型采用以下约定。

1. 干扰源位置和发射方向在一次测试中保持不变，未清除前持续发射；每个有效频道至多有一个源。
2. 同一位置、同一频道的测向误差固定。定位时采用误差范围，不将原地重复读数视为独立样本。
3. 接口示向度保留两位小数，在线几何计算取 $\alpha = 1.005^\circ$ ，其中 0.005° 用于覆盖读数舍入。
4. 以外切正 180 边形表示目标圆域，局部清除规划半径取 19.9 米，分别保留边界位置和光学距离的计算余量。

5. 机器狗仅根据已经返回的检测、清除信息选择动作；源总数和类型在测试结束后用于评价。

上述约定不对源位置或测向误差引入概率分布。随机位置和误差场只用于第9节的数值试验。

4 符号说明与评价指标

表 1: 主要符号及含义

符号	含义	单位或范围
$\mathcal{D}, R_0$	目标圆域及其半径	$R_0 = 1800$ 米
g_c, R_c	频道 c 的源位置、接收半径	$R_c \in [1000, 1500]$ 米
n_c	定向源发射前向单位向量	$\ n_c\ = 1$
s_i, θ_i	第 i 个检测位置、示向度	米、度
P_c	频道 c 的位置可行域	凸多边形
$D(P), z_*(P), r_*(P)$	直径、最小包围圆圆心及半径	米
q, V, V_c	机器狗位置、扫描骨架、频道已测索引	位置及集合
L, M, S, F	总行程、检测数、换频数、失败清除数	米、次
C	清除调用总次数, $C = F + K$	次
K, N, T	成功清除数、实际源数、虚拟总时间	个、秒

4.1 接收与清除条件

全向源在 $\|s - g_c\| \leq R_c$ 时可被检测。对定向源，还需满足前向半平面条件，即

$$\|s - g_c\| \leq R_c, \quad n_c^\top (s - g_c) \geq 0. \quad (1)$$

在满足接收条件且距离不超过 5 米时，接口返回近源信息。光学清除只要求距离不超过 20 米，与发射方向无关。因此，定向源的电磁检测失败与光学清除失败具有不同含义。

4.2 任务时间与统计口径

机器狗从上一次位置直线移动至指定位置。累计移动距离为 L ，则按题目给定的动作费用，有

$$T = \frac{L}{5} + 5M + S + 3F + 5K. \quad (2)$$

成功清除的 5 秒已包含光学定位和激光清除；清除指令中的频道不改变测向机当前频道，只有检测频道变化才计换频时间。

题目要求在全清条件下尽量缩短时间，采用清除率和平均单源时间评价：

$$\eta = \frac{K}{N}, \quad \tau = \frac{T}{K} \quad (K > 0). \quad (3)$$

对 n 局测试，逐局等权平均与汇总指标分别为

$$\bar{\tau} = \frac{1}{n} \sum_{j=1}^n \frac{T_j}{K_j}, \quad \tau_{\text{pool}} = \frac{\sum_j T_j}{\sum_j K_j}. \quad (4)$$

前者给每个案例相同权重，后者给清除源数较多的案例更大权重。策略配对比较采用 $\bar{\tau}$ ；演练和正式结果同时列出两种统计量。程序在现实中执行的时间另记，不与虚拟任务时间相加。

5 问题一：交会区域的直径计算

5.1 示向度的半平面表示

令 $u(\theta) = (\cos \theta, \sin \theta)$ 。检测点 s_i 的示向度为 θ_i ，误差范围为 $[-\alpha, \alpha]$ ，则真实源位于以 s_i 为顶点的正向楔形内。用二维叉积 $\text{cross}(a, b) = a_x b_y - a_y b_x$ 表示，有

$$W_i = \left\{ g : \begin{array}{l} \text{cross}(u(\theta_i - \alpha), g - s_i) \geq 0, \\ \text{cross}(u(\theta_i + \alpha), g - s_i) \leq 0, \\ u(\theta_i)^T (g - s_i) \geq 0 \end{array} \right\}. \quad (5)$$

前两条分别限制源位于两条边界射线之间，第三条明确射线方向。在 $\alpha > 0$ 且角度较小时，前两条已含正向信息；保留第三条也便于处理零误差情形。三角函数形式能直接处理跨越 0° 的角度区间。

仅由 m 次测向得到的交会区域为

$$P = \bigcap_{i=1}^m W_i. \quad (6)$$

问题一先计算这个区域。问题三、四还可利用源所在圆域与最大接收距离收缩位置范围，这些附加条件在后文在线更新小节处理。

图2给出两次带误差测向的几何算例。左图显示检测位置和示向范围，右图放大公共区域，最远顶点连线对应待求直径。图中按 1.005° 容差计算交集，局部放大不改变误差角度。

5.2 区域分类与顶点计算

将式(5)的约束统一记为 $Ag \leq b$ 。由于测向方向和检测位置不同，交集可能为空、无界或有界，具体按以下步骤判断。

1. 求解二维线性可行性问题。如果不存在满足全部约束的点，判为观测约束矛盾。
2. 对非空区域，分别计算 x 和 y 的最大值、最小值。四个值均有限时区域有界；否则区域直径为 $+\infty$ 。
3. 对有界区域，枚举两条不平行边界直线的交点，保留满足所有半平面约束的点，去重后按凸边界排序。

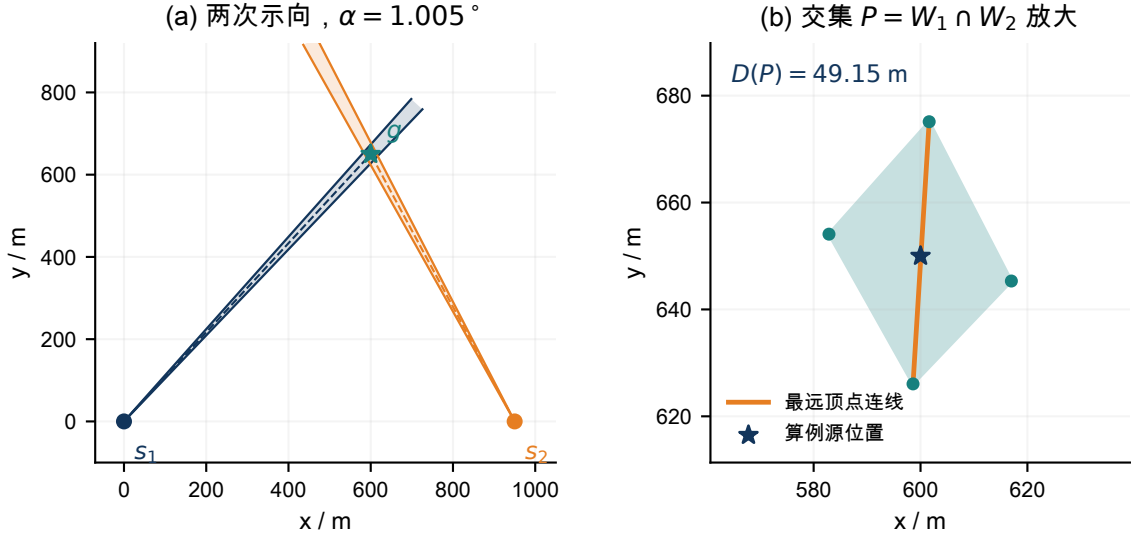


图 2: 两次测向的楔形交会与定位区域放大图 (自设几何算例)

4. 计算所有顶点对之间的欧氏距离，取最大值。若区域退化为线段，则取两端点距离；若退化为单点，则直径为 0。

设两条边界为 $a_i^T g = b_i$ 和 $a_j^T g = b_j$ ，记 $\Delta = a_{ix}a_{jy} - a_{iy}a_{jx}$ 。当 $\Delta \neq 0$ 时，交点坐标为

$$x = \frac{b_i a_{jy} - a_{iy} b_j}{\Delta}, \quad y = \frac{a_{ix} b_j - b_i a_{jx}}{\Delta}. \quad (7)$$

每个交点需检查全部约束，朴素算法的时间复杂度为 $O(m^3)$ 。本题一次局部定位使用的测向数量较少，该方法便于处理退化情形，计算量也较小。

5.3 直径的顶点性质

设非空有界区域 $P = \text{conv}\{v_1, \dots, v_k\}$ ，则

$$D(P) = \max_{1 \leq i, j \leq k} \|v_i - v_j\|. \quad (8)$$

证明如下。任意 $p, q \in P$ 均可写成顶点的凸组合： $p = \sum_i a_i v_i$ 、 $q = \sum_j b_j v_j$ ，其中系数非负且各自和为 1。因此

$$\begin{aligned} \|p - q\| &= \left\| \sum_{i,j} a_i b_j (v_i - v_j) \right\| \\ &\leq \sum_{i,j} a_i b_j \|v_i - v_j\| \leq \max_{i,j} \|v_i - v_j\|. \end{aligned} \quad (9)$$

另一方面，各顶点本身属于 P ，最大顶点距离可以在区域内取到，故式(8)成立。

5.4 直径圆的覆盖能力

区域的直径给出最大点间距离，但不直接给出覆盖整个区域的圆半径。取

$$A = (0, 0), \quad B = (36, 0), \quad C = (18, 18\sqrt{3}),$$

形成边长 36 米的等边三角形。它的直径为 36 米，外接圆半径为

$$r = \frac{36}{\sqrt{3}} \approx 20.784610 \text{ m} > 18 \text{ m}. \quad (10)$$

等边三角形的最小包围圆就是外接圆，因此任何直径 36 米的圆都不能覆盖其三个顶点。该例还说明，定位区域直径小于 40 米时，仍可能无法用 20 米光学半径一次覆盖。

为使反例对应题目的测向模型，依次沿三条有向边取单位方向 e_j ，令检测点为

$$S_j = V_j - \ell e_j, \quad \ell = 18\sqrt{3} \cot 2^\circ - 18. \quad (11)$$

示向读数分别取 1° 、 121° 、 241° ，理论误差为 $\pm 1^\circ$ 。各楔形的一条边沿三角形对应边，另一条经过对角顶点，交集恰为该三角形。检测点到三角形内各点的距离均小于 1000 米，符合接收条件。

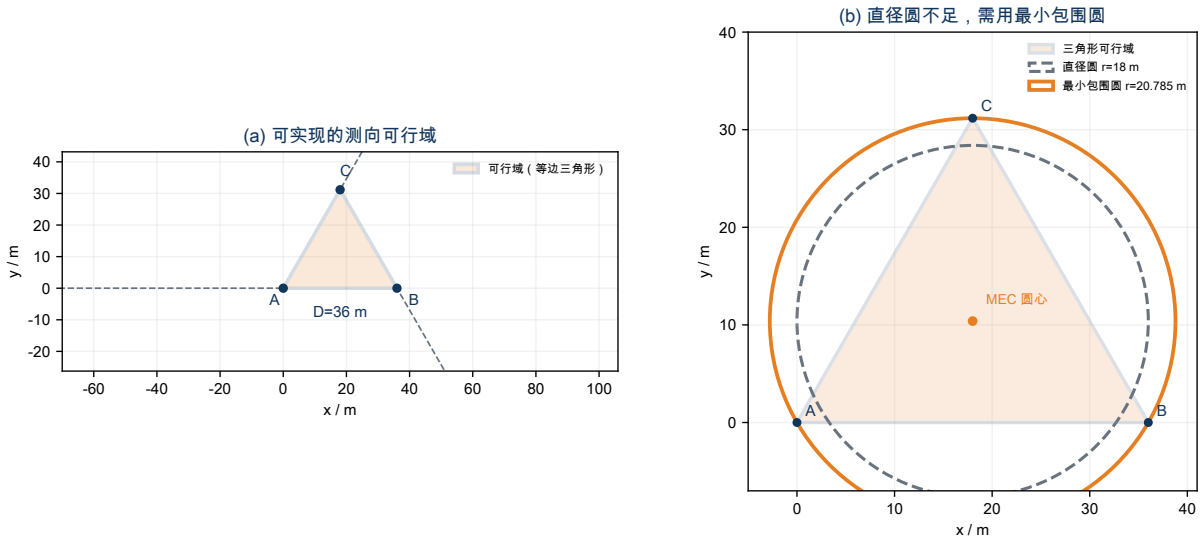


图 3: 三角形交会区域与两种覆盖圆的比较

5.5 最小包围圆与清除位置

为了从位置区域选择清除点，定义

$$r_*(P) = \min_z \max_{p \in P} \|p - z\|, \quad z_*(P) = \arg \min_z \max_{p \in P} \|p - z\|. \quad (12)$$

圆盘是凸集，只要包含多边形全部顶点，便包含整个多边形。最小包围圆可由一个、两个或三个支持点确定，采用增量算法求解 [2]。在浮点计算中，对近共线点和最终半径保留向外余量。

平面凸区域满足

$$\frac{D(P)}{2} \leq r_*(P) \leq \frac{D(P)}{\sqrt{3}}. \quad (13)$$

左侧来自圆盘中任意两点距离不超过直径。若包围圆由两个点确定，则 $r_* = D/2$ ；若由三个点确定，三个支持点中必有一对对应的圆心角不小于 120° ，弦长不小于 $\sqrt{3}r_*$ ，由此得到右侧。该平面包围圆界可由 Jung 定理得到 [3]。

因此, $r_*(P) \leq 20$ 是存在一次全域清除位置的判断条件。若只使用直径, 则 $D(P) \leq 20\sqrt{3}$ 为充分条件, $D(P) > 40$ 时不能一次覆盖; 介于二者之间时需继续求最小包围圆。程序实际采用 19.9 米半径, 并在选定点复算全部顶点距离。

5.6 在线定位区域的更新

一次有效示向还说明源距检测点不超过 1500 米。沿示向轴取投影, 得到线性约束

$$u(\theta_i)^\top (g - s_i) \leq 1500. \quad (14)$$

设 A_{180} 为半径 1800 米圆域的外切正 180 边形, 在线维护

$$P_c = A_{180} \cap \bigcap_{i \in I_c} (W_i \cap \{g : u(\theta_i)^\top (g - s_i) \leq 1500\}). \quad (15)$$

真实源初始位于 A_{180} 内, 每条新增正观测约束也包含它, 故逐步更新后仍有 $g_c \in P_c$ 。外切近似的最大径向偏差为 $1800(\sec(\pi/180) - 1) < 0.275$ 米。

新增一次观测时, 采用逐边界裁剪的方式更新凸多边形, 保留满足条件的顶点与边界交点 [4]。若出现空域, 说明观测或数值计算需要检查, 当前频道不能据此记为无源。该表示同时提供局部测向的候选范围和清除位置判断。

6 问题二：第二检测点的选择

6.1 交会几何对定位误差的影响

移动单站定位通过改变观测位置, 获得同一源在不同方向上的测向信息 [5]; 交叉定位中的方位误差还会随观测几何传播到位置结果 [6]。本问据此考察第二检测点的交会角和目标距离, 位置估计仍采用前述有界误差可行域。

单次测向将源限制在长而窄的扇形内, 但沿示向方向的距离仍不确定。设两次检测点到真实源的距离为 r_1, r_2 , 视线夹角为 β 。对小角度误差线性化, 位置扰动 δg 在每条视线法向 n_i 上满足

$$|n_i^\top \delta g| \leq \alpha r_i. \quad (16)$$

两条误差带的交集近似为平行四边形, 面积为

$$A \approx \frac{4\alpha^2 r_1 r_2}{|\sin \beta|}, \quad (17)$$

其中 α 以弧度计。固定距离时, 交会角接近 90° 有利于缩小误差; 夹角接近 0° 或 180° 时, 交会区域沿一个方向拉长。

第二点移动还会改变 r_2 , 所以不能只按夹角选择。本文先限制第二点位于可接收区域, 再通过楔形裁剪计算定位效果。式(17)用于解释选点趋势, 实际计算仍采用完整的角度约束。

图4固定两次测向距离, 比较不同交会角对应的面积因子。两条视线接近平行时, 误差面积迅速增大; 接近直角时, 该因子最小。实际选点还需同时考虑接收条件和移动距离。

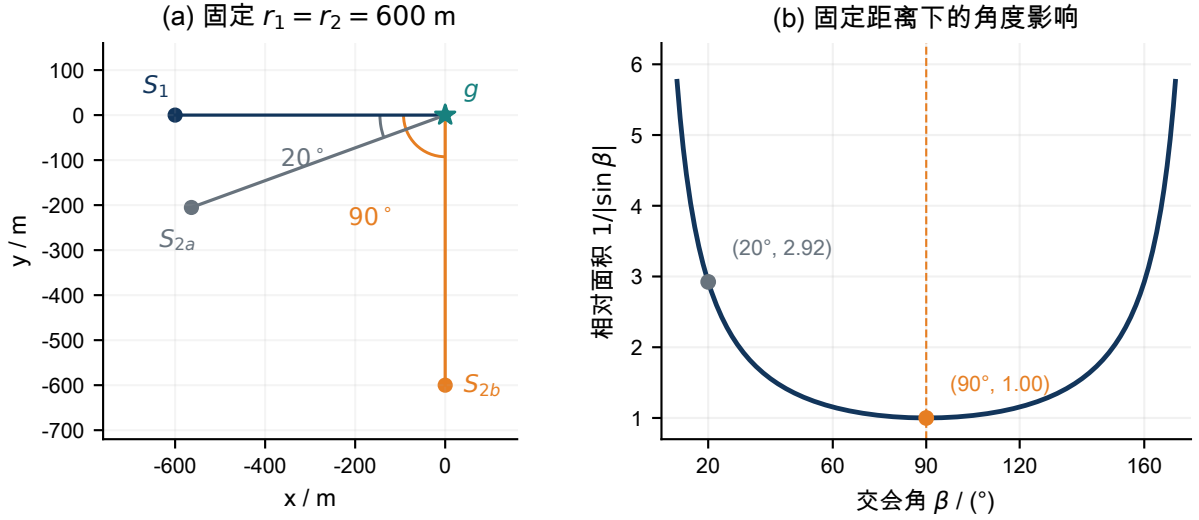


图 4: 固定测向距离时的交会角与相对误差面积

6.2 由首次接收得到的候选条件

记首次检测点为 S_1 , 读数方向为 u , 法向为 n , 第二点表示为

$$S_2 = S_1 + tu + bn. \quad (18)$$

其中 t 是沿首次方向的前进距离, b 是横向位移。设源到 S_1 的真实距离为 r 。由于首次已经接收到信号, 未知接收半径至少满足

$$R_c \geq \max(1000, r). \quad (19)$$

因此, 对首次观测允许的所有源位置 g , 若第二点满足

$$\|S_2 - g\| \leq \max(1000, \|g - S_1\|), \quad (20)$$

便能够再次接收信号。

令 $u_{\pm} = u(\theta_1 \pm \alpha)$, 把首次可行位置放宽为 $r \in [0, 1500]$ 、角差 $\varphi \in [-\alpha, \alpha]$ 的扇形, 得到三圆交候选域

$$\mathcal{C} = S_1 + [B(0, 1000) \cap B(1000u_-, 1000) \cap B(1000u_+, 1000)]. \quad (21)$$

该候选域使用接收半径下界和首次接收事实, 不需要估计目标的具体距离。

6.3 三圆交条件的推导

取 $q = S_2 - S_1$ 。从 $r = 0$ 及 $r = 1000, \varphi = \pm\alpha$ 这三种位置, 可得式 (21) 中的三个圆盘约束。下面验证这些约束对整个放宽扇形均适用。

由 $\|q - 1000u_{\pm}\| \leq 1000$, 展开可得

$$q^T u_{\pm} \geq \frac{\|q\|^2}{2000} \geq 0. \quad (22)$$

对于两端方向之间的小圆弧，投影 $q^\top u_\varphi$ 的最小值位于端点，因而也满足式(22)。

当 $0 \leq r \leq 1000$ 时， $\|q - ru_\varphi\|$ 是 r 的凸函数。 $r = 0$ 和 $r = 1000$ 两端的值均不超过 1000，内部也不超过 1000。当 $r \geq 1000$ 时，

$$\|q - ru_\varphi\|^2 - r^2 = \|q\|^2 - 2rq^\top u_\varphi \leq 0. \quad (23)$$

于是源到第二点的距离不超过 $\max(1000, r)$ ，式 (20) 成立。实际源位置还受目标圆域约束，采用放宽扇形可使选点条件保持统一。

6.4 候选点生成与排序

在以首次示向方向为 t 轴的坐标系中，三圆交写成

$$\begin{cases} t^2 + b^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b - 1000 \sin \alpha)^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b + 1000 \sin \alpha)^2 \leq 1000^2. \end{cases} \quad (24)$$

给定 t 后，横向边界为

$$b_{\max}(t) = \min \left\{ \sqrt{1000^2 - t^2}, \sqrt{1000^2 - (t - 1000 \cos \alpha)^2} - 1000 \sin \alpha \right\}. \quad (25)$$

根号有定义且 $b_{\max} \geq 0$ 时，取 $0 < t < 1000$ 和 $0 < |b| \leq b_{\max}(t)$ ，形成首次方向两侧的候选区域。计算中设置

$$\begin{aligned} t &\in \{125, 250, 375, 500, 625, 750, 875\} \text{ m}, \\ b &= \pm \kappa b_{\max}(t), \quad \kappa \in \{1/4, 1/2, 3/4\}, \end{aligned} \quad (26)$$

共得到 42 个候选点。

对每个候选点，枚举当前区域中的样本源位置及第二次测向误差，计算新楔形与当前区域的交集，再求最小包围圆半径。以样本中的最大半径作为该点的评分，优先选择评分较小的点；几何效果接近时再比较本步移动费用。这使候选点的接收条件由三圆交确定，定位效果由实际交会计算得到。

6.5 算例结果与分析

令 $S_1 = (0, 0)$ 、 $\theta_1 = 0$ ，设置 60 个径向位置、3 个首次真实角差和 3 个第二次测向误差。由此比较 42 个候选点与共线对照，结果见表2。

表 2: 第二检测点算例结果

测点类型	坐标/米	采样最坏半径/米	移动与检测/秒
横向候选	(750, -496.078)	65.426	184.844
共线对照	(750, 0)	375.231	155.000

横向候选增加约 29.844 秒的本步费用，但将样本中的最坏包围半径从 375.231 米减至 65.426 米，说明横向基线对径向误差具有明显改善作用。该半径仍大于 20 米，后续还需继续测向或实施光学搜索。此例给出离散候选中的选择结果，在线多源搜索则采用中心横向测点。

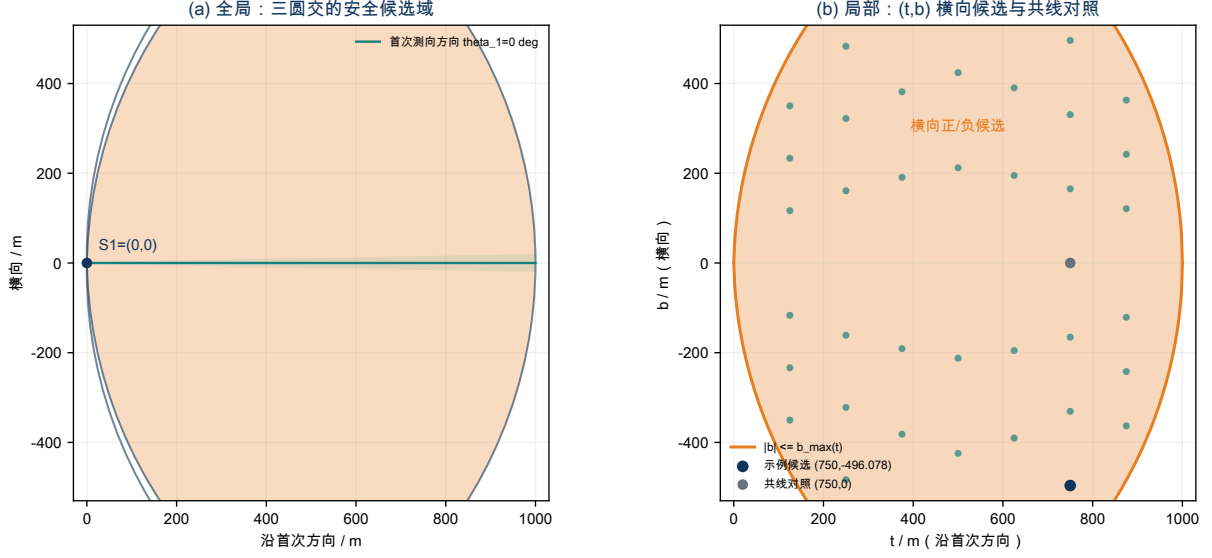


图 5: 第二检测点的接收区域与横向候选

7 问题三：全向干扰源的搜索、定位与清除

7.1 七点扫描骨架

问题三所有源均全向发射。只要检测点的 1000 米接收圆盘覆盖整个目标圆域，所有存在的频道便能在有限次扫描中至少收到一次信号。选取原点与半径 d 的正六边形环：

$$p_0 = (0, 0), \quad p_{k+1} = d u(k\pi/3), \quad k = 0, \dots, 5. \quad (27)$$

这些点只规定全域扫描位置，实际访问顺序随已发现源和机器狗当前位置调整。

为计算覆盖半径，设任意源 $g = ru(\theta)$ 。总能选择一个环点，使其极角与源相差不超过 $\pi/6$ 。在半径 r 的圆周上，距最近检测点的最坏距离为

$$f_d(r) = \min \left\{ r, \sqrt{r^2 + d^2 - \sqrt{3}dr} \right\}. \quad (28)$$

第一项对应原点，第二项对应相邻环点的角平分线。两项在 $r = d/\sqrt{3}$ 处相等；在此之前原点更近，之后环点更近。后一项的平方为凸二次函数，故其最大值在区间端点取到。

在 $1200 \leq d \leq 900\sqrt{3}$ 的构型范围内，全域覆盖半径为

$$\rho(d) = \max \left\{ \frac{d}{\sqrt{3}}, \sqrt{R_0^2 + d^2 - \sqrt{3}R_0d} \right\}. \quad (29)$$

两个候选最大值分别出现在圆内的等距点和圆周的角平分线处，因而式(29)也给出了该构型的实际最坏距离。

7.2 环半径与行程的选择

当 $d = 900\sqrt{3}$ 时，式(29)两项均为 900 米。向内收缩检测环能够缩短行程，同时会增大圆周处的最坏接收距离。选取 $d = 1200$ 米，可得

$$\begin{aligned}\rho(1200) &= \sqrt{1800^2 + 1200^2} - \sqrt{3} \cdot 1800 \cdot 1200 \\ &\approx 968.901572 \text{ m},\end{aligned}\quad (30)$$

$$1000 - \rho(1200) \approx 31.098428 \text{ m}.$$

因此七点骨架在缩短行程后仍保留约 31.10 米距离余量。若理想检测点与实际位置相差不超过 η 米，由三角不等式，最坏距离增加量不超过 η 。

从原点出发访问六个环点，首段至少为 d ，任意两个不同环点的距离不小于 d 。沿圆周依次访问恰达到

$$L_{\text{ring}}^{\min} = 6d. \quad (31)$$

故完整骨架路线由 $5400\sqrt{3}$ 米缩短为 7200 米。这个行程只包含骨架访问，完整任务还需加上定位支线和操作费用。

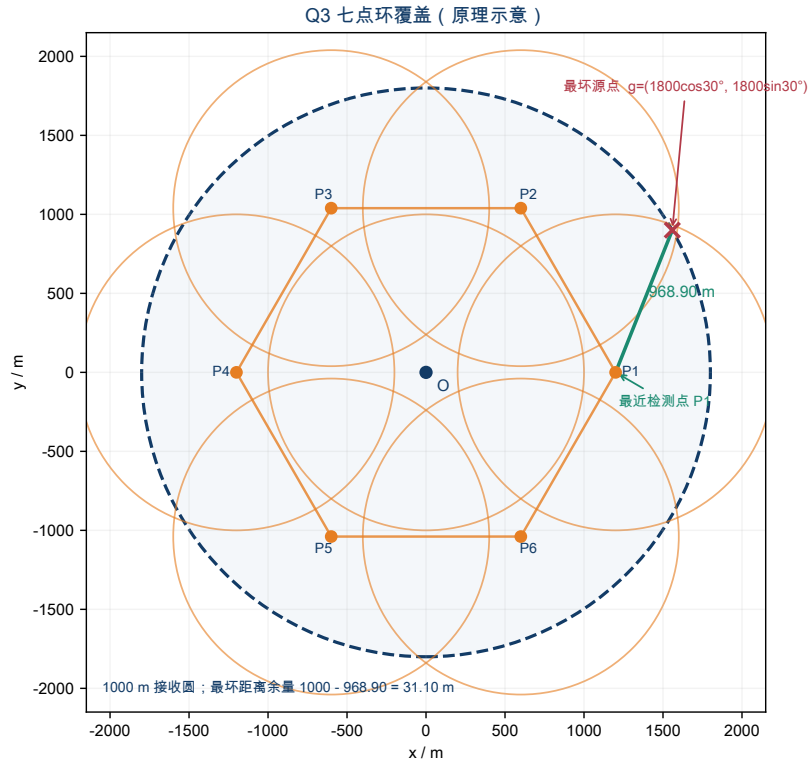


图 6: 七点环的接收覆盖及圆周最坏位置

七点方案有明确的覆盖依据。作为点数比较，在 900 米覆盖要求下，单个接收盘对半径 1800 米圆周的交弧宽度最多为 60° 。若仅用六盘覆盖整条圆周，六盘必须全部取到最大弧宽，并按正六边形排列；此时原点仍未被覆盖，还需增加一个中心盘。这个七点下界针对 900 米接收盘，1000 米物理下界下的任意布局不在该下界的结论范围内。

图7将式(29)和式(31)放在同一参数范围内比较。扩大环半径能够缩短最坏接收距离，但同时增加骨架行程；1200 米环在保留接收余量的同时采用较短的扫描路线。

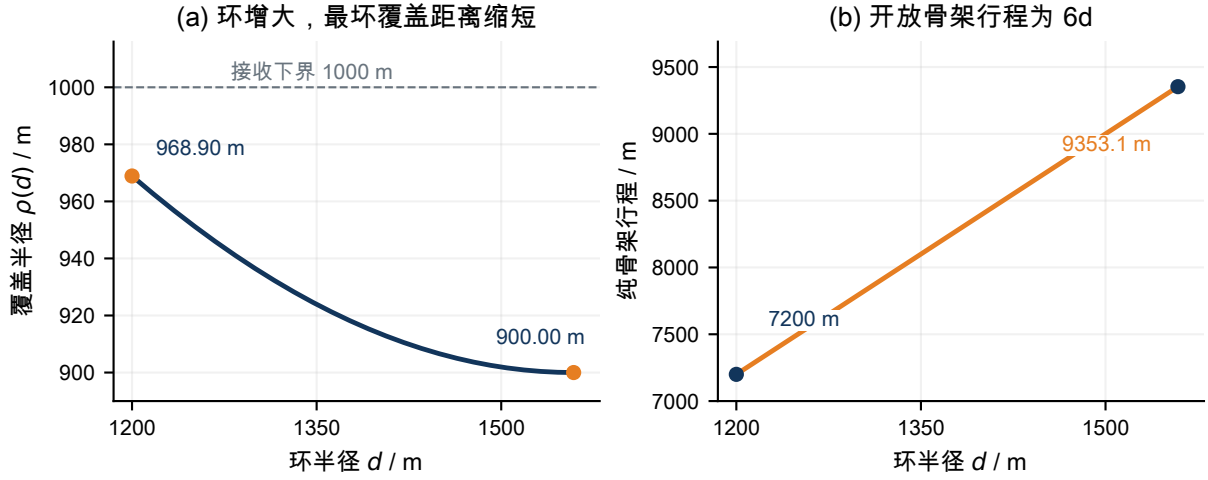


图 7: 七点环覆盖距离与开放骨架行程的变化 (不含定位及操作费用)

7.3 扫描任务与定位任务的联合排序

设当前位置为 q 。待执行任务分为两类：尚未完成的骨架扫描点，以及已经发现但未清除的源。扫描点用实际坐标表示，源任务用当前可行域的最小包围圆中心 z_c 表示。为统一符号，记各任务坐标为 z_j ，扫描任务半径为 0，源任务半径为 r_j 。

对任务排列 π ，构造开放路线评分

$$J_\gamma(\pi) = \|q - z_{\pi_1}\| + \sum_{j=1}^{n-1} \|z_{\pi_{j+1}} - z_{\pi_j}\| + \gamma r_{\pi_1}. \quad (32)$$

问题三取 $\gamma = 0$ ，按任务中心之间的行程排序。基准策略 refined 先用最近插入法形成完整排列 [7]，再进行有界 2-opt 改进 [8]：反转一段连续路线，若首尾连接费用下降则接受。每轮只执行得到的第一项任务，执行后重新计算位置区域和剩余任务。

扫描点与源任务保持各自身份，即使坐标相同也不合并。这样既可减少定位结束后返回原扫描点的绕行，也能保留未知频道的扫描义务。由于后续观测可能改变目标中心和清除终点，式(32)作为当前任务排序依据，实际费用仍按式(2)累计。

7.4 逐频道扫描规则

到达一个骨架点后，优先检测测向机当前频道，再依频道编号检测其余频道。已清除频道跳过；未知频道必须检测；已有有效示向且尚未清除的频道作为独立定位任务，不再要求在每个骨架点重复检测。

每次局部清除结束后，机器狗还可以在当前位置为其他已知频道补充测向。当该频道的包围半径大于 19.9 米、当前位置到其包围圆的最短距离不超过 1500 米，且未在完全相同坐标测过时，进行一次停靠共享检测。它不增加移动距离，但照常计入检测和换频费用。

7.5 局部定位与清除步骤

对已发现频道 c ，根据式(15)维护位置多边形，计算其包围圆 $B(z, r)$ 。局部过程先检查能否直接清除，再决定是否继续测向。

1. 若当前位置到所有顶点距离不超过 19.9 米，则原地清除；否则若 $r \leq 19.9$ 米，则前往 z 清除。
2. 若 $19.9 < r \leq 38$ 米，先在 z 作一次光学尝试。成功即结束该频道；失败后进入矩形光学覆盖。
3. 若区域仍较大，取最近一次有效测向点 s^+ ，令

$$e = \frac{z - s^+}{\|z - s^+\|}, \quad n = (-e_y, e_x),$$

在中心两侧设置候选点

$$p_{\pm} = z \pm \delta n, \quad \delta = \min\{120, \max\{25, 0.22r\}\} \text{ m}. \quad (33)$$

从未重复测过、未尝试失败的候选中选较近者。

4. 有效示向继续裁剪 P_c ；返回近源则原地清除。若无信号，在该点先作一次光学尝试，失败后再决定下一测点。问题三累计两次无信号后转入光学覆盖。
5. 每次局部定位最多使用 6 个新测向点；达到上限、没有新候选或中心方向退化时，转入光学覆盖。

其中 38 米是试清与继续测向之间的切换阈值，19.9 米是整个区域的一次清除判据。中心横向规则借助新的交会角缩小可行域，计算量小；它不要求每个在线测点都位于问题二的三圆交中，因此保留无信号处理分支。

7.6 矩形光学覆盖

当测向收缩不足时，取 P_c 的一条直径方向作为局部 x 轴，计算包含整个多边形的投影矩形 $[x_-, x_+] \times [y_-, y_+]$ 。设有效网格半径为 $\rho_o = 19.9 - \varepsilon$ ，取单元边长上限 $h = \sqrt{2}\rho_o$ ，划分数为

$$N_x = \max\left(1, \left\lceil \frac{x_+ - x_-}{h} \right\rceil\right), \quad N_y = \max\left(1, \left\lceil \frac{y_+ - y_-}{h} \right\rceil\right). \quad (34)$$

第 (i, j) 格中心为

$$a_{ij} = \left(x_- + \frac{i + 1/2}{N_x}(x_+ - x_-), \quad y_- + \frac{j + 1/2}{N_y}(y_+ - y_-)\right). \quad (35)$$

任一单元的半对角线不超过 $h/\sqrt{2} = \rho_o$ ，故半径 19.9 米的圆盘能够覆盖整个单元。由此得到

$$P_c \subseteq \bigcup_{i=0}^{N_x-1} \bigcup_{j=0}^{N_y-1} B(a_{ij}, 19.9). \quad (36)$$

这些圆盘的并集覆盖整个区域，而非只覆盖多边形顶点。逐个中心并不需要覆盖整个 P_c ，因此部分清除失败属于正常搜索过程。

区域分解后逐行往复访问，是机器人覆盖规划中的常用思路 [9]。本文按蛇形排列访问各中心，再将序列循环移位，使首点靠近当前位置。真实源在某个单元内，遍历过程中至少有一个中心落在它的清除范围内，收到成功反馈后立即结束该频道。

局部定位与光学兜底（原理示意，非观测轨迹）

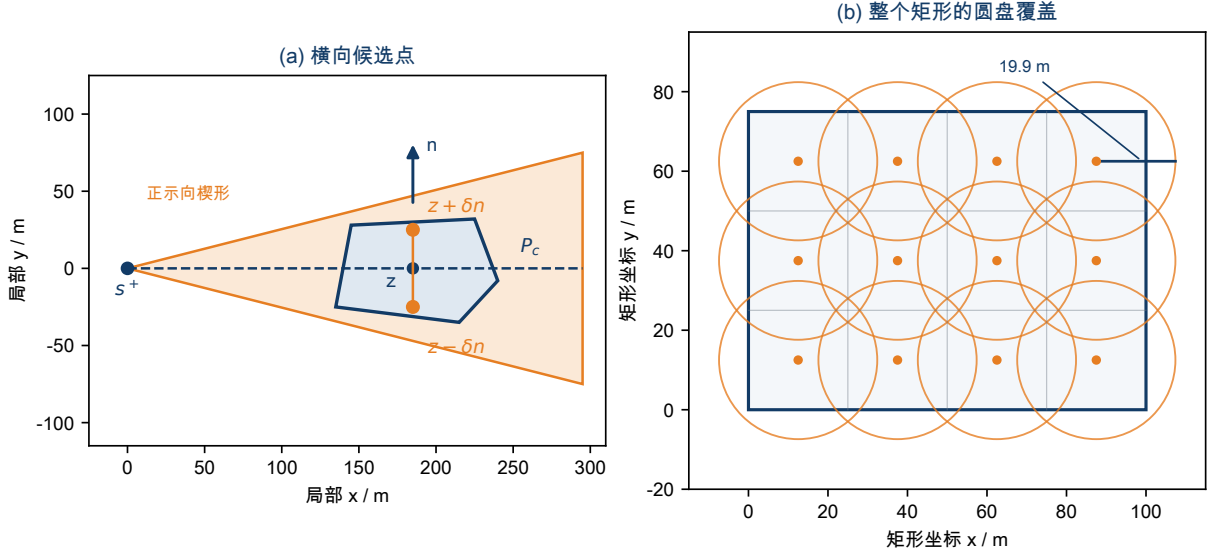


图 8: 中心横向测点与矩形光学覆盖示意

7.7 全向负观测与安全接近

在基准策略上，增强策略 field 利用全向无信号信息缩小位置区域。若某频道尚有源，在位置 s 检测无信号，则有 $\|g_c - s\| > 1000$ 。直接从凸多边形中减去圆盘会产生非凸区域，因此采用凸外包近似。

在 1000 米圆盘内部构造正十六边形

$$Q_s = \text{conv} \left\{ s + 1000 \cos \frac{\pi}{16} u \left(\frac{2j\pi}{16} \right) : j = 0, \dots, 15 \right\}. \quad (37)$$

令 H_j^{out} 为其第 j 条边的闭外侧半平面，更新为

$$P_c^+ = \text{conv} \left(\bigcup_{j=0}^{15} (P_c \cap H_j^{\text{out}}) \right). \quad (38)$$

由于 Q_s 严格位于接收圆盘内，圆盘外的真实源一定落在至少一个保留半平面中；取凸包仍包含它，且 $P_c^+ \subseteq P_c$ 。算法保存发现源之前的无信号位置，建立轨迹后也可应用这些信息。

field 还在已经满足全域清除条件时缩短接入距离。设原计划清除点为 a ，当前位置为 q ，沿 $u = (q - a) / \|q - a\|$ 取 $a' = a + tu$ 。对每个顶点 v 要求

$$t^2 + 2(a - v)^T u t + \|a - v\|^2 - 19.9^2 \leq 0. \quad (39)$$

以所有顶点给出的上界和 $\|q - a\|$ 中的最小值确定 t ，再复验新点到全部顶点的距离。该操作只改变已确定清除点的接近位置；基准 refined 不使用此项优化。

7.8 频道状态与停止条件

对每个频道记录四种状态：未确定 U 、已发现 D 、已清除 C 和完成覆盖后无源 A 。设七点骨架为 V ，频道 c 已经实际检测的骨架索引为 V_c 。只有从未发现且 $V_c = V$ 的频道，才可以由全无信号转为 A ；已经发现的频道由清除成功转为 C 。

停止条件为

$$[\forall c, \text{state}_c \in \{C, A\}] \quad \text{或} \quad [\#\{c : \text{state}_c = C\} = 16]. \quad (40)$$

第一种情况由七点全域覆盖排除了未发现源，第二种情况使用题给源数上界。已清除 10 个源或当前没有定位任务时，仍继续完成剩余未知频道的扫描。

8 问题四：定向干扰源的搜索、定位与清除

对定向源，距离足够近还需满足式 (1) 的前向条件。圆周源若朝圆外发射，所有严格位于圆内的点都可能在发射背面。因此在目标圆域外增加一层检测点，并与内部点形成连续剖分。

取内环半径 $a = 940$ 米、外环半径 $b = 1870$ 米，定义

$$\begin{aligned} O &= (0, 0), \\ I_k &= a u(k\pi/6), \\ E_k &= b u(k\pi/6 + \pi/12), \quad k = 0, \dots, 11. \end{aligned} \quad (41)$$

内外环相邻点分别相差 30° ，两环错开 15° 。点集包括原点、12 个内环点和 12 个外环点，共 25 个位置。外环点虽超出源所在圆域，但属于机器狗允许访问的位置。

8.1 三角剖分及距离界

以下下标按模 12 计算，构造

$$\begin{aligned} T_k^{(1)} &= \text{conv}\{O, I_k, I_{k+1}\}, \\ T_k^{(2)} &= \text{conv}\{I_k, E_k, I_{k+1}\}, \\ T_k^{(3)} &= \text{conv}\{I_{k+1}, E_k, E_{k+1}\}. \end{aligned} \quad (42)$$

第一类三角形填满内十二边形，后两类填满内外两环之间的带状区域。连接边位于对应的 15° 扇区内，且 $a < b \cos(\pi/12)$ ，因此各三角形内部不交叉，共同组成外正十二边形 $H = \text{conv}\{E_k\}$ 。

记 $c = \cos(\pi/12)$ 、 $s = \sin(\pi/12)$ ，剖分中仅有四类边长：

$$\ell_0 = a, \quad \ell_I = 2as, \quad \ell_E = 2bs, \quad \ell_{IE} = \sqrt{a^2 + b^2 - 2abc}. \quad (43)$$

利用 $c > 0.9659$ 、 $s < 0.259$ ，有

$$\begin{aligned} bc &> 1806.233 > 1805, \\ \ell_0 &= 940 < 993, \quad \ell_I < 486.92 < 993, \\ \ell_E &< 968.66 < 993, \\ \ell_{IE}^2 &< 984781.96 < 993^2. \end{aligned} \quad (44)$$

于是 H 包含半径 1805 米的圆盘，每个三角形直径小于 993 米。这两个界分别保证辅助点位于剖分内，并限制辅助点到三角形顶点的距离。

8.2 任意发射方向的覆盖证明

任取源位置 $g \in \mathcal{D}$ 与发射单位向量 n ，沿发射方向引入辅助点

$$h = g + 5n. \quad (45)$$

因 $\|g\| \leq 1800$ ，有 $\|h\| \leq 1805$ ，所以 h 属于剖分中的某个闭三角形 T 。将其表示为顶点的重心组合

$$h = \sum_{i=1}^3 \lambda_i v_i, \quad \lambda_i \geq 0, \quad \sum_i \lambda_i = 1,$$

沿 n 方向投影得到

$$\sum_i \lambda_i n^\top (v_i - g) = n^\top (h - g) = 5. \quad (46)$$

至少存在一个顶点 v 使 $n^\top (v - g) \geq 5$ 。同时 v, h 位于同一三角形，故

$$\|v - g\| \leq \|v - h\| + \|h - g\| < 993 + 5 = 998. \quad (47)$$

若实际检测点 v' 与理想点相差不超过 1 米，则

$$n^\top (v' - g) \geq 4 \text{ m}, \quad \|v' - g\| < 999 \text{ m}. \quad (48)$$

因此所选点严格位于发射前方，且距源小于最小接收半径 1000 米。该证明对任意 g, n 均成立，源在圆周、辅助点落在边或顶点上也适用。

辅助点仅用于证明，机器狗不需要估计源的发射方向或访问 h 。1 米位置余量表示坐标计算、舍入和停靠偏差的总量。方向覆盖的实际实现仍按式(41)给出的 25 个固定点执行。

8.3 骨架路径与扫描次数

一条访问 25 点的开放路线为

$$O, I_0, I_1, \dots, I_{11}, E_{11}, E_{10}, \dots, E_0.$$

对应长度为

$$L_{25} = a + 22(a + b) \sin \frac{\pi}{12} + \sqrt{a^2 + b^2 - 2ab \cos \frac{\pi}{12}} \approx 17932.51 \text{ m}. \quad (49)$$

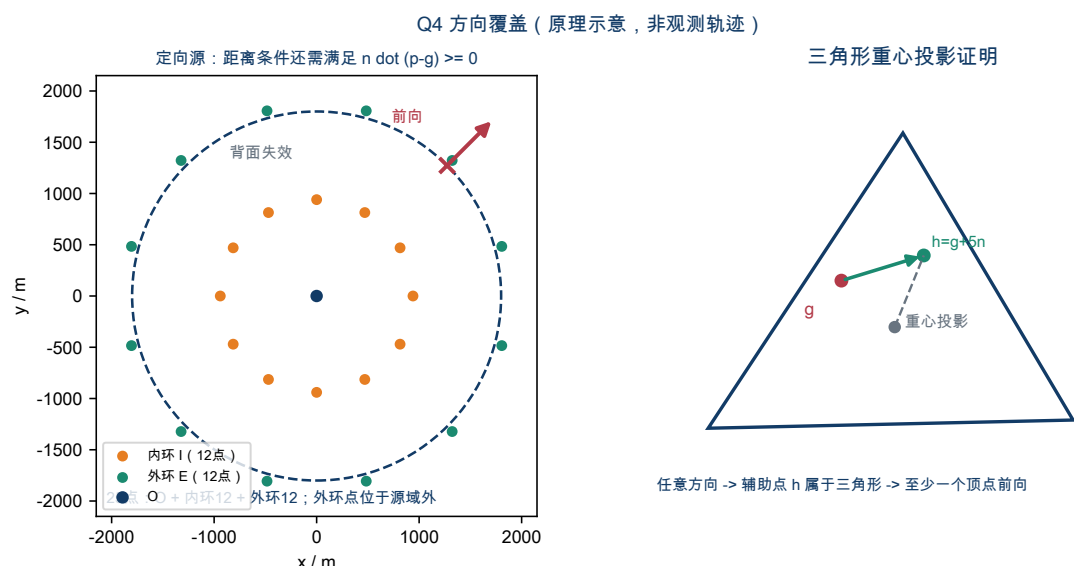


图 9: 双环检测点与前向投影的几何关系

它用于说明骨架能够连成有限路线, 实际顺序由任务调度决定。相较 31 点骨架, 若 20 个频道均完成全部扫描, 25 点方案少 120 次检测, 对应检测项少 600 秒; 实际任务会随频道清除和已知频道补测筛选而改变次数。

图10比较上述可行骨架路线与已有的 refined 本地模拟轨迹。右图取种子 20260910 的单局记录, 只按已执行动作连接位置, 标出的清除位置不是源坐标真值。两条路线的区别反映了局部定位支线与在线任务重排, 不能把纯骨架长度直接当作完整任务行程。

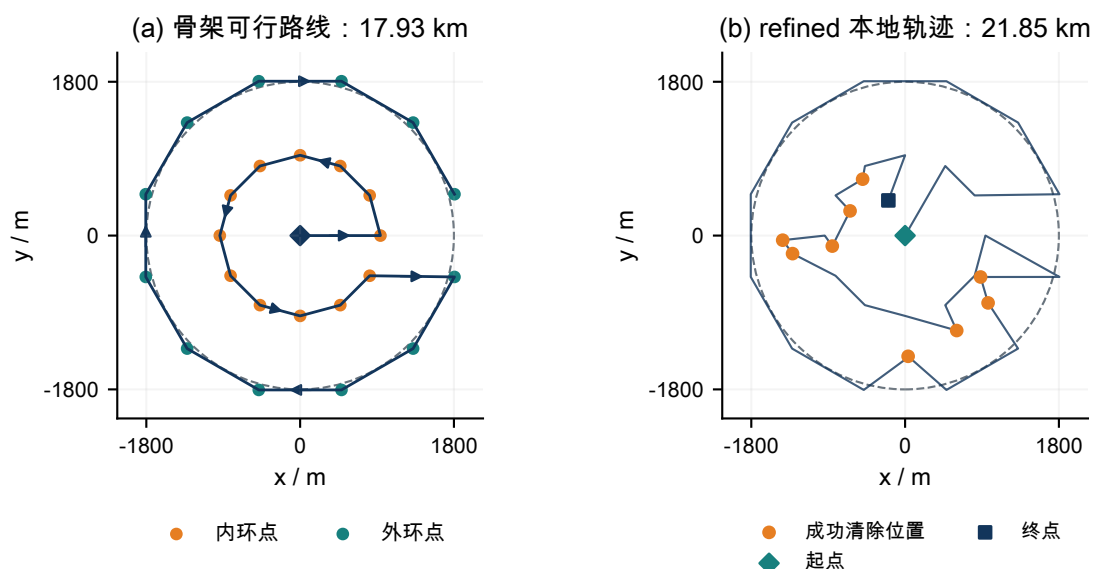


图 10: Q4 双环骨架路线与本地模拟轨迹 (右图为 refined, 非正式测试轨迹)

8.4 路线组合与补测筛选

问题四采用 field 路线组合, 对式(32)取 $\gamma = 1$ 。当首项为不确定区域较大的源任务时, 半径项增加其评分; 当首项是扫描点时, 该项为 0。由此兼顾到达距离和当前定位范围。

候选路线包括原最近插入及有界 2-opt 路线、全局最便宜插入路线, 以及以两个不同首任务起步的近邻路线。对每个不同初解, 最多进行 $\min(n, 12)$ 轮改进, 检查连续区间反转和单点搬移两类操作, 再从全部完整排列中选择评分最小者。

保留原路线使组合评分不高于原控制路线, 但观测和清除终点会在执行后变化, 最终收益由总虚拟时间评价。构建距离矩阵需要 $O(n^2)$ 空间, 有界搜索的保守时间上界为 $O(n^3)$ 。

骨架扫描采用有用补测规则。未知频道仍必须检测; 对已有轨迹的频道, 只有当

$$r_c > 19.9, \quad \|p - z_c\| - r_c \leq 1500 \quad (50)$$

时继续补测。若区域已足够小, 可直接清除; 若整个包围圆都超过接收上限, 继续测向没有价值。跳过已知频道不会形成无源记录, 该频道仍需成功清除。

field 在局部定位前还可进行一次原地预检测。触发条件为: 该频道尚未使用预检测机会, 当前位置未测过该频道, $r_c > 38$, 且 $\|q - z_c\| - r_c \leq 1500$ 。这一次检测位于原有 6 次测向之外, 照常计入检测与换频费用。它有机会在移动前收缩区域, 减少后续偏移距离。

8.5 失信号恢复

定向源在新位置可能落到发射背面。为继续利用最后一次有效测向, 设该测点为 s^+ , 由其指向当前包围圆中心的方向为 e , 法向为 n 。将失败测点写成

$$p^- - s^+ = ae + bn,$$

恢复点取

$$p^{\text{next}} = s^+ + \frac{1}{2}(ae - bn). \quad (51)$$

该操作把横向分量反向, 并将相对 s^+ 的距离缩短一半。新的有效测向出现后, 更新 s^+ 和方向轴; 若仍无信号, 则在 6 次测向预算内继续恢复, 之后转为矩形光学覆盖。

Q4 的无信号信息不用于整圆盘位置排除, 因为发射背面的源仍可能位于 1000 米以内。Q4 也不启用安全近侧清除项, 清除位置沿用基准局部过程。光学搜索按式(36)的圆盘并集覆盖区域, 每个覆盖中心只负责对应小单元; 遍历全部中心后即可覆盖整个位置域, 不要求每个中心单独覆盖整个 P_c 。

8.6 整体执行与有限终止

问题四以 $V = \{0, \dots, 24\}$ 替换问题三的七点索引, 状态转移和停止条件仍用式(40)。若未知频道在全部 25 点都无信号, 与任意方向覆盖结论矛盾的源便不存在; 已发现频道只有收到成功清除反馈才完成。

每次完整骨架扫描结束后，不再返回已经完成的扫描点。每个正常局部任务清除一个新源，最多有 16 个；每个局部任务至多一次预检测和 6 次新测向，随后进入有限光学网格。因此在正常反馈和误差条件下，扫描与定位均能在有限动作内完成。

表 3: 两类问题的主要执行规则

环节	问题三	问题四
发现骨架	原点及 1200 米六点环	原点及 940/1870 米错相双环
任务评分	首项半径权重 $\gamma = 0$	首项半径权重 $\gamma = 1$
已知频道骨架补测	跳过，保留独立定位任务	满足式(50)时补测
无信号处理	field 可用负观测凸外包	保留位置域，回退恢复
局部结束	直接清除或矩形覆盖	直接清除或矩形覆盖
全局停止	全频道完成或 16 源清除	全频道完成或 16 源清除

8.7 算法执行流程

将上述过程整理为图11。机器狗先建立扫描点和 20 个频道的状态记录，再检查是否满足停止条件；若尚有任务，就依据当前位置重排任务。选中扫描点时，完成该点的必要频道检测；选中源任务时，调用局部测向和光学清除过程。两条分支都回到状态更新，不要求机器人返回先前扫描点。

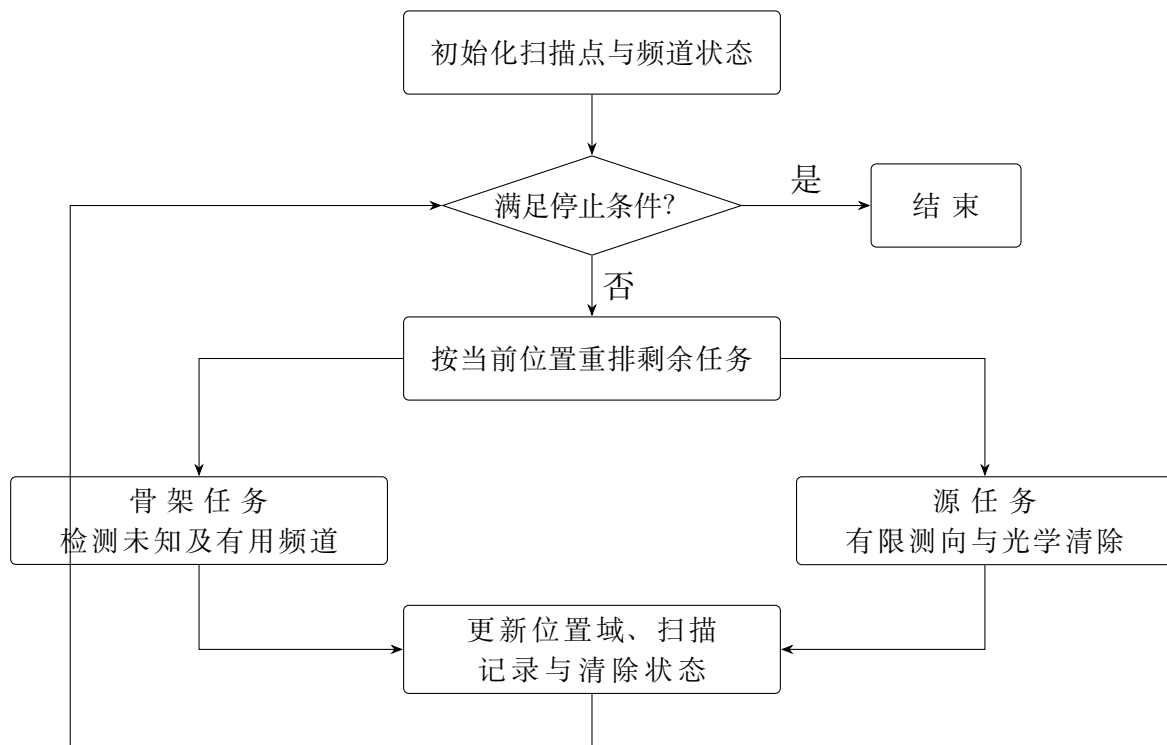


图 11: 扫描与定位交替进行的任务执行流程

光学后备的搜索规模还可以由单次测向范围估计。沿最近一次有效示向轴，位置域的纵向长度不超过约 1500 米，横向宽度不超过 $2 \times 1500 \tan(1.005^\circ) < 53$ 米。转到直径

对齐坐标后，可用长小于 1501 米、宽小于 106 米的矩形包围；按第7.5节的网格边长划分，至多需要

$$N_{\text{cell}} \leq \left\lceil \frac{1501}{\sqrt{2}(19.9 - \varepsilon)} \right\rceil \left\lceil \frac{106}{\sqrt{2}(19.9 - \varepsilon)} \right\rceil = 54 \times 4 = 216 \quad (52)$$

个中心，其中 ε 取实现中的微小向内余量。实际可行域通常经过多次裁剪，所需网格少于此上界。

8.8 算法终止性与时间上界

设接口正常响应，测向误差满足题设条件。问题三、四分别有 7 个和 25 个骨架任务，完成后即从任务集合中删除。对已发现频道，局部过程至多进行 6 次测向，随后直接清除或进入至多 216 个中心的光学覆盖。由于真实源始终位于覆盖区域内，遍历全部中心前必有一次清除成功。因此，每轮调度至少完成一个骨架任务或清除一个新源，已完成任务不再加入；源数又不超过 16，算法最终满足式(40)。

按较大的 25 点骨架计，骨架检测至多 25×20 次；停靠共享只在局部任务结束后进行，至多 16 轮，每轮至多检测 16 个已知频道；局部测向和 field 预检测分别至多 16×6 次和 16 次。由此有

$$M \leq 25 \times 20 + 16 \times 16 + 16 \times 6 + 16 = 868 \leq 1228. \quad (53)$$

下文按 $M \leq 1228$ 统一估计检测费用。每源计入 216 次网格清除、至多 6 次无信号后试清、一次小区域试清，并为其他位置的成功清除留一次余量，则清除调用总数满足

$$C = F + K \leq 16(216 + 6 + 1 + 1) = 3584. \quad (54)$$

时间上界还需计入行程。由前述位置域尺寸，包围圆中心距最近正测点小于 1501 米，横向偏移不超过 120 米，故首个候选测点距该正测点小于 $\sqrt{1501^2 + 120^2} < 1506$ 米。恢复操作只缩短这一距离，因而局部测点距源小于 $1500 + 1506 = 3006$ 米，距原点小于 4806 米。骨架点和已清除源的终点距原点均小于 2620 米，于是每段骨架抵达、局部任务接入和两次测向间的行程可分别放宽为 5240、7500 和 6020 米。圆心清除与光学搜索的接入段各不超过 4520 米；矩形内蛇形路径连同一次循环移位跳跃小于 7570 米。

$$\begin{aligned} L &\leq 25 \times 5240 + 16(7500 + 5 \times 6020 + 2 \times 4520 + 7570) \\ &= 998360 \text{ m}. \end{aligned} \quad (55)$$

每次检测至多附带一次换频，每次清除至多耗时 5 秒。将上述上界代入计时模型，得到

$$\begin{aligned} T &\leq \frac{L}{5} + 6M + 5C \\ &\leq \frac{998360}{5} + 6 \times 1228 + 5 \times 3584 \\ &= 224960 \text{ s} < 100 \times 3600 \text{ s}. \end{aligned} \quad (56)$$

该估计将各阶段的最大费用相加，用于核对 100 小时虚拟时间限制；现实运行期限仍按接口返回的剩余时间执行。

9 数值试验与结果分析

为了比较任务调度和局部机制，在同一合成世界上分别运行各配置。常规世界中，源数在 10 至 16 之间离散均匀取值，频道无放回抽样，位置在半径 1800 米圆域内按面积均匀生成，接收半径在 1000 至 1500 米之间均匀生成。问题三全部为全向源；问题四设置混合源，定向源的方向均匀生成。

误差场采用三种设置：有界双正弦平滑场、位置哈希取 $\pm 1^\circ$ 的端点场、位置哈希在 $[-1^\circ, 1^\circ]$ 中取值的误差场。同一位置的误差固定，读数保留两位小数。上述设置用于构造可重复测试场景，不作为官方生成器的分布估计。

开发阶段每题使用 150 个世界比较候选配置，之后固定参数，在每题 1000 个新世界上进行主比较。Q3 列出 refined、仅路线增强、路线与信息、完整 field 四组；Q4 比较 refined 与 field 两组，主组共 6000 次策略执行。另进行边界情形和替代误差测试，合计 9400 次执行，各组分别统计。

9.1 主组比较结果

表4按逐世界单源时间给出均值、中位数、90% 分位数和样本最大值。主组表中列示的各策略均完成所有真实源清除，并满足频道完成判据。

表 4: 每题 1000 个同世界案例的单源耗时，单位：秒/源

问题	策略	均值	中位数	p_{90}	最大值	全清
Q3	refined	258.99	254.76	305.38	370.28	1000/1000
Q3	仅路线增强	254.07	250.49	301.86	352.51	1000/1000
Q3	路线与信息	251.13	246.70	299.49	340.73	1000/1000
Q3	field	250.17	245.79	298.55	338.79	1000/1000
Q4	refined	542.41	529.08	682.86	825.03	1000/1000
Q4	field	508.60	496.94	642.48	760.05	1000/1000

问题三的 field 均值由 258.99 降至 250.17 秒/源，问题四由 542.41 降至 508.60 秒/源；两题的 p_{90} 和最大值也下降。问题四的均值差更大，与双环访问范围较广、任务间移动更多的特点相符。

为了衡量同一世界内的差异，定义

$$d_j = \tau_j^{\text{refined}} - \tau_j^{\text{field}}. \quad (57)$$

正值表示 field 较快。对配对差的均值作正态近似 [10]，近似 95% 区间采用

$$\bar{d} \pm 1.96 \frac{s_d}{\sqrt{n}}, \quad (58)$$

其中 s_d 为配对差的样本标准差。两题结果见表5。

表 5: 主组配对节省时间与更快案例数

问题	均值差	差值中位数	95% 区间	更快局数	平均降幅
Q3	8.82	6.41	[8.01, 9.62]	827/1000	3.40%
Q4	33.81	33.27	[31.61, 36.01]	843/1000	6.23%

Q3、Q4 的区间下限均大于 0，表明在当前合成分布下 field 的平均单源时间较低；分别有 827 和 843 个世界更快，也说明增强并非每局都占优。

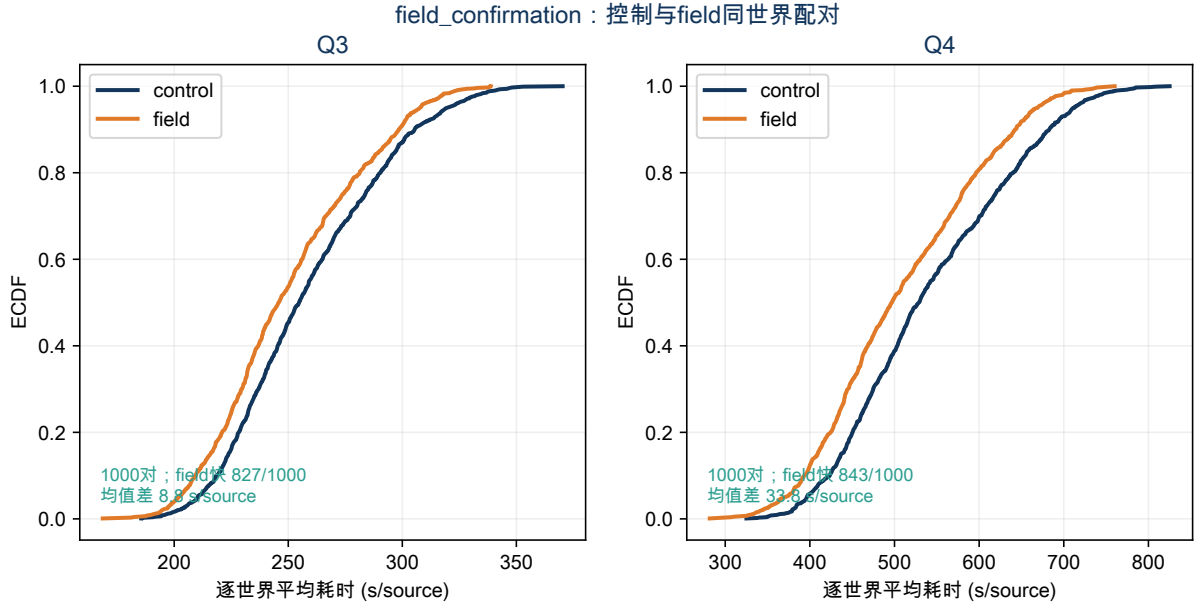


图 12: 主组 refined 与 field 的单源耗时累积分布

图12中曲线在同一累计比例下越靠左，表示相应分位点耗时越短。主组既有整体水平下降，也有尾部改善；二者与表4的均值和 p_{90} 一致。

9.2 机制消融与成本组成

Q3 的主组采用逐步加入机制的配置链。仅更改路线时，平均耗时为 254.07 秒/源；加入预检测及负观测更新后为 251.13 秒/源；再加入安全接近后为 250.17 秒/源。相邻平均节省分别为 4.92、2.94 和 0.96 秒/源。这些增量对应当前配置链，反映机制之间的共同作用。

进一步把每个世界的总时间拆成五项，先分别除以本局成功清除数，再在世界间取平均，得到表6。

表 6: 主组单源费用分解, 单位: 秒/源

问题	策略	移动	检测	换频	失败清除	成功清除	合计
Q3	refined	195.50	48.98	9.32	0.19	5.00	258.99
Q3	field	188.36	47.58	9.06	0.17	5.00	250.17
Q4	refined	383.47	129.62	23.91	0.41	5.00	542.41
Q4	field	350.93	128.53	23.66	0.48	5.00	508.60

由表可见, 移动仍是主要费用。Q3 中 field 将每源移动成本由 195.50 降至 188.36 秒; Q4 由 383.47 降至 350.93 秒。检测和换频费用也略有下降, 说明路线改变会影响后续观测和任务顺序。

Q4 失败清除尝试均值由 1.782 增至 2.089 次/局, 但失败清除的单源费用增幅小于移动节省。因此不能只用失败次数判断策略优劣, 仍需结合完整计时式。成功清除项在全清条件下固定为每源 5 秒。

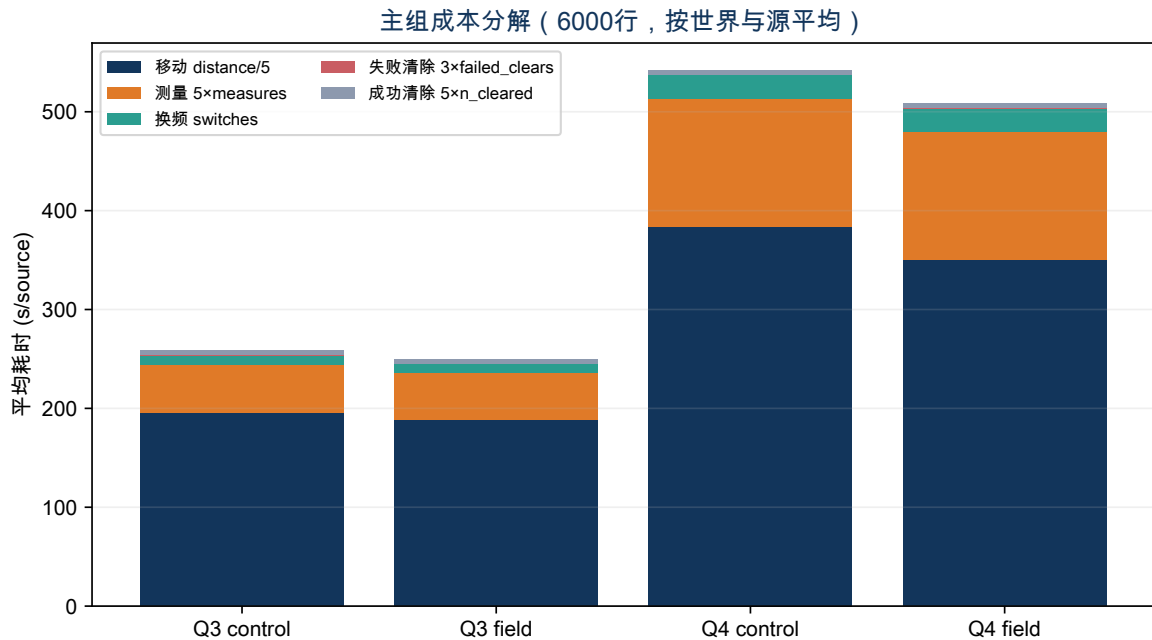


图 13: 主组五项成本在两种策略中的组成

9.3 官方演练结果

旧 refined 策略的官方演练每题各 10 局, 包含逐步请求、返回结果和测试结束后的源数量。结果见表7。

表 7: refined 官方演练汇总, 时间单位: 秒/源

问题	局数	清除源数	逐局等权	汇总指标	移动占比
Q3	10	139/139	237.998	235.823	74.38%
Q4	10	138/138	531.178	519.376	73.20%

两题均完成全部源清除，移动分别占虚拟总时间的 74.38% 和 73.20%。这一费用结构与本地试验中的移动主导现象一致，也是改进任务顺序的直接依据。

补充演练汇总中，Q3 的 field 为 11 局、134 个源，逐局平均 263.550883 秒/源；Q4 为 12 局、166 个源，逐局平均 484.966300 秒/源。这些案例与 refined 演练不同，尤其 Q3 平均源数由 13.9 降至约 12.18，搜索固定费用分摊到每源后的影响随之变化。因此将两组演练作为运行表现观察，不把均值差当作同世界的算法节省。

前文以 refined 作为 Q3 基准、field 作为 Q4 增强方案。以下正式测试的两题均采用 field，按实际执行策略分别列示。

10 正式测试结果

问题三和问题四各进行三次正式测试，均采用 field 策略。表中的 K 为成功清除源数， T 为虚拟总时间；本机运行时间由控制程序测得，与模拟器界面计时分开列示。

表 8: 问题三正式测试结果 (field)

次数	案例编码	K	T /秒	T/K /秒	本机运行/秒
1	UAKK-9823-A2SU-2CNM	14	3136.520	224.037	0.7665
2	5CV9-WP56-W8T6-YMBM	10	2701.627	270.163	0.8526
3	PNKS-Z5H4-TW3T-FDV6	16	3450.360	215.647	0.8749

Q3 三局共成功清除 40 个源，虚拟总时间合计 9288.507 秒。逐局单源时间由 215.647 至 270.163 秒不等；三局等权平均为 236.616 秒/源，汇总指标为 232.213 秒/源。两种均值的差别来自各局成功清除数量不同。

表 9: 问题四正式测试结果 (field)

次数	案例编码	K	T /秒	T/K /秒	本机运行/秒
1	BEVP-4HRX-JUKY-4XKA	15	6119.158	407.944	2.7734
2	BJKN-YUGD-Y66E-KJ43	12	6054.392	504.533	1.9247
3	A6XY-YGC6-WXC8-D2TN	10	6655.016	665.502	2.3622

Q4 三局共成功清除 37 个源，虚拟总时间合计 18828.566 秒。逐局等权平均为 525.993 秒/源，汇总指标为 508.880 秒/源。第三局成功清除数较少，单源时间较高，说明总时间和源数都影响 T/K 。

六局均满足各频道完成清除或无源判断的停止条件。正式测试未提供每局真实源数，故只报告成功清除数、虚拟时间和停止状态，不计算清除率；该结果与第9节的合成配对试验分开统计。

图14并列展示六局正式测试的单源时间，并标出各局成功清除数 K 。虚线分别表示两题的逐局等权均值，图中只比较本次案例的耗时分布，不将不同案例视为配对策略试验。

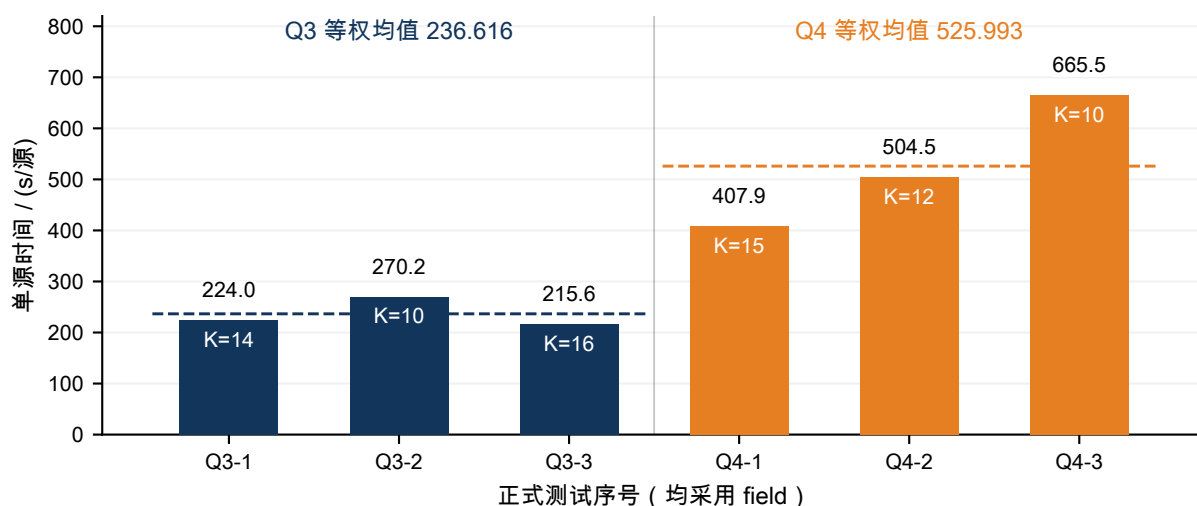


图 14: 六次正式测试的单源时间与成功清除数 (均采用 field)

11 模型检验与灵敏度分析

11.1 定位与停止条件的检验

位置可行域的核心性质是始终包含真实源。对此，使用 2000 组、每组 4 次测向的计算检查角度跨零、边界位置、端点误差及两位小数舍入；最小包围圆另与 1、2、3 个支持点的枚举结果进行 504 组交叉比较。

Q3 负观测更新检查了 3000 次真实源包含情形及边界、退化区域。安全接近在 200 组区域上重新计算候选点到全部顶点的距离。Q4 双环通过三角形边界连接、面积、无交叉及坐标余量核对，与方向覆盖推导相对应。

停止条件检验中，未知频道在全部骨架扫描完成后判断无源，已发现频道在清除成功后完成；拒绝动作不计入位置和扫描记录。再按实际移动、检测、换频及清除次数重算时间，与各步返回的虚拟时刻一致。

11.2 边界位置与不同误差场

设置三类位置情形：全部源接收半径为 1000 米；16 个源位于圆周，Q4 其中 15 个朝外发射；16 个源聚集在距圆心 1650 米附近的 35 米小圆盘中。每类分别使用三种误差场，每组 50 对世界。另对随机位置的两种替代误差各使用 200 对世界。

表 10: Q3 边界与替代误差测试, 时间单位: 秒/源

位置情形	误差场	对数	refined	field	节省 95% 区间	更快
最小接收半径	平滑	50	273.27	261.22	[7.84, 16.26]	38/50
最小接收半径	端点	50	278.63	260.86	[12.25, 23.29]	41/50
最小接收半径	位置哈希	50	275.22	259.63	[10.41, 20.78]	40/50
圆周边界	平滑	50	284.80	281.87	[2.03, 3.84]	44/50
圆周边界	端点	50	285.15	281.29	[2.80, 4.92]	43/50
圆周边界	位置哈希	50	284.70	281.81	[1.91, 3.88]	43/50
聚集源	平滑	50	105.59	105.02	[0.34, 0.79]	42/50
聚集源	端点	50	105.96	105.29	[0.40, 0.94]	44/50
聚集源	位置哈希	50	105.88	105.23	[0.13, 1.16]	32/50
随机位置	端点	200	260.50	250.14	[8.44, 12.29]	167/200
随机位置	位置哈希	200	258.70	248.75	[8.07, 11.83]	157/200

Q3 全部组完成全清。最小接收半径组的平均节省较大, 说明在接收余量较小时, 观测与路线的配合仍有效。圆周和聚集组的节省较小: 前者的固定扫描位置作用更突出, 后者则减少了任务间远距离移动, 留给路线改进的空间相应缩小。

表 11: Q4 边界与替代误差测试, 时间单位: 秒/源

位置情形	误差场	对数	refined	field	节省 95% 区间	更快
最小接收半径	平滑	50	563.55	527.52	[26.32, 45.74]	42/50
最小接收半径	端点	50	562.47	522.74	[29.89, 49.57]	43/50
最小接收半径	位置哈希	50	558.90	525.95	[23.69, 42.20]	41/50
圆周边界	平滑	50	664.91	625.95	[29.03, 48.89]	42/50
圆周边界	端点	50	668.95	632.25	[26.02, 47.39]	41/50
圆周边界	位置哈希	50	663.47	625.63	[25.90, 49.78]	39/50
聚集源	平滑	50	350.77	333.04	[-9.37, 44.84]	32/50
聚集源	端点	50	360.36	332.49	[4.02, 51.73]	35/50
聚集源	位置哈希	50	355.06	318.77	[10.95, 61.63]	35/50
随机位置	端点	200	536.35	500.09	[31.56, 40.95]	173/200
随机位置	位置哈希	200	539.87	502.81	[32.16, 41.96]	172/200

Q4 全部组也完成全清。圆周朝外发射组检验了外环点的作用, 在三类误差下均保持有效搜索。聚集平滑误差组的节省区间跨越 0, 当前 50 对样本未证实该组均值改善; 其余结果结合位置结构分别解释, 不作为每个场景均更快的结论。

11.3 横向偏移比例的影响

局部测向式(33)中, 包围半径前的比例决定候选点离中心的距离。比例过小, 交会改善有限; 比例过大, 移动和失信号风险增加。对 refined 取比例 0.12、0.22 和 0.40, 在每题 50 个相同世界上比较, 结果见表12。

表 12: 横向偏移比例的灵敏度试验 (refined), 时间单位: 秒/源

问题	偏移比例	案例	均值	p_{90}	失败清除/局	全清
Q3	0.12	50	254.26	306.06	0.84	50/50
Q3	0.22	50	254.78	304.77	0.68	50/50
Q3	0.4	50	255.87	306.02	0.66	50/50
Q4	0.12	50	526.15	648.72	2.24	50/50
Q4	0.22	50	526.46	648.80	2.18	50/50
Q4	0.4	50	525.74	646.00	2.00	50/50

三种比例下各组均完成全清。均值的变化小于 2 秒/源, 尾部耗时和失败尝试数也有变化, 但没有随比例增大而一致改善的趋势。因此采用 0.22 作为既定参数。这组试验与 field 主比较使用不同案例, 结果分别统计。

11.4 局部搜索与整体耗时的关系

最近插入、2-opt 和单点搬移均对当前有限任务排列进行改进。小规模静态路线可以穷举全部排列作为对照, 而在线任务会随着示向结果、域收缩和清除终点发生变化。两者的区别在于: 静态路线只计算固定任务坐标, 完整任务还包含定位与操作费用。

因此, 判断算法效果时, 一方面检查候选排列保留全部任务, 另一方面比较逐世界的 T/K 、检测次数、换频次数和失败清除次数。表6中 Q4 失败尝试上升而总时间下降, 正体现了局部指标与总体费用的差别。

12 模型评价与结论

12.1 模型的优点

1. **定位与清除衔接明确。**用交会区域表示测向误差, 再用最小包围圆判断一次清除, 避免将方向读数直接当作精确坐标。
2. **搜索过程具有覆盖依据。**七点环解决全向距离覆盖, 25 点双环同时处理源位置和发射方向, 圆周朝外发射情形包含在构造内。
3. **任务顺序随观测调整。**扫描点和定位任务共同排序, 清除后从新位置继续选择任务, 减少固定往返带来的行程。
4. **完成条件可逐频道检查。**成功清除、已发现未清除和完成覆盖后的无源状态分别记录, 源数量未知时仍有明确的停止方式。

12.2 模型的不足与适用范围

固定骨架便于证明覆盖, 但没有充分利用所有已知源位置调整后续检测点; Q3 负观测取凸外包后会填回部分已排除区域, 损失一些信息; 路线采用有限局部改进, 对完整自适应任务没有全局最优保证。

数值试验的源位置和误差场是人为设置的合成分布，官方案例中的运行表现还受源分布、源数和发射方向影响。当前模型适用于静止源、已知接收上下界和有界示向误差条件。推广到其他区域尺度时，应同时调整检测骨架并重新核对覆盖距离，而不能仅按比例缩放所有经验参数。

12.3 结论

本文以交会可行域、固定覆盖骨架和联合调度完成四问求解。问题一给出区域分类和顶点直径算法，并用三角形反例说明直径圆不能总是覆盖；问题二得到三圆交安全候选域，以离散交会计算选择第二检测点；问题三和问题四分别采用七点环与 25 点双环，结合有限测向和矩形光学覆盖完成清除。

主组合成配对试验中，field 相对 refined 的平均单源时间分别降低 3.40% 和 6.23%，全部主组和列示边界组均全清。六次正式测试采用 field，Q3、Q4 三局等权平均分别为 236.616 和 525.993 秒/源，均按频道完成条件结束。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试等辅助工作，详细使用情况见支撑材料。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题: 无线电干扰源的快速自动定位与清除, 以及附件 1、附件 2[Z]. 2026.
- [2] Welzl E. Smallest enclosing disks (balls and ellipsoids)[C]//Maurer H, ed. New Results and New Trends in Computer Science. Lecture Notes in Computer Science, 555. Berlin, Heidelberg: Springer, 1991: 359–370. DOI: 10.1007/BFb0038202.
- [3] Jung H. Ueber die kleinste Kugel, die eine räumliche Figur einschliesst[J]. Journal für die reine und angewandte Mathematik, 1901, 123: 241–257. DOI: 10.1515/crll.1901.123.241.
- [4] Sutherland I E, Hodgman G W. Reentrant polygon clipping[J]. Communications of the ACM, 1974, 17(1): 32–42. DOI: 10.1145/360767.360802.
- [5] 秦明峰, 郝青儒, 范广伟. 运动单站无源定位性能分析 [J]. 无线电工程, 2014, 44(4): 50–53.
- [6] 李亚军. 机载塔康无源定位方法研究 [J]. 无线电工程, 2019, 49(3): 229–233.
- [7] Rosenkrantz D J, Stearns R E, Lewis P M II. An analysis of several heuristics for the traveling salesman problem[J]. SIAM Journal on Computing, 1977, 6(3): 563–581. DOI: 10.1137/0206041.
- [8] Croes G A. A method for solving traveling-salesman problems[J]. Operations Research, 1958, 6(6): 791–812. DOI: 10.1287/opre.6.6.791.
- [9] Galceran E, Carreras M. A survey on coverage path planning for robotics[J]. Robotics and Autonomous Systems, 2013, 61(12): 1258–1276. DOI: 10.1016/j.robot.2013.09.004.
- [10] Wasserman L. All of Statistics: A Concise Course in Statistical Inference[M]. New York: Springer, 2004. DOI: 10.1007/978-0-387-21736-9.

A 附录：程序与支撑材料

A.1 核心运行与验证代码

Listing 1: 规划器与统一运行入口

```
1
2
3 from dataclasses import dataclass, field
4 from math import dist, hypot
5 from time import monotonic
6
7 from submission.支撑材料.active_policy_candidate import choose_probe
8 from submission.支撑材料.client import AmbiguousActionError, DeadlineExceeded
9 from submission.支撑材料.coverage import directional_waypoints, omni_waypoints
10 from submission.支撑材料.geometry import clip_bearing, enclosing_circle, initial_polygon, optical_cover
11 from submission.支撑材料.route_policy_candidate import optimize_waypoints
12 from submission.支撑材料.joint_dispatch_candidate import choose_task
13 from submission.支撑材料.optical_policy_candidate import certified_optical_cover
14 from submission.支撑材料.polar_coverage_candidate import polar_waypoints
15
16
17 class SessionTerminated(RuntimeError):
18     pass
19
20
21 class InconsistentObservations(RuntimeError):
22     pass
23
24
25 @dataclass
26 class Track:
27     polygon: list = field(default_factory=initial_polygon)
28     readings: list = field(default_factory=list)
29     observations: list = field(default_factory=list)
30     failures: list = field(default_factory=list)
31
32
33 class Planner:
34
35
36     def __init__(
37         self,
38         environment,
39         problem=3,
40         strategy="refined",
41         spacing=990.0,
42         probe_scale=0.22,
43         max_probes=6,
44         loss_recovery=True,
45         anchor_policy=None,
46         probe_policy="fixed",
47         ring_radius=None,
48         coverage_policy=None,
49         optical_policy="rectangle",
50         dispatch_policy=None,
51         scan_policy=None,
52     ):
53         if problem not in (3, 4) or strategy not in (
54             "baseline",
55             "batched",
56             "integrated",
57             "enhanced",
58             "refined",
```



```

59     ):
60         raise ValueError("Unsupported problem or strategy")
61     if anchor_policy is None:
62         anchor_policy = "rolling" if strategy in ("enhanced", "refined") and problem == 4 else "nearest"
63     if ring_radius is None and strategy in ("enhanced", "refined") and problem == 3:
64         ring_radius = 1200.0
65     if coverage_policy is None:
66         coverage_policy = "compact25" if strategy == "refined" and problem == 4 else "lattice"
67     if dispatch_policy is None:
68         dispatch_policy = ("center" if problem == 3 else "guarded") if strategy == "refined" else "separate"
69
70     if scan_policy is None:
71         scan_policy = ("unknown" if problem == 3 else "useful") if strategy == "refined" else "all"
72     if anchor_policy not in ("nearest", "two_opt", "rolling"):
73         raise ValueError("Unsupported anchor policy")
74     if probe_policy not in ("fixed", "adaptive"):
75         raise ValueError("Unsupported probe policy")
76     if not 900 <= spacing <= 999:
77         raise ValueError("Directional lattice spacing must be in [900, 999] m")
78     if not 0 < probe_scale <= 1 or max_probes < 1:
79         raise ValueError("Invalid localization policy parameters")
80     if coverage_policy not in ("lattice", "seed25", "compact25", "fast25"):
81         raise ValueError("Unsupported discovery coverage policy")
82     if optical_policy not in ("rectangle", "slabs"):
83         raise ValueError("Unsupported optical covering policy")
84     if dispatch_policy not in ("separate", "center", "guarded"):
85         raise ValueError("Unsupported task dispatch policy")
86     if scan_policy not in ("all", "unknown", "useful"):
87         raise ValueError("Unsupported known-channel scan policy")
88     self.scan_policy = scan_policy
89     self.coverage_policy = coverage_policy if problem == 4 else "ring"
90     self.optical_policy = optical_policy
91     self.dispatch_policy = dispatch_policy
92     self.environment = environment
93     self.problem = problem
94     self.strategy = strategy
95     self.waypoints = (
96         omni_waypoints(ring_radius) if problem == 3 else
97         directional_waypoints(spacing) if coverage_policy == "lattice" else
98         polar_waypoints(coverage_policy)
99     )
100     self.anchor_policy = anchor_policy
101     self.anchor_order = optimize_waypoints(self.waypoints) if anchor_policy == "two_opt" else []
102     self.anchor_rank = {index: rank for rank, index in enumerate(self.anchor_order)}
103     self.pending = set(range(len(self.waypoints)))
104     self.visited = []
105     self.tracks = {}
106     self.cleared = set()
107     self.scan_records = {ch: set() for ch in range(1, 21)}
108     self.position = (0.0, 0.0)
109     self.radio_channel = 1
110     self.virtual_time = 0.0
111     self.deadline = float("inf")
112     self.max_virtual_duration = float("inf")
113     self.terminal_reason = None
114     self.observation_points = {ch: [] for ch in range(1, 21)}
115     self.channel_state = {ch: "unresolved" for ch in range(1, 21)}
116     self.probe_scale = probe_scale
117     self.max_probes = max_probes
118     self.loss_recovery = bool(loss_recovery) and problem == 4
119     self.fallbacks = 0
120     self.probe_policy = probe_policy
121     self.adaptive_probes = 0
122     self.termination = None
123
124     def action(self, path, point=None, channel=None):

```

```

124     if self.terminal_reason is not None:
125         raise SessionTerminated(self.terminal_reason)
126     if monotonic() >= self.deadline - 2.0 and path != "/exit":
127         raise TimeoutError("Insufficient real time: completeness not certified")
128     response = self.environment.act(path, position=point, channel=channel)
129     if response.get("accepted") is not True:
130         raise RuntimeError("Rejected action; cached state was not advanced")
131     self.virtual_time = float(response["virtual_time_s"])
132     if path == "/enter":
133         self.max_virtual_duration = float(response.get("max_virtual_duration_s", float("inf")))
134     if point is not None:
135         self.position = tuple(point)
136     if path == "/measure":
137         self.radio_channel = channel
138     if path in ("/measure", "/clear") and self.virtual_time >= self.max_virtual_duration:
139         self.terminal_reason = "virtual_timeout"
140     return response
141
142 def clear(self, channel, point, guaranteed=False):
143     result = self.action("/clear", point, channel)["clear_result"]
144     if result == "success":
145         self.cleared.add(channel)
146         self.channel_state[channel] = "cleared"
147         self.tracks.pop(channel, None)
148         return True
149     if result != "no_target_in_range":
150         raise RuntimeError(f"Unknown clear result: {result}")
151     if guaranteed:
152         raise InconsistentObservations("Certified radius-20 clearing failed")
153     if channel in self.tracks:
154         self.tracks[channel].failures.append(tuple(point))
155     return False
156
157 def measure(self, channel, point):
158     result = self.action("/measure", point, channel)
159     status = result["measure_result"]
160     self.observation_points[channel].append(tuple(point))
161     track = self.tracks.get(channel)
162     if track is None and status == "direction":
163         track = Track(observations=self.observation_points[channel])
164         self.tracks[channel] = track
165     if track is not None:
166         track.observations = self.observation_points[channel]
167     if status == "near":
168         self.channel_state[channel] = "detected"
169         if self.terminal_reason is None:
170             self.clear(channel, point, guaranteed=True)
171     elif status == "direction":
172         self.channel_state[channel] = "detected"
173         track.polygon = clip_bearing(track.polygon, point, result["svd_deg"])
174         if not track.polygon:
175             raise InconsistentObservations(
176                 f"Empty feasible region on channel {channel}"
177             )
178         track.readings.append((tuple(point), float(result["svd_deg"])))
179     elif status != "no_signal":
180         raise RuntimeError(f"Unknown measure result: {status}")
181
182     return status
183
184
185 def scan_anchor(self, index):
186     point = self.waypoints[index]
187     channels = [ch for ch in range(1, 21) if ch not in self.cleared]
188     channels.sort(key=lambda ch: (ch != self.radio_channel, ch))
189     for channel in channels:

```

```

190         if channel in self.cleared:
191             continue
192         if channel in self.tracks:
193             if self.scan_policy == "unknown":
194                 continue
195             if self.scan_policy == "useful":
196                 center, radius = enclosing_circle(self.tracks[channel].polygon)
197                 if radius <= 19.9 or dist(point, center) - radius > 1500:
198                     continue
199         self.measure(channel, point)
200         self.scan_records[channel].add(index)
201         if len(self.cleared) == 16:
202             self.termination = "known_count_upper_bound_16"
203             break
204         if self.terminal_reason is not None:
205             break
206         if self.strategy == "baseline" and channel in self.tracks:
207             self.localize(channel)
208         if len(self.cleared) == 16:
209             self.termination = "known_count_upper_bound_16"
210             break
211         if self.terminal_reason is not None:
212             break
213         self.pending.discard(index)
214         if index not in self.visited:
215             self.visited.append(index)
216
217     def optical_finish(self, channel):
218         self.fallbacks += 1
219         covering = certified_optical_cover if self.optical_policy == "slabs" else optical_cover
220         cover = covering(self.tracks[channel].polygon, radius=19.9)
221         if not cover:
222             raise InconsistentObservations("An empty region cannot certify clearing")
223         start = min(range(len(cover)), key=lambda k: dist(self.position, cover[k]))
224
225         order = cover[start:] + cover[:start]
226         for point in order:
227             if self.clear(channel, point):
228                 return
229             if self.terminal_reason is not None:
230                 return
231         raise InconsistentObservations("Exhausted a certified optical cover without clearing")
232
233     def localize(self, channel):
234         losses = 0
235         failed_probe = None
236         for probe_index in range(self.max_probes):
237             if channel in self.cleared or self.terminal_reason is not None:
238                 return
239             track = self.tracks[channel]
240             center, radius = enclosing_circle(track.polygon)
241             if max(dist(self.position, p) for p in track.polygon) <= 19.9:
242                 self.clear(channel, self.position, guaranteed=True)
243                 return
244             if radius <= 19.9:
245                 self.clear(channel, center, guaranteed=True)
246                 return
247
248         if radius <= 38 and center not in track.failures:
249             if self.clear(channel, center):
250                 return
251             if self.terminal_reason is not None:
252                 return
253             self.optical_finish(channel)

```

```

256         return
257     previous = track.readings[-1][0]
258     dx, dy = center[0] - previous[0], center[1] - previous[1]
259     length = hypot(dx, dy)
260     if length < 1e-8:
261         self.optical_finish(channel)
262         return
263     unit, normal = (dx / length, dy / length), (-dy / length, dx / length)
264     if failed_probe is not None and self.loss_recovery:
265         vector = (failed_probe[0] - previous[0], failed_probe[1] - previous[1])
266         axial = vector[0] * unit[0] + vector[1] * unit[1]
267         lateral = vector[0] * normal[0] + vector[1] * normal[1]
268         point = (
269             previous[0] + 0.5 * (axial * unit[0] - lateral * normal[0]),
270             previous[1] + 0.5 * (axial * unit[1] - lateral * normal[1]),
271         )
272         if any(point == p for p in track.observations) or point in track.failures:
273             self.optical_finish(channel)
274             return
275     else:
276         point = None
277         if self.probe_policy == "adaptive":
278             point = choose_probe(
279                 track.polygon,
280                 track.readings,
281                 self.position,
282                 self.problem,
283                 probes_remaining=self.max_probes - probe_index,
284                 rejected_points=track.failures + track.observations,
285                 probe_scale=self.probe_scale,
286             )
287             if point is not None:
288                 self.adaptive_probes += 1
289             if point is None:
290                 offset = min(120.0, max(25.0, self.probe_scale * radius))
291                 candidates = [
292                     (
293                         center[0] + sign * offset * normal[0],
294                         center[1] + sign * offset * normal[1],
295                     )
296                     for sign in (-1, 1)
297                 ]
298                 fresh = [
299                     p
300                     for p in candidates
301                     if p not in track.observations
302                     and all(p != s for s in track.failures)
303                 ]
304                 if not fresh:
305                     self.optical_finish(channel)
306                     return
307                 point = min(fresh, key=lambda p: dist(self.position, p))
308             status = self.measure(channel, point)
309             if channel in self.cleared or self.terminal_reason is not None:
310                 return
311             if status == "no_signal":
312                 losses += 1
313                 if self.clear(channel, point):
314                     return
315                 if self.terminal_reason is not None:
316                     return
317             failed_probe = point
318             if losses >= 2 and not self.loss_recovery:
319                 self.optical_finish(channel)
320                 return
321         else:

```

```

322         failed_probe = None
323     if channel not in self.cleared and self.terminal_reason is None:
324         self.optical_finish(channel)
325
326 def share_current_stop(self):
327
328     if self.strategy not in ("integrated", "enhanced", "refined") or self.terminal_reason is not None or
329     len(self.cleared) == 16:
330         return
331     for channel in list(self.tracks):
332         if channel in self.cleared:
333             continue
334         track = self.tracks[channel]
335         center, radius = enclosing_circle(track.polygon)
336         if radius <= 19.9:
337             continue
338         if any(self.position == p for p in track.observations):
339             continue
340         if dist(self.position, center) - radius > 1500:
341             continue
342         self.measure(channel, self.position)
343         if self.terminal_reason is not None:
344             return
345
346 def next_anchor(self):
347     if self.anchor_policy == "two_opt":
348         return min(self.pending, key=lambda i: self.anchor_rank[i])
349     if self.anchor_policy == "rolling":
350         indices = sorted(self.pending)
351         order = optimize_waypoints([self.waypoints[i] for i in indices], self.position)
352         return indices[order[0]]
353     return min(self.pending, key=lambda i: (dist(self.position, self.waypoints[i]), i))
354 def _result(self):
355     complete = len(self.cleared) == 16
356     if not complete:
357         for channel in range(1, 21):
358             if channel in self.cleared:
359                 self.channel_state[channel] = "cleared"
360             elif (self.channel_state[channel] != "detected"
361                   and len(self.scan_records[channel]) == len(self.waypoints)):
362                 self.channel_state[channel] = "absent-certified"
363             else:
364                 complete = False
365                 break
366     else:
367         complete = True
368     if self.terminal_reason is not None:
369         self.termination = ("complete_after_" if complete else "incomplete_") + self.terminal_reason
370     return {
371         "problem": self.problem,
372         "strategy": self.strategy,
373         "cleared_channels": sorted(self.cleared),
374         "n_cleared": len(self.cleared),
375         "virtual_time_s": self.virtual_time,
376         "average_time_s": self.virtual_time / len(self.cleared)
377         if self.cleared else None,
378         "probe_policy": self.probe_policy,
379         "probe_scale": self.probe_scale,
380         "anchor_policy": self.anchor_policy,
381         "ring_radius_m": hypot(*self.waypoints[1]) if self.problem == 3 else None,
382         "loss_recovery": self.loss_recovery,
383         "adaptive_probes": self.adaptive_probes,
384         "channel_certificates": dict(self.channel_state),
385         "certificate_complete": complete,
386         "terminal_reason": self.terminal_reason,

```

```

387         "visited_anchors": len(self.visited),
388         "total_anchors": len(self.waypoints),
389         "optical_fallbacks": self.fallbacks,
390         "termination": self.termination,
391         "coverage_policy": self.coverage_policy,
392         "optical_policy": self.optical_policy,
393         "dispatch_policy": self.dispatch_policy,
394         "scan_policy": self.scan_policy,
395     }
396
397     def run(self):
398         try:
399             return self._run_body()
400         except (TimeoutError, ConnectionError, AmbiguousActionError,
401                 DeadlineExceeded, SessionTerminated) as exc:
402             self.terminal_reason = self.terminal_reason or type(exc).__name__
403             self.termination = self.termination or "incomplete_" + type(exc).__name__
404             return self._result()
405
406     def select_task(self):
407         return choose_task(
408             self.position,
409             {i: self.waypoints[i] for i in sorted(self.pending)},
410             {ch: enclosing_circle(track.polygon) for ch, track in self.tracks.items()},
411             choice=self.dispatch_policy,
412         )
413
414     def _run_body(self):
415         entry_started = monotonic()
416         entered = self.action("/enter")
417         self.deadline = entry_started + entered["remaining_real_duration_s"]
418         self.scan_anchor(0)
419         while (self.pending or self.tracks) and self.terminal_reason is None:
420             if len(self.cleared) == 16:
421                 self.termination = "known_count_upper_bound_16"
422                 break
423             if self.strategy == "baseline":
424                 if self.tracks:
425                     self.localize(next(iter(self.tracks)))
426                 elif self.pending:
427                     self.scan_anchor(self.next_anchor())
428                 continue
429             if self.dispatch_policy != "separate":
430                 kind, index = self.select_task()
431                 if kind == "target":
432                     self.localize(index)
433                     self.share_current_stop()
434                 else:
435                     self.scan_anchor(index)
436                 continue
437             targets = [
438                 (dist(self.position, enclosing_circle(track.polygon)[0]), ch)
439                 for ch, track in self.tracks.items()
440             ]
441             anchors = [
442                 (
443                     dist(self.position, self.waypoints[i])
444                     + 30.0 * (20 - len(self.cleared)),
445                     i,
446                 )
447                 for i in ([self.next_anchor()] if self.pending and self.anchor_policy == "rolling" else self.
448                 pending)
449             ]
450
451             if targets and (not anchors or min(targets)[0] <= min(anchors)[0]):

```

```

452         self.localize(min(targets)[1])
453         self.share_current_stop()
454     else:
455         self.scan_anchor(self.next_anchor())
456     if self.termination is None:
457         if self.terminal_reason is None:
458             for channel in range(1, 21):
459                 if channel in self.cleared:
460                     self.channel_state[channel] = "cleared"
461                 elif (self.channel_state[channel] != "detected"
462                       and len(self.scan_records[channel]) == len(self.waypoints)):
463                     self.channel_state[channel] = "absent-certified"
464                 else:
465                     raise RuntimeError("Incomplete per-channel discovery coverage")
466             self.termination = "all_channels_cleared_or_coverage_certified_absent"
467         else:
468             current = self._result()
469             if current["certificate_complete"]:
470                 self.termination = "complete_after_" + self.terminal_reason
471             else:
472                 self.termination = "incomplete_" + self.terminal_reason
473     if self.terminal_reason is None:
474         self.action("/exit")
475     return self._result()

```

Listing 2: 本地环境与模拟器接口

```

1
2
3 from __future__ import annotations
4
5 import hashlib
6 import math
7 import random
8 import struct
9 import threading
10 import time
11 from dataclasses import dataclass
12
13 Point = tuple[float, float]
14 SCENARIOS = ("random", "minimum_radius", "outward_boundary", "clustered")
15 ERROR_MODES = ("smooth", "extreme", "iid_location")
16
17
18 def _action_args(path: str, position: Point | None, channel: int | None):
19
20     if path not in ("/enter", "/exit", "/measure", "/clear"):
21         raise ValueError("Unknown action path (paths must be exact)")
22     if path in ("/enter", "/exit"):
23         if position is not None or channel is not None:
24             raise ValueError("enter/exit do not accept position or channel")
25         return None, None
26     if not isinstance(position, (tuple, list)) or len(position) != 2:
27         raise ValueError("position must contain exactly two finite coordinates")
28     if any(isinstance(v, bool) or not isinstance(v, (int, float)) for v in position):
29         raise ValueError("Coordinates must be JSON numbers")
30     try:
31         point = (float(position[0]), float(position[1]))
32     except OverflowError as exc:
33         raise ValueError("Coordinate outside protocol range") from exc
34     if any(not math.isfinite(v) or abs(v) > 2_000_000 for v in point):
35         raise ValueError("Coordinates must be finite and within +/-2000000m")
36     if (
37         isinstance(channel, bool)
38         or not isinstance(channel, (int, float))

```

```

39         or not 1 <= channel <= 20
40         or int(channel) != channel
41     ):
42         raise ValueError("channel must be an integer in 1..20")
43     return point, int(channel)
44
45
46 @dataclass(slots=True)
47 class _Source:
48     position: Point
49     radius: float
50     normal: Point | None
51     phase: float
52     cleared: bool = False
53
54
55 class LocalSimulator:
56
57
58     def __init__(
59         self,
60         seed: int,
61         problem: int = 3,
62         error_mode: str = "smooth",
63         scenario: str = "random",
64     ):
65         if isinstance(seed, bool) or not isinstance(seed, int):
66             raise TypeError("seed must be an integer")
67         if problem not in (3, 4):
68             raise ValueError("problem must be 3 or 4")
69         if error_mode not in ERROR_MODES or scenario not in SCENARIOS:
70             raise ValueError(f"Use scenarios {SCENARIOS} and modes {ERROR_MODES}")
71         self._seed, self._problem = seed, problem
72         self._error_mode, self._scenario = error_mode, scenario
73         self._lock = threading.Lock()
74         self._status = "ready"
75         self._sources: dict[int, _Source] = {}
76         self._position: Point = (0.0, 0.0)
77         self._radio_channel = 1
78         self._virtual_us = 0
79         self._distance = 0.0
80         self._measures = self._switches = self._clear_attempts = self._failed_clears = 0
81         self._started: float | None = None
82         self._finished: float | None = None
83         self._exit_reason: str | None = None
84         self.log: list[dict] = []
85
86     def _generate(self):
87         rng = random.Random(self._seed)
88         n = (
89             16
90             if self._scenario in ("outward_boundary", "clustered")
91             else rng.randint(10, 16)
92         )
93         channels = rng.sample(range(1, 21), n)
94         if self._problem == 3:
95             directional = set()
96         elif self._scenario == "outward_boundary":
97             directional = set(channels[:-1])
98         else:
99             directional = set(rng.sample(channels, rng.randint(1, n - 1)))
100         rotation = rng.uniform(0, math.tau)
101         center = (1650 * math.cos(rotation), 1650 * math.sin(rotation))
102         sources = {}
103         for i, channel in enumerate(channels):
104             theta = rng.uniform(0, math.tau)

```



```

105         if self._scenario == "outward_boundary":
106             theta = rotation + math.tau * i / n
107             position = (1800 * math.cos(theta), 1800 * math.sin(theta))
108         elif self._scenario == "clustered":
109             r = 35 * math.sqrt(rng.random())
110             position = (
111                 center[0] + r * math.cos(theta),
112                 center[1] + r * math.sin(theta),
113             )
114         else:
115             r = 1800 * math.sqrt(rng.random())
116             position = (r * math.cos(theta), r * math.sin(theta))
117         radius = rng.uniform(1000, 1500) if self._scenario == "random" else 1000.0
118         orientation = (
119             theta
120             if self._scenario == "outward_boundary"
121             else rng.uniform(0, math.tau)
122         )
123         normal = (
124             (math.cos(orientation), math.sin(orientation))
125             if channel in directional
126             else None
127         )
128         sources[channel] = _Source(
129             position, radius, normal, rng.uniform(0, math.tau)
130         )
131     self._sources = sources
132
133     def _bearing_error(self, point: Point, channel: int, source: _Source) -> float:
134         x, y = point
135         if self._error_mode == "smooth":
136             return 0.6 * math.sin(x / 173 + y / 251 + source.phase) + 0.4 * math.sin(
137                 x / 419 - y / 137 + 2 * source.phase
138             )
139
140         packed = struct.pack("!dB", x or 0.0, y or 0.0, channel)
141         salt = str(self._seed).encode("ascii") + b":"
142         value = int.from_bytes(
143             hashlib.blake2b(salt + packed, digest_size=8).digest(), "big"
144         )
145         if self._error_mode == "extreme":
146             return 1.0 if value & 1 else -1.0
147         return 2 * (value / ((1 << 64) - 1)) - 1
148
149     def _visible(self, source: _Source, point: Point) -> bool:
150         dx, dy = point[0] - source.position[0], point[1] - source.position[1]
151         if math.hypot(dx, dy) > source.radius:
152             return False
153         if source.normal is None:
154             return True
155         a, b = dx * source.normal[0], dy * source.normal[1]
156
157         return a + b >= -8 * math.ulp(max(abs(a), abs(b), 1.0))
158
159     def _response(self, accepted: bool = True, **fields) -> dict:
160         return {
161             "accepted": accepted,
162             "real_timestamp_ms": int(time.time() * 1000),
163             "virtual_time_s": self._virtual_us / 1_000_000 if accepted else 0,
164             **fields,
165         }
166
167     def _finish(self, reason: str):
168         self._status = "exited"
169         self._finished = time.monotonic()
170

```

```

171         self._exit_reason = reason
172
173     def act(
174         self, path: str, position: Point | None = None, channel: int | None = None
175     ) -> dict:
176         position, channel = _action_args(path, position, channel)
177         with self._lock:
178             if self._status == "exited":
179                 raise ConnectionError(
180                     "Local session interface closed after exit/timeout"
181                 )
182             if self._started is not None and time.monotonic() >= self._started + 1200:
183                 self._finish("real_timeout")
184                 raise ConnectionError("Local session reached its real deadline")
185             request = {"path": path}
186             if position is not None:
187                 request.update(
188                     position={"x": position[0], "y": position[1]}, channel=channel
189                 )
190             before = self._virtual_us
191             radio_before = self._radio_channel
192             distance = 0.0
193             switched = False
194             action_seconds = 0
195             if path == "/enter":
196                 if self._status != "ready":
197                     response = self._response(False)
198                 else:
199                     self._generate()
200                     self._position, self._radio_channel = (0.0, 0.0), 1
201                     self._virtual_us = 0
202                     self._started = time.monotonic()
203                     self._status = "active"
204                     response = self._response(
205                         max_virtual_duration_s=360000,
206                         max_real_duration_s=1200,
207                         remaining_real_duration_s=1200,
208                     )
209             elif self._status != "active":
210                 response = self._response(False)
211             elif path == "/exit":
212                 self._finish("user_exit")
213                 response = self._response(exit_reason="user_exit")
214             else:
215                 distance = math.dist(self._position, position)
216                 source = self._sources.get(channel)
217                 live = source is not None and not source.cleared
218                 if path == "/measure":
219                     switched = channel != self._radio_channel
220                     self._radio_channel = channel
221                     self._switches += int(switched)
222                     self._measures += 1
223                     action_seconds = 5
224                     fields = {"measure_result": "no_signal"}
225                     if live and self._visible(source, position):
226                         if math.dist(position, source.position) <= 5:
227                             fields = {"measure_result": "near"}
228                         else:
229                             angle = math.degrees(
230                                 math.atan2(
231                                     source.position[1] - position[1],
232                                     source.position[0] - position[0],
233                                 )
234                             )
235                             bearing = (
236                                 round(

```

```

237         (
238             angle
239             + self._bearing_error(position, channel, source)
240         )
241         % 360,
242         2,
243     )
244     % 360
245 )
246     fields = {"measure_result": "direction", "svd_deg": bearing}
247 else:
248     self._clear_attempts += 1
249     success = live and math.dist(position, source.position) <= 20
250     if success:
251         source.cleared = True
252     else:
253         self._failed_clears += 1
254         action_seconds = 5 if success else 3
255         fields = {
256             "clear_result": "success" if success else "no_target_in_range"
257         }
258     self._position = position
259     self._distance += distance
260     self._virtual_us += (
261         round(distance / 5 * 1_000_000)
262         + (action_seconds + int(switched)) * 1_000_000
263     )
264     response = self._response(**fields)
265     if self._virtual_us >= 360000 * 1_000_000:
266
267         self._finish("virtual_timeout")
268     self.log.append(
269         {
270             "request": request,
271             "response": dict(response),
272             "distance_m": distance,
273             "move_time_s": round(distance / 5, 6),
274             "switch_time_s": int(switched),
275             "action_time_s": action_seconds,
276             "radio_channel_before": radio_before,
277             "radio_channel_after": self._radio_channel,
278             "action_virtual_time_s": (self._virtual_us - before) / 1_000_000,
279         }
280     )
281     return response
282
283 def summary(self) -> dict:
284
285     with self._lock:
286         if self._status != "exited":
287             raise RuntimeError("summary is available only after session exit")
288         n = len(self._sources)
289         cleared = sum(source.cleared for source in self._sources.values())
290         virtual_time = self._virtual_us / 1_000_000
291         directional = sum(
292             source.normal is not None for source in self._sources.values()
293         )
294         return {
295             "evidence": "local_synthetic_not_official",
296             "seed": self._seed,
297             "problem": self._problem,
298             "scenario": self._scenario,
299             "error_mode": self._error_mode,
300             "n_sources": n,
301             "n_cleared": cleared,
302             "n_omnidirectional": n - directional,

```

```

303         "n_directional": directional,
304         "fraction_cleared": cleared / n,
305         "virtual_time_s": virtual_time,
306         "average_time_s": virtual_time / cleared if cleared else None,
307         "distance_m": self._distance,
308         "measures": self._measures,
309         "switches": self._switches,
310         "clear_attempts": self._clear_attempts,
311         "failed_clears": self._failed_clears,
312         "real_duration_s": self._finished - self._started,
313         "exit_reason": self._exit_reason,
314     }

```

Listing 3: 几何计算与覆盖工具

```

1
2
3 from __future__ import annotations
4
5 from decimal import Decimal, localcontext
6 from math import ceil, cos, hypot, isfinite, nextafter, pi, radians, sin, sqrt, ulp
7 from random import Random
8
9 Point = tuple[float, float]
10 Circle = tuple[Point, float]
11 ARENA_RADIUS = 1800.0
12 MAX_RADIO_RADIUS = 1500.0
13 BEARING_ERROR_DEG = 1.005
14
15
16 def _check_points(points: list[Point]) -> None:
17     if any(not (isfinite(x) and isfinite(y)) for x, y in points):
18         raise ValueError("Geometry requires finite coordinates")
19
20
21 def _padding(scale: float) -> float:
22     return max(1e-9, 64.0 * ulp(max(1.0, scale)))
23
24
25 def initial_polygon(sides: int = 180) -> list[Point]:
26
27     if isinstance(sides, bool) or not isinstance(sides, int) or sides < 3:
28         raise ValueError("sides must be an integer at least three")
29     radius = (ARENA_RADIUS + _padding(ARENA_RADIUS)) / cos(pi / sides)
30     return [
31         (radius * cos(2 * pi * i / sides), radius * sin(2 * pi * i / sides))
32         for i in range(sides)
33     ]
34
35
36 def _clip_halfplane(
37     poly: list[Point], origin: Point, normal: Point, bound: float, padding: float
38 ) -> list[Point]:
39
40     if not poly:
41         return []
42     nx, ny = normal
43     ox, oy = origin
44     limit = bound + padding
45     result: list[Point] = []
46     previous = poly[-1]
47     dp = nx * (previous[0] - ox) + ny * (previous[1] - oy) - limit
48     for current in poly:
49         dc = nx * (current[0] - ox) + ny * (current[1] - oy) - limit
50         if (dp <= 0.0) != (dc <= 0.0):

```

```

51
52
53     if abs(dp) <= abs(dc):
54         t = dp / (dp - dc)
55         crossing = (
56             previous[0] + t * (current[0] - previous[0]),
57             previous[1] + t * (current[1] - previous[1]),
58         )
59     else:
60         t = dc / (dc - dp)
61         crossing = (
62             current[0] + t * (previous[0] - current[0]),
63             current[1] + t * (previous[1] - current[1]),
64         )
65     if not result or crossing != result[-1]:
66         result.append(crossing)
67     if dc <= 0.0 and (not result or current != result[-1]):
68         result.append(current)
69     previous, dp = current, dc
70     if len(result) > 1 and result[0] == result[-1]:
71         result.pop()
72     return result
73
74
75 def clip_bearing(
76     poly: list[Point],
77     point: Point,
78     bearing_deg: float,
79     error_deg: float = BEARING_ERROR_DEG,
80 ) -> list[Point]:
81
82     _check_points(poly)
83     _check_points([point])
84     if not isfinite(bearing_deg) or not isfinite(error_deg) or not 0 <= error_deg < 90:
85         raise ValueError("Use finite bearing and 0 <= error_deg < 90")
86     if not poly:
87         return []
88     theta = radians(bearing_deg % 360.0)
89     error = radians(error_deg)
90     low, high = theta - error, theta + error
91     scale = max(
92         MAX_RADIO_RADIUS,
93         abs(point[0]),
94         abs(point[1]),
95         *(max(abs(x), abs(y)) for x, y in poly),
96     )
97     padding = _padding(scale)
98
99     result = _clip_halfplane(poly, point, (sin(low), -cos(low)), 0.0, padding)
100    result = _clip_halfplane(result, point, (-sin(high), cos(high)), 0.0, padding)
101    result = _clip_halfplane(result, point, (-cos(theta), -sin(theta)), 0.0, padding)
102    return _clip_halfplane(
103        result, point, (cos(theta), sin(theta)), MAX_RADIO_RADIUS, padding
104    )
105
106
107 def _farthest_pair(poly: list[Point]) -> tuple[Point, Point, float]:
108     first = second = poly[0]
109     distance = 0.0
110     for i, a in enumerate(poly):
111         for j in range(i):
112             b = poly[j]
113             candidate = hypot(a[0] - b[0], a[1] - b[1])
114             if candidate > distance:
115                 first, second, distance = a, b, candidate
116     return first, second, distance

```

```

117
118
119 def diameter(poly: list[Point]) -> float:
120
121     _check_points(poly)
122     if not poly:
123         raise ValueError("An infeasible region has no localization diameter")
124     return _farthest_pair(poly)[2]
125
126
127 def _pair_circle(a: Point, b: Point) -> Circle:
128     center = (a[0] + (b[0] - a[0]) / 2, a[1] + (b[1] - a[1]) / 2)
129     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b))
130
131
132 def _cross(a: Point, b: Point, c: Point) -> float:
133     bx, by = b[0] - a[0], b[1] - a[1]
134     cx, cy = c[0] - a[0], c[1] - a[1]
135     determinant = bx * cy - by * cx
136     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
137     if abs(determinant) > 64 * ulp(max(1.0, scale * scale)):
138         return determinant
139     with localcontext() as context:
140         context.prec = 80
141         ax, ay = Decimal(a[0]), Decimal(a[1])
142         return float(
143             (Decimal(b[0]) - ax) * (Decimal(c[1]) - ay)
144             - (Decimal(b[1]) - ay) * (Decimal(c[0]) - ax)
145         )
146
147
148 def _circumcircle(a: Point, b: Point, c: Point) -> Circle | None:
149     bx, by = b[0] - a[0], b[1] - a[1]
150     cx, cy = c[0] - a[0], c[1] - a[1]
151     determinant = bx * cy - by * cx
152     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
153     if abs(determinant) <= 64 * ulp(max(1.0, scale * scale)):
154
155
156         with localcontext() as context:
157             context.prec = 80
158             axd, ayd = Decimal(a[0]), Decimal(a[1])
159             bxd, byd = Decimal(b[0]) - axd, Decimal(b[1]) - ayd
160             cxd, cyd = Decimal(c[0]) - axd, Decimal(c[1]) - ayd
161             det = 2 * (bxd * cyd - byd * cxd)
162             if not det:
163                 return None
164             bb, cc = bxd * bxd + byd * byd, cxd * cxd + cyd * cyd
165             center = (
166                 float(axd + (cyd * bb - byd * cc) / det),
167                 float(ayd + (bxd * cc - cxd * bb) / det),
168             )
169     else:
170         bb, cc = bx * bx + by * by, cx * cx + cy * cy
171         center = (
172             a[0] + (cy * bb - by * cc) / (2 * determinant),
173             a[1] + (bx * cc - cx * bb) / (2 * determinant),
174         )
175     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b, c))
176
177
178 def _contains(circle: Circle, point: Point) -> bool:
179     center, radius = circle
180     return hypot(point[0] - center[0], point[1] - center[1]) <= nextafter(
181         radius, float("inf")
182     )

```

```

183
184
185 def _two_boundary_circle(points: list[Point], end: int, a: Point, b: Point) -> Circle:
186     base = _pair_circle(a, b)
187     left: Circle | None = None
188     right: Circle | None = None
189     left_cross = right_cross = 0.0
190     for i in range(end):
191         p = points[i]
192         if _contains(base, p):
193             continue
194         side = _cross(a, b, p)
195         candidate = _circumcircle(a, b, p)
196         if candidate is None:
197             continue
198         center_side = _cross(a, b, candidate[0])
199         if side > 0 and (left is None or center_side > left_cross):
200             left, left_cross = candidate, center_side
201         elif side < 0 and (right is None or center_side < right_cross):
202             right, right_cross = candidate, center_side
203     if left is None:
204         return base if right is None else right
205     if right is None:
206         return left
207     return left if left[1] <= right[1] else right
208
209
210 def enclosing_circle(poly: list[Point]) -> Circle:
211
212     _check_points(poly)
213     if not poly:
214         raise ValueError("An infeasible region has no enclosing-circle center")
215     points = list(dict.fromkeys(poly))
216     Random(0).shuffle(points)
217     circle: Circle | None = None
218     for i, p in enumerate(points):
219         if circle is not None and _contains(circle, p):
220             continue
221         circle = (p, 0.0)
222         for j in range(i):
223             q = points[j]
224             if _contains(circle, q):
225                 continue
226             if circle[1] == 0.0:
227                 circle = _pair_circle(p, q)
228             else:
229                 circle = _two_boundary_circle(points, j + 1, p, q)
230     assert circle is not None
231     center = circle[0]
232     radius = max(hypot(p[0] - center[0], p[1] - center[1]) for p in points)
233     if radius == 0.0:
234         return center, 0.0
235     scale = max(radius, abs(center[0]), abs(center[1]))
236     return center, nextafter(radius + _padding(scale), float("inf"))
237
238
239 def optical_cover(poly: list[Point], radius: float = 19.9) -> list[Point]:
240
241     _check_points(poly)
242     if not isfinite(radius) or radius <= 0:
243         raise ValueError("Optical covering radius must be finite and positive")
244     if not poly:
245         return []
246     center, enclosing_radius = enclosing_circle(poly)
247     if enclosing_radius <= radius:
248         return [center]

```

```

249     a, b, distance = _farthest_pair(poly)
250     if distance == 0.0:
251         return [a]
252     ux, uy = (b[0] - a[0]) / distance, (b[1] - a[1]) / distance
253
254     norm = hypot(ux, uy)
255     ux, uy = ux / norm, uy / norm
256     projected = [
257         ((x - a[0]) * ux + (y - a[1]) * uy, -(x - a[0]) * uy + (y - a[1]) * ux)
258         for x, y in poly
259     ]
260     scale = max(distance, *(max(abs(x), abs(y)) for x, y in poly))
261     guard = _padding(scale)
262     safe_radius = radius - 8 * guard
263     if safe_radius <= 0:
264         raise ValueError(
265             "Radius is below floating-point resolution at these coordinates"
266         )
267     xmin, xmax = (
268         min(p[0] for p in projected) - guard,
269         max(p[0] for p in projected) + guard,
270     )
271     ymin, ymax = (
272         min(p[1] for p in projected) - guard,
273         max(p[1] for p in projected) + guard,
274     )
275     side = sqrt(2.0) * safe_radius
276     nx = max(1, ceil((xmax - xmin) / side))
277     ny = max(1, ceil((ymax - ymin) / side))
278     dx, dy = (xmax - xmin) / nx, (ymax - ymin) / ny
279     centers: list[Point] = []
280     for j in range(ny):
281         y = ymin + (j + 0.5) * dy
282         indices = range(nx) if j % 2 == 0 else range(nx - 1, -1, -1)
283         for i in indices:
284             x = xmin + (i + 0.5) * dx
285             centers.append((a[0] + x * ux - y * uy, a[1] + x * uy + y * ux))
286     return centers

```

Listing 4: 官方接口客户端

```

1
2
3 from __future__ import annotations
4
5 import http.client
6 import json
7 import math
8 import queue
9 import threading
10 import time
11 import unicodedata
12 import uuid
13 from pathlib import Path
14 from urllib.error import HTTPError, URLError
15 from urllib.parse import urlsplit
16 from urllib.request import HTTPRedirectHandler, ProxyHandler, Request, build_opener
17
18 from submission.支撑材料.environment import Point, _action_args
19
20
21 class ClientError(RuntimeError):
22     pass
23
24

```



```

25 class RejectedActionError(ClientError):
26     def __init__(self, status: int, response: dict):
27         self.status, self.response = status, response
28         super().__init__(
29             f"Simulator rejected action: HTTP {status}, accepted={response.get('accepted')}!r}"
30         )
31
32
33 class AmbiguousActionError(ClientError):
34     pass
35
36
37 class ClientProtocolError(ClientError):
38     pass
39
40
41 class DeadlineExceeded(ClientError):
42     pass
43
44
45 class _NoRedirect(HTTPRedirectHandler):
46     def redirect_request(self, req, fp, code, msg, headers, newurl):
47
48         return None
49
50
51 def _number(value) -> bool:
52     return (
53         not isinstance(value, bool)
54         and isinstance(value, (int, float))
55         and math.isfinite(value)
56     )
57
58
59 def _unique_object(pairs):
60     result = {}
61     for key, value in pairs:
62         if key in result:
63             raise ValueError(f"Duplicate response key: {key}")
64         result[key] = value
65     return result
66
67
68 class HTTPClient:
69
70     _MAX_ATTEMPTS = 4
71     _REQUEST_TIMEOUT_S = 5.0
72     _ACTION_RETRY_BUDGET_S = 20.0
73     _MAX_RESPONSE_BYTES = 65536
74     _TRANSIENT_HTTP = frozenset((429, 500, 502, 503, 504))
75
76     def __init__(self, base_url: str, robot_id: str, log_path: str):
77         parsed = urlsplit(base_url)
78         if (
79             parsed.scheme not in ("http", "https")
80             or not parsed.hostname
81             or parsed.path not in ("", "/")
82             or parsed.query
83             or parsed.fragment
84             or parsed.username is not None
85             or parsed.password is not None
86         ):
87             raise ValueError(
88                 "base_url must be an HTTP(S) origin, without credentials/path/query"
89             )
90

```

```

91
92     _ = parsed.port
93     if (
94         not isinstance(robot_id, str)
95         or not 1 <= len(robot_id.encode("utf-8")) <= 64
96     ):
97         raise ValueError("robot_id must contain 1..64 UTF-8 bytes")
98     if any(unicodedata.category(c) in ("Cc", "Cf") for c in robot_id):
99         raise ValueError("robot_id cannot contain control or format characters")
100     self._base_url, self._robot_id = base_url.rstrip("/"), robot_id
101     self._lock = threading.Lock()
102     self._opener = build_opener(ProxyHandler({}), _NoRedirect())
103     self._closed = False
104     self._uncertain = False
105     self._entered = self._exited = False
106     self._deadline: float | None = None
107     self._max_virtual_duration_s: float | None = None
108     self.position: Point = (0.0, 0.0)
109     self.radio_channel = 1
110     self.virtual_time_s = 0.0
111     self._log = Path(log_path).open("a", encoding="utf-8", buffering=1)
112
113     @property
114     def remaining_real_duration_s(self) -> float | None:
115
116         return (
117             None
118             if self._deadline is None
119             else max(0.0, self._deadline - time.monotonic())
120         )
121
122     def _record(self, event: dict):
123         self._log.write(
124             json.dumps(
125                 {"logged_at_ms": int(time.time() * 1000), **event},
126                 ensure_ascii=False,
127                 allow_nan=False,
128                 separators=(",", ":"),
129             )
130             + "\n"
131         )
132         self._log.flush()
133
134     def _decode(self, raw: bytes) -> dict:
135         if len(raw) > self._MAX_RESPONSE_BYTES:
136             raise ClientProtocolError("Response exceeds bounded JSON body size")
137         try:
138             response = json.loads(raw.decode("utf-8"), object_pairs_hook=_unique_object)
139         except (UnicodeDecodeError, ValueError) as exc:
140             raise ClientProtocolError(
141                 "Response is not valid unambiguous UTF-8 JSON"
142             ) from exc
143         if not isinstance(response, dict) or type(response.get("accepted")) is not bool:
144             raise ClientProtocolError("Response lacks a boolean accepted field")
145         if (
146             not _number(response.get("real_timestamp_ms"))
147             or not _number(response.get("virtual_time_s"))
148             or response["virtual_time_s"] < 0
149         ):
150             raise ClientProtocolError("Response lacks valid numeric timestamps")
151         return response
152
153     def _validate_accepted(self, path: str, response: dict):
154         virtual_time = response["virtual_time_s"]
155         if path == "/enter":
156             remaining = response.get("remaining_real_duration_s")

```

```

157         if (
158             not _number(remaining)
159             or not 0 <= remaining <= 1200
160             or int(remaining) != remaining
161         ):
162             raise ClientProtocolError("Invalid actual remaining real duration")
163         if (
164             not _number(response.get("max_virtual_duration_s"))
165             or response["max_virtual_duration_s"] <= 0
166             or not _number(response.get("max_real_duration_s"))
167             or response["max_real_duration_s"] <= 0
168             or virtual_time != 0
169         ):
170             raise ClientProtocolError(
171                 "Invalid accepted entry limits or initial time"
172             )
173     elif virtual_time < self.virtual_time_s:
174         raise ClientProtocolError("Accepted action moved virtual time backwards")
175     if path == "/measure":
176         result = response.get("measure_result")
177         if result not in ("near", "no_signal", "direction"):
178             raise ClientProtocolError("Unknown measure result")
179         if result == "direction":
180             angle = response.get("svd_deg")
181             if not _number(angle) or not 0 <= angle < 360:
182                 raise ClientProtocolError("Invalid direction angle")
183         elif "svd_deg" in response:
184             raise ClientProtocolError("Bearing supplied without direction result")
185     elif path == "/clear" and response.get("clear_result") not in (
186         "success",
187         "no_target_in_range",
188     ):
189         raise ClientProtocolError("Unknown clear result")
190     elif path == "/exit" and response.get("exit_reason") != "user_exit":
191         raise ClientProtocolError("Unexpected successful exit reason")
192
193     def _commit(
194         self,
195         path: str,
196         position: Point | None,
197         channel: int | None,
198         response: dict,
199         first_send: float,
200     ):
201         if path == "/enter":
202             self._entered = True
203             self.position, self.radio_channel = (0.0, 0.0), 1
204             self._deadline = first_send + response["remaining_real_duration_s"]
205             self._max_virtual_duration_s = response["max_virtual_duration_s"]
206         elif path == "/exit":
207             self._exited = True
208         else:
209             self.position = position
210             if path == "/measure":
211                 self.radio_channel = channel
212             self.virtual_time_s = response["virtual_time_s"]
213             self._uncertain = False
214
215     def _exchange(self, request: Request, deadline: float) -> tuple[int, bytes]:
216
217         completed = queue.Queue(maxsize=1)
218
219         def receive():
220             try:
221                 available = deadline - time.monotonic()
222                 if available <= 0:

```

```

223         raise TimeoutError("Absolute HTTP deadline elapsed")
224     try:
225         stream = self._opener.open(
226             request, timeout=min(self._REQUEST_TIMEOUT_S, available)
227         )
228     except HTTPError as exc:
229         stream = exc
230     with stream:
231         status = stream.code
232         expected_length = stream.headers.get("Content-Length")
233         if expected_length is not None:
234             try:
235                 expected_length = int(expected_length)
236             except ValueError as exc:
237                 raise ClientProtocolError("Invalid Content-Length") from exc
238             if not 0 <= expected_length <= self._MAX_RESPONSE_BYTES:
239                 raise ClientProtocolError(
240                     "Response exceeds bounded JSON body size"
241                 )
242             raw = stream.read(self._MAX_RESPONSE_BYTES + 1)
243             if expected_length is not None and len(raw) < expected_length:
244                 raise http.client.IncompleteRead(
245                     raw, expected_length - len(raw)
246                 )
247             completed.put((True, (status, raw)))
248     except (
249         ClientError,
250         OSError,
251         ValueError,
252         TypeError,
253         TimeoutError,
254         http.client.HTTPException,
255     ) as exc:
256         completed.put((False, exc))
257
258     threading.Thread(target=receive, daemon=True, name="simulator-response").start()
259     try:
260         succeeded, value = completed.get(
261             timeout=max(0.0, deadline - time.monotonic())
262         )
263     except queue.Empty as exc:
264         raise TimeoutError(
265             "Absolute HTTP deadline elapsed; outcome remains unknown"
266         ) from exc
267     if not succeeded:
268         raise value
269     return value
270
271     def act(
272         self, path: str, position: Point | None = None, channel: int | None = None
273     ) -> dict:
274         position, channel = _action_args(path, position, channel)
275         with self._lock:
276             if self._closed:
277                 raise ClientError("Client is closed")
278             if self._uncertain:
279                 raise AmbiguousActionError(
280                     "Previous outcome is unresolved; no different action may be sent"
281                 )
282             if self._exited:
283                 raise ClientError("Session already exited; do not query /exit again")
284             if self._deadline is not None and time.monotonic() >= self._deadline:
285                 raise DeadlineExceeded(
286                     "Actual remaining real duration has elapsed; interface may be closed"
287                 )
288             if (

```

```

289         self._max_virtual_duration_s is not None
290         and self.virtual_time_s >= self._max_virtual_duration_s
291     ):
292         raise DeadlineExceeded(
293             "Virtual deadline reached; no post-timeout /exit query is legal"
294         )
295     if path == "/enter" and self._entered:
296         raise ClientError("Cannot enter the same test twice")
297     if path != "/enter" and not self._entered:
298         raise ClientError(
299             "Explicit successful /enter is required before other actions"
300         )
301     request_id = uuid.uuid4().hex
302     payload = {
303         "arena_id": "default",
304         "robot_id": self._robot_id,
305         "request_id": request_id,
306     }
307     if position is not None:
308         payload.update(
309             position={"x": position[0], "y": position[1]}, channel=channel
310         )
311     body = json.dumps(
312         payload, ensure_ascii=False, allow_nan=False, separators=(",", ":")
313     ).encode("utf-8")
314     first_send = time.monotonic()
315     stop = first_send + self._ACTION_RETRY_BUDGET_S
316     if self._deadline is not None:
317         stop = min(stop, self._deadline)
318     last_error = "No response"
319     for attempt in range(1, self._MAX_ATTEMPTS + 1):
320         available = stop - time.monotonic()
321         if available <= 0:
322             break
323         self._record(
324             {
325                 "event": "request",
326                 "path": path,
327                 "request_id": request_id,
328                 "attempt": attempt,
329                 "body": payload,
330             }
331         )
332         request = Request(
333             self._base_url + path,
334             data=body,
335             headers={"Content-Type": "application/json"},
336             method="POST",
337         )
338
339         self._uncertain = True
340         try:
341             status, raw = self._exchange(request, stop)
342             self._record(
343                 {
344                     "event": "response",
345                     "path": path,
346                     "request_id": request_id,
347                     "attempt": attempt,
348                     "http_status": status,
349                     "body_utf8": raw.decode("utf-8", errors="replace"),
350                 }
351             )
352         except (URLError, OSError, http.client.HTTPException) as exc:

```

```

355         last_error = f"{type(exc).__name__}: {exc}"
356     self._record(
357         {
358             "event": "transport_error",
359             "path": path,
360             "request_id": request_id,
361             "attempt": attempt,
362             "error": last_error,
363         }
364     )
365     else:
366         if status in self._TRANSIENT_HTTP:
367             last_error = f"Transient HTTP {status}"
368         else:
369             response = self._decode(raw)
370             if status != 200 or response["accepted"] is not True:
371                 if response["accepted"] is True:
372                     raise ClientProtocolError(
373                         "accepted=true with non-200 HTTP status"
374                     )
375                 self._uncertain = False
376                 raise RejectedActionError(status, response)
377             self._validate_accepted(path, response)
378             radio_before = self.radio_channel
379             distance = (
380                 math.dist(self.position, position)
381                 if position is not None
382                 else 0.0
383             )
384             switch = int(path == "/measure" and channel != radio_before)
385             self._record(
386                 {
387                     "event": "accepted",
388                     "path": path,
389                     "request_id": request_id,
390                     "distance_m": distance,
391                     "radio_channel_before": radio_before,
392                     "radio_channel_after": channel
393                     if path == "/measure"
394                     else radio_before,
395                     "switch_time_s": switch,
396                     "action_virtual_time_s": response["virtual_time_s"]
397                     - self.virtual_time_s,
398                 }
399             )
400             self._commit(path, position, channel, response, first_send)
401             return response
402         if attempt < self._MAX_ATTEMPTS:
403             delay = min(
404                 0.2 * 2 ** (attempt - 1), max(0.0, stop - time.monotonic())
405             )
406             if delay > 0:
407                 time.sleep(delay)
408             raise AmbiguousActionError(
409                 f"Action {request_id} has no confirmed outcome after bounded retries: {last_error}"
410             )
411
412     def close(self):
413
414         with self._lock:
415             if not self._closed:
416                 self._log.close()
417                 self._closed = True

```

Listing 5: 问题三、四策略实现

```

1
2
3 from __future__ import annotations
4
5 from math import dist, sqrt
6
7 from submission.支撑材料.geometry import enclosing_circle
8 from submission.支撑材料.joint_dispatch_candidate import choose_task
9 from submission.支撑材料.negative_geometry import exclude_omni_disk
10 from submission.支撑材料.planner import InconsistentObservations, Planner
11 from submission.支撑材料.route_portfolio import choose_portfolio_task
12
13
14 class FieldPlanner(Planner):
15     def __init__(
16         self,
17         environment,
18         problem=3,
19         *,
20         premeasure=True,
21         negative_updates=True,
22         portfolio=True,
23         risk_weight=None,
24         probe_scale=0.22,
25         approach_clear=None,
26     ):
27         super().__init__(
28             environment, problem=problem, strategy="refined", probe_scale=probe_scale
29         )
30         self.premeasure = premeasure
31         self.negative_updates = negative_updates and problem == 3
32         self.portfolio = portfolio
33         self.approach_clear = problem == 3 if approach_clear is None else approach_clear
34         self.risk_weight = (
35             (0.0 if problem == 3 else 1.0) if risk_weight is None else risk_weight
36         )
37         self.negative_stations = {channel: [] for channel in range(1, 21)}
38         self.premeasured_channels = set()
39         self.zero_travel_probes = 0
40         self.negative_cuts = 0
41         self.clear_access_saved_m = 0.0
42
43     def measure(self, channel, point):
44         status = super().measure(channel, point)
45         if not self.negative_updates or channel in self.cleared:
46             return status
47         if status == "no_signal":
48             self.negative_stations[channel].append(tuple(point))
49             track = self.tracks.get(channel)
50             if track is not None and status in ("direction", "no_signal"):
51
52
53                 for station in self.negative_stations[channel]:
54                     before = track.polygon
55                     track.polygon = exclude_omni_disk(before, station, sides=16)
56                     self.negative_cuts += track.polygon != before
57                     if not track.polygon:
58                         raise InconsistentObservations(
59                             "Omnidirectional negative readings exclude the entire track"
60                         )
61             return status
62
63     def localize(self, channel):
64         track = self.tracks.get(channel)

```

```

65         if (
66             self.premeasure
67             and channel not in self.premeasured_channels
68             and track is not None
69             and self.terminal_reason is None
70         ):
71             center, radius = enclosing_circle(track.polygon)
72             if (
73                 radius > 38
74                 and tuple(self.position) not in track.observations
75                 and dist(self.position, center) - radius <= 1500
76             ):
77                 self.zero_travel_probes += 1
78                 self.measure(channel, self.position)
79                 self.premeasured_channels.add(channel)
80                 if channel in self.cleared or self.terminal_reason is not None:
81                     return
82             return super().localize(channel)
83
84     def clear(self, channel, point, guaranteed=False):
85         track = self.tracks.get(channel)
86         length = dist(self.position, point)
87         if (
88             self.approach_clear
89             and guaranteed
90             and track is not None
91             and length > 1e-6
92             and max(dist(point, p) for p in track.polygon) <= 19.9
93         ):
94
95
96             u = (
97                 (self.position[0] - point[0]) / length,
98                 (self.position[1] - point[1]) / length,
99             )
100             shift = length
101             for p in track.polygon:
102                 dx, dy = point[0] - p[0], point[1] - p[1]
103                 dot = dx * u[0] + dy * u[1]
104                 discriminant = dot * dot + 19.9**2 - dx * dx - dy * dy
105                 if discriminant < 0.0:
106                     shift = 0.0
107                     break
108                 shift = min(shift, -dot + sqrt(discriminant))
109             shift = max(0.0, shift - 1e-6)
110             candidate = (point[0] + shift * u[0], point[1] + shift * u[1])
111             if max(dist(candidate, p) for p in track.polygon) <= 19.9:
112                 self.clear_access_saved_m += length - dist(self.position, candidate)
113                 point = candidate
114             return super().clear(channel, point, guaranteed=guaranteed)
115
116     def select_task(self):
117         anchors = {i: self.waypoints[i] for i in sorted(self.pending)}
118         targets = {
119             ch: enclosing_circle(track.polygon) for ch, track in self.tracks.items()
120         }
121         if self.portfolio:
122             return choose_portfolio_task(
123                 self.position, anchors, targets, risk_weight=self.risk_weight
124             )
125         return choose_task(
126             self.position,
127             anchors,
128             targets,
129             choice="guarded",
130             risk_weight=self.risk_weight,

```



```

131         )
132
133     def _result(self):
134         result = super()._result()
135         result.update(
136             strategy="field",
137             premeasure=self.premeasure,
138             negative_updates=self.negative_updates,
139             portfolio=self.portfolio,
140             risk_weight=self.risk_weight,
141             zero_travel_probes=self.zero_travel_probes,
142             negative_cuts=self.negative_cuts,
143             approach_clear=self.approach_clear,
144             clear_access_saved_m=self.clear_access_saved_m,
145         )
146         return result

```

Listing 6: 路线组合与负观测几何

```

1
2
3 from __future__ import annotations
4
5 from math import dist, fsum, isfinite
6
7 Point = tuple[float, float]
8 Task = tuple[str, int]
9
10
11 def _cost(
12     route: list[int],
13     origin: list[float],
14     edges: list[list[float]],
15     penalty: list[float],
16 ) -> float:
17     try:
18         value = fsum(
19             (
20                 origin[route[0]],
21                 penalty[route[0]],
22                 *(edges[a][b] for a, b in zip(route, route[1:])),
23             )
24         )
25     except OverflowError as error:
26         raise ValueError("route proxy must be finite") from error
27     if not isfinite(value):
28         raise ValueError("route proxy must be finite")
29     return value
30
31
32 def _insertion_delta(
33     route: list[int],
34     node: int,
35     gap: int,
36     origin: list[float],
37     edges: list[list[float]],
38     penalty: list[float],
39 ) -> float:
40     if gap == 0:
41         return (
42             origin[node]
43             + edges[node][route[0]]
44             - origin[route[0]]
45             + penalty[node]
46             - penalty[route[0]]

```

```

47     )
48     before = route[gap - 1]
49     delta = edges[before][node]
50     if gap < len(route):
51         after = route[gap]
52         delta += edges[node][after] - edges[before][after]
53     return delta
54
55
56 def _control_route(
57     origin: list[float], edges: list[list[float]], penalty: list[float]
58 ) -> list[int]:
59
60     n = len(origin)
61     first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
62     route = [first]
63     remaining = set(range(n))
64     remaining.remove(first)
65     nearest = [min(origin[i], edges[first][i]) for i in range(n)]
66     while remaining:
67         node = min(remaining, key=lambda i: (nearest[i], i))
68         best_gap = 0
69         best_delta = _insertion_delta(route, node, 0, origin, edges, penalty)
70         for gap in range(1, len(route) + 1):
71             delta = _insertion_delta(route, node, gap, origin, edges, penalty)
72             if delta < best_delta:
73                 best_gap, best_delta = gap, delta
74         route.insert(best_gap, node)
75         remaining.remove(node)
76         for other in remaining:
77             nearest[other] = min(nearest[other], edges[node][other])
78
79     for _ in range(n):
80         changed = False
81         for left in range(n - 1):
82             for right in range(left + 1, n):
83                 old = (
84                     origin[route[left]]
85                     if left == 0
86                     else edges[route[left - 1]][route[left]]
87                 )
88                 new = (
89                     origin[route[right]]
90                     if left == 0
91                     else edges[route[left - 1]][route[right]]
92                 )
93                 if right + 1 < n:
94                     old += edges[route[right]][route[right + 1]]
95                     new += edges[route[left]][route[right + 1]]
96                 if left == 0:
97                     old += penalty[route[left]]
98                     new += penalty[route[right]]
99                 if new + 1e-12 * max(1.0, old, new) < old:
100                     route[left : right + 1] = reversed(route[left : right + 1])
101                     changed = True
102                     break
103             if changed:
104                 break
105         if not changed:
106             break
107     return route
108
109
110 def _cheapest_route(
111     origin: list[float], edges: list[list[float]], penalty: list[float]
112 ) -> list[int]:

```

```

113
114     n = len(origin)
115     first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
116     route = [first]
117     remaining = set(range(n))
118     remaining.remove(first)
119     while remaining:
120
121         _, node, gap = min(
122             (_insertion_delta(route, node, gap, origin, edges, penalty), node, gap)
123             for node in remaining
124             for gap in range(len(route) + 1)
125         )
126         route.insert(gap, node)
127         remaining.remove(node)
128     return route
129
130
131 def _nearest_route(first: int, edges: list[list[float]]) -> list[int]:
132     remaining = set(range(len(edges)))
133     remaining.remove(first)
134     route = [first]
135     while remaining:
136         node = min(remaining, key=lambda i: (edges[route[-1]][i], i))
137         route.append(node)
138         remaining.remove(node)
139     return route
140
141
142 def _improve(
143     route: list[int],
144     origin: list[float],
145     edges: list[list[float]],
146     penalty: list[float],
147 ) -> tuple[list[int], float]:
148
149     n = len(route)
150     cost = _cost(route, origin, edges, penalty)
151     for _ in range(min(n, 12)):
152
153         best_delta = -1e-12 * max(1.0, cost)
154         move: tuple[str, int, int] | None = None
155         for left in range(n - 1):
156             for right in range(left + 1, n):
157                 old = (
158                     origin[route[left]]
159                     if left == 0
160                     else edges[route[left - 1]][route[left]]
161                 )
162                 new = (
163                     origin[route[right]]
164                     if left == 0
165                     else edges[route[left - 1]][route[right]]
166                 )
167                 if right + 1 < n:
168                     old += edges[route[right]][route[right + 1]]
169                     new += edges[route[left]][route[right + 1]]
170                 if left == 0:
171                     old += penalty[route[left]]
172                     new += penalty[route[right]]
173                 delta = new - old
174                 if delta < best_delta:
175                     best_delta, move = delta, ("reverse", left, right)
176
177     for index, node in enumerate(route):

```

```

179         if n == 1:
180             break
181         if index == 0:
182             following = route[1]
183             removal = (
184                 origin[following]
185                 + penalty[following]
186                 - origin[node]
187                 - penalty[node]
188                 - edges[node][following]
189             )
190         else:
191             before = route[index - 1]
192             removal = -edges[before][node]
193             if index + 1 < n:
194                 following = route[index + 1]
195                 removal += edges[before][following] - edges[node][following]
196
197     for gap in range(n + 1):
198         if gap == index or gap == index + 1:
199             continue
200         delta = removal + _insertion_delta(
201             route, node, gap, origin, edges, penalty
202         )
203         if delta < best_delta:
204             best_delta, move = delta, ("relocate", index, gap)
205
206     if move is None:
207         break
208     candidate = route.copy()
209     kind, left, right = move
210     if kind == "reverse":
211         candidate[left : right + 1] = reversed(candidate[left : right + 1])
212     else:
213         node = candidate.pop(left)
214         candidate.insert(right if right < left else right - 1, node)
215     candidate_cost = _cost(candidate, origin, edges, penalty)
216     if not candidate_cost < cost:
217         break
218     route, cost = candidate, candidate_cost
219
220 return route, cost
221
222
223
224 def choose_portfolio_task(
225     position: Point,
226     anchors: dict[int, Point],
227     targets: dict[int, tuple[Point, float]],
228     risk_weight: float = 1.0,
229 ) -> Task:
230
231     target_keys = sorted(targets)
232     anchor_keys = sorted(anchors)
233     tasks: list[Task] = [("target", key) for key in target_keys]
234     tasks.extend(("anchor", key) for key in anchor_keys)
235     n = len(tasks)
236     if not n:
237         raise ValueError("cannot dispatch an empty task set")
238     coords = [targets[key][0] for key in target_keys]
239     coords.extend(anchors[key] for key in anchor_keys)
240     radii = [targets[key][1] for key in target_keys]
241     radii.extend(0.0 for _ in anchor_keys)
242     if any(
243         len(point) != 2 or not all(isfinite(x) for x in point)
244         for point in [position, *coords]

```

```

245 ):
246     raise ValueError("positions must be finite two-dimensional points")
247 if any(not isfinite(radius) or radius < 0 for radius in radii):
248     raise ValueError("MEC radii must be finite and nonnegative")
249 if not isfinite(risk_weight) or risk_weight < 0:
250     raise ValueError("risk_weight must be finite and nonnegative")
251 penalty = [risk_weight * radius for radius in radii]
252 if any(not isfinite(value) for value in penalty):
253     raise ValueError("weighted radii must be finite")
254 origin = [dist(position, point) for point in coords]
255 edges = [[0.0] * n for _ in range(n)]
256 for i in range(n):
257     for j in range(i):
258         edges[i][j] = edges[j][i] = dist(coords[i], coords[j])
259 if any(not isfinite(value) for value in origin) or any(
260     not isfinite(value) for row in edges for value in row
261 ):
262     raise ValueError("pairwise distances must be finite")
263
264
265 try:
266     bound = fsum((max(origin), max(penalty), *(max(row) for row in edges)))
267 except OverflowError as error:
268     raise ValueError("route proxy bound must be finite") from error
269 if not isfinite(bound):
270     raise ValueError("route proxy bound must be finite")
271
272 control = _control_route(origin, edges, penalty)
273 control_cost = _cost(control, origin, edges, penalty)
274 best_route, best_cost = control, control_cost
275
276
277 starts = sorted(range(n), key=lambda i: (origin[i] + penalty[i], i))[2:]
278 seeds = [control, _cheapest_route(origin, edges, penalty)]
279 seeds.extend(_nearest_route(first, edges) for first in starts)
280 seen: set[tuple[int, ...]] = set()
281 for seed in seeds:
282     identity = tuple(seed)
283     if identity in seen:
284         continue
285     seen.add(identity)
286     route, cost = _improve(seed, origin, edges, penalty)
287     if cost < best_cost:
288         best_route, best_cost = route, cost
289
290 if best_cost > control_cost:
291     return tasks[control[0]]
292 return tasks[best_route[0]]

```

Listing 7: 本地实验与官方日志分析入口

```

1
2
3 from __future__ import annotations
4 import json
5 from pathlib import Path
6 from submission.支撑材料.field_study import run_field, pair_stats, summarize
7
8 CONFIGS = {
9     3: {
10         "control": None,
11         "route_only": {"premeasure": False, "portfolio": True},
12         "route_info": {"premeasure": True, "negative_updates": True, "portfolio": True},
13         "field": {
14             "premeasure": True,

```

```

15         "negative_updates": True,
16         "portfolio": True,
17         "approach_clear": True,
18     },
19 },
20 4: {"control": None, "field": {"premeasure": True, "portfolio": True}},
21 }
22
23
24 def group(problem, name, seeds, scenario="random", mode="smooth"):
25     rows = [
26         run_field(seed, problem, CONFIGS[problem][name], scenario, mode)
27         for seed in seeds
28     ]
29     for row in rows:
30         row["configuration"] = name
31     return rows
32
33
34 def main():
35     records = []
36     summary = {}
37     comparisons = {}
38     stress = {}
39     for p in (3, 4):
40         groups = {}
41         for name in CONFIGS[p]:
42             groups[name] = group(p, name, range(7100000, 7101000))
43             records.extend(groups[name])
44             summary[f"Q{p}_{name}"] = summarize(groups[name])
45             comparisons[f"Q{p}"] = pair_stats(groups["control"], groups["field"])
46             comparisons[f"Q{p}_ablation"] = {
47                 f"{a}_to_{b}": pair_stats(groups[a], groups[b])
48                 for a, b in zip(groups, list(groups)[1:])
49             }
50             print(f"Q{p}", json.dumps(comparisons[f"Q{p}"]), flush=True)
51             cases = [
52                 (s, m, range(7200000, 7200050))
53                 for s in ("minimum_radius", "outward_boundary", "clustered")
54                 for m in ("smooth", "extreme", "iid_location")
55             ]
56             cases += [
57                 ("random", m, range(7210000, 7210200)) for m in ("extreme", "iid_location")
58             ]
59             for s, m, seeds in cases:
60                 a = group(p, "control", seeds, s, m)
61                 b = group(p, "field", seeds, s, m)
62                 records.extend(a)
63                 records.extend(b)
64                 key = f"Q{p}_{s}_{m}"
65                 stress[key] = {
66                     "control": summarize(a),
67                     "field": summarize(b),
68                     "paired": pair_stats(a, b),
69                 }
70                 print(key, json.dumps(stress[key]["paired"]), flush=True)
71     result = {
72         "evidence": "fresh_local_paired_confirmation_not_official_counterfactual",
73         "selection": "Selected using official-observation bottlenecks and development seeds6100000..6100149 only. Refined probe=.22 control is unchanged. No retuning after this study.",
74         "main_seed_range": [7100000, 7100999],
75         "stress_seed_range": [7200000, 7200049],
76         "alternate_error_seed_range": [7210000, 7210199],
77         "development_seed_range": [6100000, 6100149],
78         "main_runs": 6000,
79         "stress_runs": 1800,

```

```

80         "alternate_error_runs": 1600,
81         "total_runs": len(records),
82         "configs": CONFIGS,
83         "summary": summary,
84         "comparison": comparisons,
85         "stress": stress,
86         "runs": records,
87     }
88     assert len(records) == 9400
89     Path("results/field_confirmation.json").write_text(
90         json.dumps(result, ensure_ascii=False, indent=2) + "\n"
91     )
92     print(json.dumps({"total_runs": len(records), "summary": summary}, indent=2))
93
94
95 if __name__ == "__main__":
96     main()

```

Listing 8: 统一打包与复现验证

```

1
2
3 from __future__ import annotations
4
5 import argparse
6 import csv
7 import hashlib
8 import importlib.metadata
9 import json
10 import os
11 import re
12 import shutil
13 import subprocess
14 import sys
15 import tempfile
16 import time
17 import zipfile
18 from pathlib import Path, PurePosixPath
19
20 import pymupdf
21
22 ROOT = Path(__file__).resolve().parent
23 SOURCES = (
24     "geometry.py",
25     "coverage.py",
26     "planner.py",
27     "environment.py",
28     "client.py",
29     "run.py",
30     "official_run.py",
31     "intersection.py",
32     "second_point.py",
33     "experiments.py",
34     "compare_recovery.py",
35     "compare_routes.py",
36     "compare_probes.py",
37     "verify_geometry.py",
38     "verify_coverage_limits.py",
39     "verify_refinements.py",
40     "verify_terminal.py",
41     "verify_http.py",
42     "make_enhancement_assets.py",
43     "make_assets.py",
44     "audit_pdf.py",
45     "package.py",

```

```

46     "active_policy_candidate.py",
47     "route_policy_candidate.py",
48     "active_policy_notes.txt",
49     "route_policy_notes.txt",
50     "polar_coverage_candidate.py",
51     "polar_coverage_notes.txt",
52     "joint_dispatch_candidate.py",
53     "joint_dispatch_notes.txt",
54     "optical_policy_candidate.py",
55     "optical_policy_notes.txt",
56     "refinement_study.py",
57     "research_experiments.py",
58     "verify_research.py",
59     "make_research_assets.py",
60     "make_official_assets.py",
61     "fast_vs_compact_410.py",
62     "field_policy.py",
63     "negative_geometry.py",
64     "route_portfolio.py",
65     "field_study.py",
66     "field_confirmation.py",
67     "verify_field.py",
68     "official_analysis.py",
69     "make_field_assets.py",
70     "make_competition_assets.py",
71     "redraw_q1q2.py",
72     "redraw_q34.py",
73     "redraw_results.py",
74     "negative_geometry_notes.txt",
75     "route_portfolio_notes.txt",
76     "contracts.json",
77     "report_contracts.json",
78     "enhancements.py",
79     "ring_sweep.py",
80     "develop_anchors.py",
81     "theory_additions.txt",
82     "paper_expansion.txt",
83     "coverage_notes.txt",
84     "coverage_limits.tex",
85     "requirements.txt",
86     "reproduce.txt",
87     "official_status.json",
88     "audit/route_exact_audit.py",
89     "audit/q3_ablation_audit.py",
90     "audit/correctness_audit.py",
91     "audit/alternative_screen.py",
92     "audit/freeze_compare.py",
93 )
94 RESULTS = (
95     "final_experiments.json",
96     "geometry_verification.json",
97     "intersection_verification.json",
98     "second_point_example.json",
99     "recovery_development.json",
100     "route_candidate_comparison.json",
101     "adaptive_candidate_comparison.json",
102     "terminal_verification.json",
103     "ring_sweep.json",
104     "enhancement_experiments.json",
105     "anchor_development.json",
106     "enhancement_review.json",
107     "coverage_limits.json",
108     "refinement_verification.json",
109     "refinement_development.json",
110     "research_experiments.json",
111     "research_verification.json",

```



```

112     "research_review.json",
113     "field_development.json",
114     "field_confirmation.json",
115     "field_verification.json",
116     "field_review.json",
117     "official_practice_analysis.json",
118     "fast_vs_compact_410.json",
119     "practice_optimization_screen.json",
120     "http_verification.json",
121     "http_deadline_verification.json",
122     "review_resolutions.json",
123     "operation_bound.json",
124     "paper_review.json",
125     "visual_review.json",
126     "final_review.json",
127     "user_supplied_field_practice_summary.json",
128     "environment_manifest.json",
129     "audit_route_exact.json",
130     "audit_q3_ablation.json",
131     "audit_distribution.json",
132     "audit_correctness.json",
133     "audit_alternatives.json",
134     "audit_freeze_compare.json",
135     "formal_results.json",
136 )
137
138 def verify_asset_inputs(root: Path):
139     for metadata_name in ("metadata.json", "enhancement_metadata.json", "research_metadata.json", "
140         official_practice_metadata.json", "field_metadata.json"):
141         metadata = json.loads((root / "report/generated" / metadata_name).read_text())
142         for name, expected in metadata["input_sha256"].items():
143             actual = hashlib.sha256((root / name).read_bytes()).hexdigest()
144             if actual != expected:
145                 raise ValueError(f"Stale generated asset input: {name}")
146
147 def deterministic_content(value):
148
149     if isinstance(value, dict):
150         return {key: deterministic_content(item) for key, item in value.items()
151             if key not in ("real_duration_s", "max_real_duration_s", "max_real_s")}
152     if isinstance(value, list):
153         return [deterministic_content(item) for item in value]
154     return value
155
156
157 def practice_files(root: Path):
158
159     root = root.resolve()
160     manifest = root / "manifest.csv"
161     if manifest.is_symlink():
162         raise ValueError("Practice manifest must not be a symbolic link")
163     with manifest.open(encoding="utf-8", newline="") as stream:
164         rows = list(csv.DictReader(stream))
165         if len(rows) != 20 or {(int(r["problem"]), int(r["round"])) for r in rows} != {
166             (p, n) for p in (3, 4) for n in range(1, 11)
167         }:
168             raise ValueError("Expected exactly ten supplied practice rounds per problem")
169     files = {"manifest.csv": manifest}
170
171 def check_identifiers(value):
172     if isinstance(value, dict):
173         for key, item in value.items():
174             if key == "robot_id" and item != "TEAM_REDACTED":
175                 raise ValueError("Unredacted robot_id in practice data")
176             check_identifiers(item)

```

```

177         elif isinstance(value, list):
178             for item in value:
179                 check_identifiers(item)
180
181     for row in rows:
182         controller = f"data/controller/q{int(row['problem'])}/round_{int(row['round']):02d}"
183         for key in ("private_requests", "observed_result", "simulator_result"):
184             name = row[key]
185             relative = PurePosixPath(name)
186             if relative.is_absolute() or "." in relative.parts or "\\" in name:
187                 raise ValueError(f"Unsafe practice path: {name}")
188             if key == "simulator_result":
189                 allowed = (relative.parent == PurePosixPath("data/simulator_results")
190                             and re.fullmatch(r"practice-p[34]-[0-9]+-[A-Z0-9-]+\result\.json", relative.name))
191             else:
192                 filename = "private_requests.json" if key == "private_requests" else "observed_result.json"
193                 allowed = name == f"{controller}/{filename}"
194             path = root / relative
195             if not allowed or path.is_symlink() or not path.resolve().is_relative_to(root):
196                 raise ValueError(f"Practice path outside permitted source scope: {name}")
197             if name in files:
198                 raise ValueError(f"Repeated practice source: {name}")
199             text = path.read_text(encoding="utf-8")
200             values = [json.loads(line) for line in text.splitlines()] if key == "private_requests" else [json.
loads(text)]
201             for value in values:
202                 check_identifiers(value)
203                 if key == "private_requests" and value.get("event") == "request":
204                     if value["body"].get("robot_id") != "TEAM_REDACTED":
205                         raise ValueError(f"Missing sanitized request identity: {name}")
206                     if isinstance(value, dict) and "body_utf8" in value:
207                         check_identifiers(json.loads(value["body_utf8"]))
208             files[name] = path
209 if len(files) != 61:
210     raise ValueError("Expected 60 distinct practice files plus manifest")
211 return files
212
213
214 def verify_practice_inputs(root: Path):
215     files = practice_files(root / "practice_data")
216     analysis = json.loads((root / "results/official_practice_analysis.json").read_text())
217     actual = {name: hashlib.sha256(path.read_bytes()).hexdigest() for name, path in files.items()}
218     if actual != analysis["input_sha256"]:
219         raise ValueError("Official practice source hashes do not match analysis")
220     return actual
221
222 def verify_archive(archive: Path):
223     records = []
224     with tempfile.TemporaryDirectory(prefix="b-reproduce-") as directory:
225         extracted = Path(directory)
226         with zipfile.ZipFile(archive) as bundle:
227             for item in bundle.infolist():
228                 relative = PurePosixPath(item.filename)
229                 if relative.is_absolute() or "." in relative.parts:
230                     raise ValueError("Unsafe member in generated archive")
231             bundle.extractall(extracted)
232         manifest = json.loads((extracted / "MANIFEST.json").read_text())
233         for name, information in manifest["files"].items():
234             raw = (extracted / name).read_bytes()
235             assert hashlib.sha256(raw).hexdigest() == information["sha256"], name
236         verify_asset_inputs(extracted)
237         practice_hashes = verify_practice_inputs(extracted)
238         immutable_names = [name for name in manifest["files"]
239                             if name in SOURCES or name.startswith("practice_data/")
240                             or (name.startswith("report/") and name.endswith(".tex")
241                                 and not name.startswith("report/generated/"))]

```

```

242     expected_runs = {
243         "final_experiments.json": 4430,
244         "enhancement_experiments.json": 3080,
245         "ring_sweep.json": 2200,
246         "refinement_development.json": 2550,
247         "research_experiments.json": 10700,
248         "field_development.json": 2250,
249         "field_confirmation.json": 9400,
250     }
251     baselines = {
252         name: json.loads((extracted / "results" / name).read_text())
253         for name in (*expected_runs, "research_verification.json", "field_verification.json", "
official_practice_analysis.json", "fast_vs_compact_410.json", "practice_optimization_screen.json")
254     }
255     commands = [
256         [sys.executable, "verify_geometry.py"],
257         [sys.executable, "intersection.py"],
258         [sys.executable, "second_point.py"],
259         [
260             sys.executable,
261             "experiments.py",
262             "--cases",
263             "500",
264             "--output",
265             "results/final_experiments.json",
266         ],
267         [sys.executable, "enhancements.py"],
268         [sys.executable, "ring_sweep.py"],
269         [sys.executable, "verify_terminal.py"],
270         [sys.executable, "verify_refinements.py"],
271         [sys.executable, "verify_coverage_limits.py"],
272         [sys.executable, "compare_routes.py"],
273         [sys.executable, "compare_probes.py"],
274         [sys.executable, "verify_research.py"],
275         [sys.executable, "official_analysis.py", "--handoff", "practice_data"],
276         [sys.executable, "make_official_assets.py"],
277         [sys.executable, "fast_vs_compact_410.py"],
278         [sys.executable, "refinement_study.py", "--cases", "150", "--seed", "3100000"],
279         [sys.executable, "research_experiments.py"],
280         [sys.executable, "verify_field.py"],
281         [sys.executable, "field_study.py", "--cases", "150", "--seed", "6100000"],
282         [sys.executable, "field_confirmation.py"],
283         [sys.executable, "make_assets.py"],
284         [sys.executable, "make_enhancement_assets.py"],
285         [sys.executable, "make_research_assets.py"],
286         [sys.executable, "make_field_assets.py"],
287         [sys.executable, "redraw_q1q2.py"],
288         [sys.executable, "redraw_q34.py"],
289         [sys.executable, "redraw_results.py"],
290         [
291             "latexmk",
292             "-xelatex",
293             "-interaction=nonstopmode",
294             "-halt-on-error",
295             "paper.tex",
296         ],
297         [
298             "latexmk",
299             "-xelatex",
300             "-interaction=nonstopmode",
301             "-halt-on-error",
302             "ai_usage.tex",
303         ],
304     ]
305     environment = {key: value for key, value in os.environ.items() if key != "PYTHONPATH"}
306     environment["MPLBACKEND"] = "Agg"

```

```

307     for command in commands:
308         started = time.monotonic()
309         cwd = extracted / "report" if command[0] == "latexmk" else extracted
310         completed = subprocess.run(
311             command,
312             cwd=cwd,
313             text=True,
314             stdout=subprocess.PIPE,
315             stderr=subprocess.STDOUT,
316             timeout=1800,
317             check=False,
318             env=environment,
319         )
320         records.append(
321             {
322                 "command": [Path(command[0]).name, *command[1:]],
323                 "returncode": completed.returncode,
324                 "seconds": time.monotonic() - started,
325             }
326         )
327         if completed.returncode:
328             raise RuntimeError(
329                 f"Extracted archive command failed: {command}\n{completed.stdout[-12000:]}"
330             )
331     reproduced_runs = {}
332     auxiliary_runs = {}
333     for name, baseline in baselines.items():
334         replay = json.loads((extracted / "results" / name).read_text())
335         if name in expected_runs:
336             expected = expected_runs[name]
337             assert baseline["total_runs"] == replay["total_runs"] == expected, name
338             if name == "ring_sweep.json":
339                 count = sum(len(group["runs"]) for phase in ("development", "held_out")
340                             for group in replay[phase].values())
341             else:
342                 count = len(replay["runs"])
343                 for old, new in zip(baseline["runs"], replay["runs"], strict=True):
344                     assert deterministic_content(old) == deterministic_content(new), (name, old["seed"])
345             assert count == expected, (name, count)
346             reproduced_runs[name] = count
347         if name == "fast_vs_compact_410.json":
348             assert baseline["total_runs"] == replay["total_runs"] == 2000
349             for label in ("compact25", "fast25"):
350                 old_rows = [r for r in baseline["runs"] if r["configuration"] == label]
351                 new_rows = [r for r in replay["runs"] if r["configuration"] == label]
352                 assert [r["seed"] for r in old_rows] == list(range(4100000, 4101000))
353                 assert [r["seed"] for r in new_rows] == list(range(4100000, 4101000))
354                 for old, new in zip(old_rows, new_rows, strict=True):
355                     assert deterministic_content(old) == deterministic_content(new)
356             auxiliary_runs[name] = len(replay["runs"])
357             assert deterministic_content(baseline) == deterministic_content(replay), name
358         verify_asset_inputs(extracted)
359         assert verify_practice_inputs(extracted) == practice_hashes
360         for name in immutable_names:
361             assert hashlib.sha256((extracted / name).read_bytes()).hexdigest() == manifest["files"][name]["sha256"], name
362         assert sum(reproduced_runs.values()) == 34610
363         for filename in ("paper", "ai_usage"):
364             log = (extracted / "report" / f"{filename}.log").read_text(errors="replace")
365             assert "Missing character:" not in log and "Overfull" not in log
366             assert (
367                 "undefined references" not in log and "undefined citations" not in log
368             )
369         assert len(pymupdf.open(extracted / "report" / f"{filename}.pdf")) > 0
370     return {
371         "status": "passed",

```

```

372     "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
373     "manifest_hashes_verified": True,
374     "all_4430_behavioral_runs_reproduced_exactly": True,
375     "all_3080_enhancement_runs_reproduced_exactly": True,
376     "ring_sweep_reproduced_exactly": True,
377     "all_2200_ring_runs_reproduced_exactly": True,
378     "all_10700_research_runs_reproduced_exactly": True,
379     "all_2550_development_runs_reproduced_exactly": True,
380     "research_verification_reproduced_exactly": True,
381     "all_2250_field_development_runs_reproduced_exactly": True,
382     "all_9400_field_confirmation_runs_reproduced_exactly": True,
383     "field_verification_reproduced_exactly": True,
384     "practice_source_hashes_verified_before_and_after": practice_hashes,
385     "immutable_source_hashes_verified_after_replay": True,
386     "pythonpath_removed": True,
387     "official_practice_analysis_reproduced_exactly": True,
388     "fast_vs_compact_screen_reproduced_exactly": True,
389     "generated_asset_input_hashes_verified": True,
390     "experiment_runs_reproduced_by_dataset": reproduced_runs,
391     "experiment_runs_reproduced_total": sum(reproduced_runs.values()),
392     "auxiliary_runs_excluded_from_core_total": auxiliary_runs,
393     "excluded_nondeterminism": "local measured wall-clock real_duration_s, max_real_duration_s and
max_real_s only; official timestamp/runtime observations remain exact",
394     "commands": records,
395 }
396
397
398 def main():
399     parser = argparse.ArgumentParser()
400     parser.add_argument("--verify", action="store_true")
401     parser.add_argument("--handoff", type=Path, default=(ROOT / "practice_data" if (ROOT / "practice_data").
is_dir()
402                                     else ROOT.parent / "ai_optimization_handoff_20260912"))
403
404     args = parser.parse_args()
405     verify_asset_inputs(ROOT)
406     for prefix, name in (("paper", "paper"), ("ai", "ai_usage")):
407         audit = json.loads((ROOT / "report/visual" / f"{prefix}_audit.json").read_text())
408         actual = hashlib.sha256((ROOT / "report" / f"{name}.pdf").read_bytes()).hexdigest()
409         if audit["sha256"] != actual or audit["out_of_page_spans"] or audit["replacement_characters"]:
410             raise ValueError(f"Missing or stale PDF audit: {name}")
411         if prefix == "paper" and not (audit["abstract_keywords_on_first_page"]
and 0 < audit["body_pages_excluding_abstract"] <= 30):
412             raise ValueError("Paper abstract/body page rule violated")
413
414     environment = {
415         "python": sys.version,
416         "environment": "conda py314",
417         "packages": {
418             name: importlib.metadata.version(name)
419             for name in ("numpy", "scipy", "matplotlib", "pymupdf")
420         },
421     }
422     (ROOT / "results/environment_manifest.json").write_text(
423         json.dumps(environment, indent=2) + "\n"
424     )
425     files = {name: ROOT / name for name in SOURCES}
426     files.update({"results/" + name: ROOT / "results" / name for name in RESULTS})
427     supplied_practice = practice_files(args.handoff)
428     analysis = json.loads((ROOT / "results/official_practice_analysis.json").read_text())
429     practice_hashes = {name: hashlib.sha256(path.read_bytes()).hexdigest()
for name, path in supplied_practice.items()}
430
431     if practice_hashes != analysis["input_sha256"]:
432         raise ValueError("Supplied practice files do not match official analysis input hashes")
433     files.update({"practice_data/" + name: path for name, path in supplied_practice.items()})
434     files["AI工具使用详情.pdf"] = ROOT / "report/ai_usage.pdf"
435     for path in (ROOT / "report").iterdir():
436         if path.suffix in (".tex", ".pdf"):

```

```

436         files[path.relative_to(ROOT).as_posix()] = path
437     for folder in ("generated", "figures"):
438         for path in (ROOT / "report" / folder).iterdir():
439             if path.is_file() and not path.name.startswith("."):
440                 files[path.relative_to(ROOT).as_posix()] = path
441     for filename in ("paper_audit.json", "ai_audit.json"):
442         path = ROOT / "report" / "visual" / filename
443         files[path.relative_to(ROOT).as_posix()] = path
444     missing = [name for name, path in files.items() if not path.is_file()]
445     if missing:
446         raise RuntimeError(f"Deliverable proof is missing: {missing}")
447     anonymous_pattern = re.compile(r"/Users/[A-Za-z0-9._-]+/")
448     for name, path in files.items():
449         if path.suffix in (".py", ".txt", ".tex", ".json", ".jsonl", ".csv") and anonymous_pattern.search(
450             path.read_text(encoding="utf-8")
451         ):
452             raise ValueError(f"Personal absolute path in anonymous material: {name}")
453         if path.suffix == ".pdf":
454             document = pymupdf.open(path)
455             if anonymous_pattern.search(
456                 "\n".join(page.get_text() for page in document)
457             ):
458                 raise ValueError(f"Personal absolute path in PDF: {name}")
459     manifest = {
460         "status": "FORMAL_TEST_SUMMARIES_INTEGRATED_ENCRYPTED_LOGS_PRIVATE_MANUAL_REVIEW_PENDING",
461         "not_ready_for_contest_submission": True,
462         "official_practice": "Twenty supplied Q3/Q4 practice observations and six formal test summaries were
463         analyzed; six original encrypted logs remain in the private test directory",
464         "official_practice_replay_boundary": "All 20 supplied practice traces are independently costed; strict
465         baseline policy replay matches only 8/20. Formal summaries are user-provided and are not reconstructed
466         from encrypted payloads in the anonymous package.",
467         "practice_source_files": 60,
468         "default_strategy": "Q3 refined, Q4 field; explicit --strategy overrides",
469         "official_practice_control": "refined, probe_scale=0.22 for Q3 and Q4",
470         "human_review": "No actual contestant review record; AI review is not human review",
471         "python_environment": "conda py314",
472         "python_version": sys.version.split()[0],
473         "files": {
474             name: {
475                 "sha256": hashlib.sha256(path.read_bytes()).hexdigest(),
476                 "bytes": path.stat().st_size,
477             }
478             for name, path in sorted(files.items())
479         },
480     }
481     destination = ROOT / "delivery"
482     destination.mkdir(exist_ok=True)
483     archive = destination / "B题研究支撑材料.zip"
484     with zipfile.ZipFile(
485         archive, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9
486     ) as bundle:
487         for name, path in sorted(files.items()):
488             bundle.write(path, name)
489         bundle.writestr(
490             "MANIFEST.json", json.dumps(manifest, ensure_ascii=False, indent=2) + "\n"
491         )
492     with zipfile.ZipFile(archive) as bundle:
493         assert bundle.testzip() is None
494         for name, information in manifest["files"].items():
495             assert hashlib.sha256(bundle.read(name)).hexdigest() == information["sha256"], name
496     verify_asset_inputs(ROOT)
497     for name, path in supplied_practice.items():
498         assert hashlib.sha256(path.read_bytes()).hexdigest() == practice_hashes[name], name
499     shutil.copy2(ROOT / "report/paper.pdf", destination / "B题研究论文.pdf")
500     shutil.copy2(ROOT / "report/ai_usage.pdf", destination / "AI工具使用详情.pdf")
501     for path in (

```

```

499     archive,
500     destination / "B题研究论文.pdf",
501     destination / "AI工具使用详情.pdf",
502 ):
503     assert path.stat().st_size <= 20 * 2**20, path.name
504     status = {
505         "status": manifest["status"],
506         "official_tests": "FORMAL_TEST_SUMMARIES_INTEGRATED_LOGS_PRIVATE",
507         "human_review": "OUTSTANDING",
508         "source_files": len(files),
509         "archive_bytes": archive.stat().st_size,
510         "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
511     }
512     if args.verify:
513         proof = verify_archive(archive)
514         (destination / "复现验证.json").write_text(
515             json.dumps(proof, ensure_ascii=False, indent=2) + "\n"
516         )
517         status["extracted_archive_reproduction"] = "passed"
518     (destination / "交付状态.json").write_text(
519         json.dumps(status, ensure_ascii=False, indent=2) + "\n"
520     )
521     print(json.dumps(status, ensure_ascii=False, indent=2))
522
523
524 if __name__ == "__main__":
525     main()

```

Listing 9: 关键几何和状态核验

```

1
2
3 from __future__ import annotations
4 import json
5 import math
6 from pathlib import Path
7 import random
8
9 from submission.支撑材料.field_policy import FieldPlanner
10 from submission.支撑材料.geometry import _cross, clip_bearing, enclosing_circle, initial_polygon
11 from submission.支撑材料.negative_geometry import exclude_omni_disk
12 from submission.支撑材料.planner import Track
13 from submission.支撑材料.route_portfolio import _control_route, _cost, _improve, choose_portfolio_task
14 from submission.支撑材料.joint_dispatch_candidate import choose_task
15 from submission.支撑材料.verify_terminal import DeadlineAdapter
16
17
18 def contains(poly, p, tolerance=2e-6):
19     if not poly:
20         return False
21     if len(poly) == 1:
22         return math.dist(poly[0], p) <= tolerance
23     if len(poly) == 2:
24         a, b = poly
25         return abs(math.dist(a, p) + math.dist(b, p) - math.dist(a, b)) <= tolerance
26     return all(
27         _cross(poly[i], poly[(i + 1) % len(poly)], p)
28         >= -tolerance * max(1, math.dist(poly[i], poly[(i + 1) % len(poly)]))
29         for i in range(len(poly))
30     )
31
32
33 def negative_checks():
34     rng = random.Random(6311000)
35     count = 0

```

```

36     largest_cut = 0.0
37     for sides in (16, 32):
38         for index in range(500):
39             phi = rng.uniform(0, math.tau)
40             r = 1800 * math.sqrt(rng.random())
41             source = (r * math.cos(phi), r * math.sin(phi))
42             theta = rng.uniform(0, math.tau)
43             distance = rng.uniform(6, 1500)
44             station = (
45                 source[0] - distance * math.cos(theta),
46                 source[1] - distance * math.sin(theta),
47             )
48             poly = clip_bearing(
49                 initial_polygon(),
50                 station,
51                 (math.degrees(theta) + rng.uniform(-1, 1)) % 360,
52             )
53             for j in range(3):
54                 a = rng.uniform(0, math.tau)
55                 gap = 1000 + 1e-8 if index % 3 == 0 else rng.uniform(1000, 2500)
56                 negative = (
57                     source[0] + gap * math.cos(a),
58                     source[1] + gap * math.sin(a),
59                 )
60                 before = poly.copy()
61                 poly = exclude_omni_disk(poly, negative, sides=sides)
62                 assert before and poly and contains(poly, source), (
63                     sides,
64                     index,
65                     source,
66                     negative,
67                 )
68                 assert all(contains(before, v) for v in poly)
69                 largest_cut = max(
70                     largest_cut, enclosing_circle(before)[1] - enclosing_circle(poly)[1]
71                 )
72                 count += 1
73                 assert exclude_omni_disk([(1000.0, 0.0)], (0.0, 0.0), sides=sides) == [
74                     (1000.0, 0.0)
75                 ]
76                 assert exclude_omni_disk([(0.0, 0.0)], (0.0, 0.0), sides=sides) == []
77                 segment = [(-1200.0, 0.0), (1200.0, 0.0)]
78                 assert exclude_omni_disk(segment, (0.0, 0.0), sides=sides) == segment
79                 for shift in ((2e6, -2e6), (0.0, 0.0)):
80                     p = [
81                         (shift[0] + 1200, shift[1] - 5),
82                         (shift[0] + 1400, shift[1] - 5),
83                         (shift[0] + 1400, shift[1] + 5),
84                     ]
85                     assert exclude_omni_disk(p, shift, sides=sides) == p
86             try:
87                 exclude_omni_disk([(0.0, 0.0)], (1e308, 1e308), radius=1e308)
88             except ValueError:
89                 pass
90             else:
91                 raise AssertionError("Unrepresentable half-plane arithmetic must be rejected")
92         return {
93             "true_source_containment_cases": count,
94             "boundary_and_degenerate_cases": 10,
95             "maximum_observed_radius_cut_m": largest_cut,
96             "scope": "sampling complements finite inscribed-polygon exclusion proof",
97         }
98
99
100 def route_checks():
101     rng = random.Random(6312000)

```



```

102     cases = 0
103     for n in (1, 2, 3, 7, 25, 41, 51):
104         for iteration in range(10):
105             pts = [
106                 (rng.uniform(-2500, 2500), rng.uniform(-2500, 2500)) for _ in range(n)
107             ]
108             pos = (rng.uniform(-1000, 1000), rng.uniform(-1000, 1000))
109             penalty = [rng.uniform(0, 750) for _ in pts]
110             origin = [math.dist(pos, p) for p in pts]
111             edges = [[math.dist(a, b) for b in pts] for a in pts]
112             control = _control_route(origin, edges, penalty)
113             improved, cost = _improve(control, origin, edges, penalty)
114             assert sorted(improved) == list(range(n)) and cost <= _cost(
115                 control, origin, edges, penalty
116             )
117             targets = {i: (p, penalty[i]) for i, p in enumerate(pts)}
118             assert choose_task(pos, {}, targets, choice="guarded") == (
119                 "target",
120                 control[0],
121             )
122             assert choose_portfolio_task(pos, {}, targets)[1] in targets
123             cases += 1
124
125     for positions in (
126         ((1.0, 0.0), (2.0, 0.0)),
127         ((-1.0, 0.0), (2.0, 0.0)),
128         ((0.0, 0.0), (0.0, 0.0)),
129     ):
130         for risk in (0.0, 1.0, 2.0):
131             targets = {i: (point, float(i * 4)) for i, point in enumerate(positions)}
132             selected = choose_portfolio_task((0.0, 0.0), {}, targets, risk)
133             costs = {
134                 i: math.dist((0, 0), positions[i])
135                 + math.dist(*positions)
136                 + risk * targets[i][1]
137                 for i in range(2)
138             }
139             assert costs[selected[1]] == min(costs.values())
140     return {
141         "random_route_cases": cases,
142         "exact_two_task_cases": 9,
143         "control_never_worsened": True,
144     }
145
146
147 def clearing_checks():
148     rng = random.Random(6313000)
149     count = 0
150     saved = 0.0
151     for _ in range(200):
152         env = DeadlineAdapter(max_virtual=360000)
153         policy = FieldPlanner(env, 3, approach_clear=True)
154         policy.action("/enter")
155         c = (rng.uniform(-1500, 1500), rng.uniform(-1500, 1500))
156         radius = rng.uniform(0.01, 19.8)
157         poly = [
158             (
159                 c[0] + radius * math.cos(k * math.pi / 3),
160                 c[1] + radius * math.sin(k * math.pi / 3),
161             )
162             for k in range(6)
163         ]
164         policy.tracks[1] = Track(polygon=poly)
165         policy.channel_state[1] = "detected"
166         original = list(poly)
167         before = math.dist(policy.position, c)

```

```

168     assert policy.clear(1, c, guaranteed=True)
169     assert all(math.dist(v, policy.position) <= 19.9 for v in original)
170     assert math.dist((0, 0), policy.position) <= before + 1e-7
171     saved += policy.clear_access_saved_m
172     count += 1
173     for p in (3, 4):
174         env = DeadlineAdapter(remaining=1, max_virtual=360000)
175         result = FieldPlanner(env, p).run()
176         assert not result["certificate_complete"] and "/exit" not in env.paths
177     return {
178         "clearing_regions": count,
179         "shortened_approach_total_m": saved,
180         "real_deadline_cases": 2,
181     }
182
183
184 def main():
185     result = {
186         "evidence": "local_field_geometry_and_behavior_verification_not_official",
187         "negative_geometry": negative_checks(),
188         "routes": route_checks(),
189         "clearing": clearing_checks(),
190     }
191     Path("results/field_verification.json").write_text(
192         json.dumps(result, indent=2) + "\n"
193     )
194     print(json.dumps(result, indent=2))
195
196
197 if __name__ == "__main__":
198     main()

```

Listing 10: 竞赛版图表生成

```

1
2 from __future__ import annotations
3
4 import hashlib
5 import json
6 import math
7 from pathlib import Path
8
9 import matplotlib
10 matplotlib.use("Agg")
11 import matplotlib.pyplot as plt
12 from matplotlib.patches import Circle, Polygon, Rectangle, FancyArrowPatch
13 import numpy as np
14
15 ROOT = Path(__file__).resolve().parent
16 FIGURES = ROOT / "report" / "figures"
17 GENERATED = ROOT / "report" / "generated"
18 BLUE, ORANGE, GREEN, GREY = "#0072B2", "#E69F00", "#009E73", "#666666"
19
20
21 def save(fig, name):
22     FIGURES.mkdir(parents=True, exist_ok=True)
23     fig.savefig(FIGURES / name, format="pdf", metadata={"Creator": "CUMCM competition assets"})
24     plt.close(fig)
25
26
27 def base(title):
28     fig, ax = plt.subplots(figsize=(16 / 2.54, 7 / 2.54), layout="constrained")
29     ax.set_title(title, fontsize=11, pad=5)
30     ax.axis("off")
31     return fig, ax

```

```

32
33
34 def taskmap():
35     fig, ax = base("四问：从几何覆盖到定向定位")
36     xs = [0.08, .31, .54, .77]; labels = ["Q1\n几何边界", "Q2\n接收盘与覆盖", "Q3\n全向清除", "Q4\n定向定位"]
37     for i, (x, lab) in enumerate(zip(xs, labels)):
38         ax.add_patch(Rectangle((x, .38), .15, .25, facecolor="#EAF2F8" if i < 2 else "#FFF2DF", edgecolor=BLUE
39             if i < 2 else ORANGE, lw=1.5))
40         ax.text(x + .075, .505, lab, ha="center", va="center", fontsize=9)
41         if i < 3: ax.add_patch(FancyArrowPatch((x+.15, .505), (xs[i+1], .505), arrowstyle="->", lw=1.4, color=GREY))
42     ax.text(.5, .18, "可行域 → 最小包围盘 → 扫描停止 → 局部测向与光学覆盖", ha="center", fontsize=8, color=GREY)
43     save(fig, "competition_taskmap.pdf")
44
45 def ring():
46     fig, ax = base("Q3 环形布点")
47     ax.set_aspect("equal"); ax.set_xlim(-2050,2050); ax.set_ylim(-2050,2050)
48     ax.add_patch(Circle((0,0),1800,fill=False,ls="--",color=GREY,lw=1))
49     u=np.array([np.cos(np.pi/6),np.sin(np.pi/6)]); g=1800*u; p=np.array([1200.,0.]);
50     angles=np.arange(6)*np.pi/3; pts=np.c_[1200*np.cos(angles),1200*np.sin(angles)]
51     ax.plot(*np.vstack([pts,pts[0]]).T,color=ORANGE,lw=1.5); ax.scatter(pts[:,0],pts[:,1],s=20,color=ORANGE,
52         zorder=3)
53     g=np.array([1800*np.cos(np.pi/6),1800*np.sin(np.pi/6)]); nearest=1200*np.array([1.,0.]); ax.scatter(*g,
54         color="#D55E00",marker="x",s=45,zorder=4); ax.plot([g[0],nearest[0]],[g[1],nearest[1]],color=GREEN,lw=2)
55     ax.annotate("最坏源点",xy=g,xytext=(g[0]+120,g[1]+120),fontsize=8); ax.annotate("968.90 m, 余量31.10 m",xy
56         =((g[0]+nearest[0])*5,(g[1]+nearest[1])*5),fontsize=8,color=GREEN)
57     for c in pts: ax.add_patch(Circle(c,1000,fill=False,color=BLUE,lw=.8,alpha=.65))
58     ax.text(-1750,-1930,"7 个接收盘 (半径 1000 m) ; 6 环点 + 原点",fontsize=8,color=BLUE)
59     save(fig,"competition_ring.pdf")
60
61 def direction():
62     fig, axes = plt.subplots(1,2,figsize=(16/2.54,7/2.54),layout="constrained")
63     for ax in axes: ax.set_aspect("equal"); ax.axis("off")
64     ax=axes[0]; ax.add_patch(Circle((0,0),1,fill=False,color=BLUE)); g=np.array([.7,.7]); ax.scatter(*g,color=
65         ORANGE); ax.arrow(*g,.35,.35,width=.015,color=ORANGE); ax.fill_between([-1,.9],[-.9,.9],color=ORANGE,alpha
66         =.08); ax.text(-.9,-1.15,"g 在源圆周; 前向半平面朝外",fontsize=8)
67     ax=axes[1]; tri=np.array([-1.8,-.6],[.75,-.5],[.05,.65]); ax.add_patch(Polygon(tri,fill=False,color=BLUE,
68         lw=1.5)); g=np.array([-1.05,.25]); n=np.array([.45,.25]); h=g+5*n; ax.scatter(*g,color=ORANGE); ax.scatter
69         (*h,color=GREEN); c=tri.mean(0); ax.scatter(*c,color=GREY); ax.plot([c[0],0],[c[1],-.95],"--",color=GREEN)
70         ; ax.text(-.9,-1.15,"含 h=g+5n 的三角形重心投影",fontsize=8); ax.text(*g," g",fontsize=8); ax.text(*h," h",
71             fontsize=8)
72     fig.text(.5,.98,"Q4 方向性与双环 (原理示意, 非观测轨迹)",ha="center",va="top",fontsize=10); save(fig,"
73         competition_direction.pdf")
74
75 def localize():
76     fig, axes = plt.subplots(1,2,figsize=(16/2.54,7/2.54),layout="constrained")
77     for ax in axes: ax.set_aspect("equal"); ax.axis("off")
78     ax=axes[0]; n=np.array([0.,1.]); z=np.array([-1.6,0.]); delta=.18; ax.plot([-1,1],[0,0],color=BLUE); ax.plot
79         ([-1,1],[delta,delta],"--",color=ORANGE); ax.plot([-1,1],[-delta,-delta],"--",color=ORANGE); ax.scatter(np
80             .linspace(-.8,.8,5),np.zeros(5),color=BLUE,s=16); ax.arrow(*z,*n,width=.01,color=GREEN); ax.text
81             (-.95,-.95,"横向测点: z plus/minus delta n",fontsize=8)
82     ax=axes[1]; ax.add_patch(Rectangle((-1.9,-.65),1.8,1.3,fill=False,color=BLUE)); xx=np.linspace(-.75,.75,5);
83     yy=np.linspace(-.5,.5,4); X,Y=np.meshgrid(xx,yy)
84     for x,y in zip(X.flat,Y.flat): ax.add_patch(Circle((x,y),.12,fill=False,color=ORANGE,lw=.7))
85     ax.scatter(X,Y,s=10,color=ORANGE); ax.text(-.9,-.95,"半径 19.9 m 圆覆盖矩形网格 (2 面板)",fontsize=8)
86     fig.text(.5,.98,"局部测向与光学覆盖 (原理示意)",ha="center",va="top",fontsize=10); save(fig,"
87         competition_localize.pdf")
88
89 def dispatch():
90     fig, ax = base("Q3 扫描闭环")
91     ax.text(.5,.78,"公开观测",ha="center",fontsize=9); ax.add_patch(FancyArrowPatch((.5,.72),(.5,.58),
92         arrowstyle="->",color=GREY))
93     ax.text(.5,.52,"选择扫描 / 定位",ha="center",fontsize=9); ax.add_patch(FancyArrowPatch((.5,.46),(.5,.31),

```

```

arrowstyle="->",color=GREY))
81 ax.text(.5,.24,"清除频道并更新骨架",ha="center",fontsize=9); ax.add_patch(FancyArrowPatch((.5,.18),(.3,.08),
connectionstyle="arc3,rad=.3",arrowstyle="->",color=GREY))
82 ax.text(.3,.03,"继续循环",ha="center",fontsize=8,color=BLUE); ax.text(.76,.08,"停止: 各频道已清除 / 全骨架无
信号\n或 16 个不同频道已清除",ha="center",fontsize=8,color=GREEN)
83 save(fig,"competition_dispatch.pdf")
84
85
86 def costs():
87     d=json.loads((ROOT/"results/field_confirmation.json").read_text(encoding="utf-8"))
88     rows=[r for r in d["runs"] if 7100000<=int(r["seed"])<=7100999]
89     assert len(rows)==6000
90     groups=[]
91     for p in (3,4):
92         for cfg in (("control","控制"),("field","field")):
93             rr=[r for r in rows if r["problem"]==p and r["configuration"]==cfg[0]]; assert len(rr)==1000
94             vals=[]
95             for r in rr:
96                 op=5*r["measures"]+r["switches"]+3*r["failed_clears"]+5*r["n_cleared"]
97                 move=r["virtual_time_s"]-op
98                 assert math.isclose(move,r["distance_m"]/5,abs_tol=1e-5)
99                 vals.append((move/r["n_cleared"],op/r["n_cleared"]))
100             groups.append((f"Q{p} {cfg[1]}",np.mean(vals,axis=0)))
101 fig,ax=plt.subplots(figsize=(16/2.54,7/2.54),layout="constrained"); x=np.arange(4); mv=np.array([g[1][0]
for g in groups]); op=np.array([g[1][1] for g in groups]); ax.bar(x,mv,color=BLUE,label="移动"); ax.bar(x,
op,bottom=mv,color=ORANGE,label="操作"); ax.set_xticks(x,[g[0] for g in groups],fontsize=8); ax.set_ylabel
("T/K (s)"); ax.set_title("主组成本分解",fontsize=10); ax.legend(fontsize=8); ax.grid(axis="y",alpha=.2);
save(fig,"competition_costs.pdf")
102 def main():
103     plt.rcParams.update({"font.size":9,"font.family":"sans-serif","font.sans-serif":["FandolSong","Arial
Unicode MS","PingFang SC","DejaVu Sans"],"pdf.fonttype":42})
104     for f in (taskmap,ring,direction,localize,dispatch,costs): f()
105     inputs=[ROOT/"results/field_confirmation.json",ROOT/"geometry.py",ROOT/"coverage.py",Path(__file__)]
106     meta={"generator":str(Path(__file__).relative_to(ROOT)),"figures":[f"figures/competition_{n}.pdf" for n in
("taskmap","ring","direction","localize","dispatch","costs")],"input_sha256":{str(p.relative_to(ROOT)):
hashlib.sha256(p.read_bytes()).hexdigest() for p in inputs}}
107     GENERATED.mkdir(parents=True,exist_ok=True); (GENERATED/"competition_metadata.json").write_text(json.dumps(
meta,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")
108
109 if __name__ == "__main__": main()

```

Listing 11: 任务调度与候选策略

```

1
2
3 from __future__ import annotations
4
5 from math import dist, isfinite
6
7 Point = tuple[float, float]
8 Task = tuple[str, int]
9
10
11 def choose_task(
12     position: Point,
13     anchors: dict[int, Point],
14     targets: dict[int, tuple[Point, float]],
15     *,
16     choice: str = "guarded",
17     risk_weight: float = 1.0,
18 ) -> Task:
19
20     if choice not in ("center", "guarded"):
21         raise ValueError("choice must be 'center' or 'guarded'")
22     tasks: list[Task] = [("target", key) for key in sorted(targets)]

```

```

23 tasks.extend(("anchor", key) for key in sorted(anchors))
24 n = len(tasks)
25 if not n:
26     raise ValueError("cannot dispatch an empty task set")
27
28 coords = [targets[key][0] for key in sorted(targets)]
29 coords.extend(anchors[key] for key in sorted(anchors))
30 radii = [targets[key][1] for key in sorted(targets)]
31 radii.extend(0.0 for _ in anchors)
32 if any(len(point) != 2 or not all(isfinite(x) for x in point)
33       for point in [position, *coords]):
34     raise ValueError("positions must be finite two-dimensional points")
35 if any(not isfinite(radius) or radius < 0 for radius in radii):
36     raise ValueError("MEC radii must be finite and nonnegative")
37 if not isfinite(risk_weight) or risk_weight < 0:
38     raise ValueError("risk_weight must be finite and nonnegative")
39 penalty = [risk_weight * radius for radius in radii] if choice == "guarded" else [0.0] * n
40 origin = [dist(position, point) for point in coords]
41 edges = [[0.0] * n for _ in range(n)]
42 for i in range(n):
43     for j in range(i):
44         edges[i][j] = edges[j][i] = dist(coords[i], coords[j])
45 if (any(not isfinite(value) for value in origin)
46     or any(not isfinite(value) for row in edges for value in row)):
47     raise ValueError("pairwise distances must be finite")
48
49
50
51 first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
52 route = [first]
53 remaining = set(range(n))
54 remaining.remove(first)
55 nearest = [min(origin[i], edges[first][i]) for i in range(n)]
56 while remaining:
57     node = min(remaining, key=lambda i: (nearest[i], i))
58     best_gap = 0
59     best_delta = (origin[node] + edges[node][route[0]] - origin[route[0]]
60                  + penalty[node] - penalty[route[0]])
61     for gap in range(1, len(route) + 1):
62         before = route[gap - 1]
63         delta = edges[before][node]
64         if gap < len(route):
65             after = route[gap]
66             delta += edges[node][after] - edges[before][after]
67         if delta < best_delta:
68             best_gap, best_delta = gap, delta
69     route.insert(best_gap, node)
70     remaining.remove(node)
71     for other in remaining:
72         nearest[other] = min(nearest[other], edges[node][other])
73
74
75
76
77
78 for _ in range(n):
79     changed = False
80     for left in range(n - 1):
81         for right in range(left + 1, n):
82             old = (origin[route[left]] if left == 0
83                   else edges[route[left - 1]][route[left]])
84             new = (origin[route[right]] if left == 0
85                   else edges[route[left - 1]][route[right]])
86             if right + 1 < n:
87                 old += edges[route[right]][route[right + 1]]
88                 new += edges[route[left]][route[right + 1]]

```

```

89         if left == 0:
90             old += penalty[route[left]]
91             new += penalty[route[right]]
92             if new + 1e-12 * max(1.0, old, new) < old:
93                 route[left:right + 1] = reversed(route[left:right + 1])
94                 changed = True
95                 break
96         if changed:
97             break
98         if not changed:
99             break
100     return tasks[route[0]]

```

Listing 12: Q3 负观测几何

```

1
2
3 from __future__ import annotations
4
5 from math import cos, isfinite, pi, sin
6
7 from submission.支撑材料.geometry import Point, _clip_halfplane, _cross, _padding
8
9
10 def _hull(points: list[Point]) -> list[Point]:
11
12     pts = sorted(set(points))
13     if len(pts) <= 2:
14         return pts
15
16     lower: list[Point] = []
17     for p in pts:
18         while len(lower) >= 2 and _cross(lower[-2], lower[-1], p) <= 0.0:
19             lower.pop()
20         lower.append(p)
21     upper: list[Point] = []
22     for p in reversed(pts):
23         while len(upper) >= 2 and _cross(upper[-2], upper[-1], p) <= 0.0:
24             upper.pop()
25         upper.append(p)
26     return lower[:-1] + upper[:-1]
27
28
29 def exclude_omni_disk(
30     poly: list[Point], station: Point, radius: float = 1000.0, *, sides: int = 16
31 ) -> list[Point]:
32
33     if sides not in (16, 32):
34         raise ValueError("sides must be 16 or 32")
35     if len(station) != 2 or not all(isfinite(v) for v in station):
36         raise ValueError("station must have finite coordinates")
37     if not isfinite(radius) or radius <= 0.0:
38         raise ValueError("radius must be a positive finite number")
39     if not poly:
40         return []
41     if any(len(p) != 2 or not all(isfinite(v) for v in p) for p in poly):
42         raise ValueError("polygon requires finite coordinates")
43     scale = max(1.0, radius, *(abs(v) for p in poly for v in p), *map(abs, station))
44     if not isfinite(16.0 * scale * scale):
45         raise ValueError("Geometry scale exceeds finite half-plane arithmetic")
46
47
48
49     qradius = radius * cos(pi / sides)
50     cx, cy = station

```

```

51 q = [
52     (
53         cx + qradius * cos(2.0 * pi * i / sides),
54         cy + qradius * sin(2.0 * pi * i / sides),
55     )
56     for i in range(sides)
57 ]
58 pad = _padding(scale)
59 pieces: list[Point] = []
60
61
62
63 for i, a in enumerate(q):
64     b = q[(i + 1) % sides]
65     ex, ey = b[0] - a[0], b[1] - a[1]
66     normal = (ey, -ex)
67     bound = 0.0
68
69     if all(
70         normal[0] * (x - a[0]) + normal[1] * (y - a[1]) >= -pad for x, y in poly
71     ):
72         return list(poly)
73     part = _clip_halfplane(poly, a, (-normal[0], -normal[1]), -bound, pad)
74     pieces.extend(part)
75 if not pieces:
76     return []
77 return _hull(pieces)

```

Listing 13: Q3Q4 实验运行

```

1
2
3 from __future__ import annotations
4 import argparse
5 import json
6 import math
7 import statistics
8 from pathlib import Path
9 from submission.支撑材料.environment import LocalSimulator
10 from submission.支撑材料.field_policy import FieldPlanner
11 from submission.支撑材料.planner import Planner
12
13 CONFIGS = {
14     "control": None,
15     "premeasure": {"premeasure": True},
16     "negative": {"premeasure": False, "negative_updates": True},
17     "pre_negative": {"premeasure": True, "negative_updates": True},
18     "risk2": {"premeasure": True, "risk_weight": 2.0},
19     "risk4": {"premeasure": True, "risk_weight": 4.0},
20     "risk8": {"premeasure": True, "risk_weight": 8.0},
21     "portfolio": {"premeasure": True, "portfolio": True},
22     "portfolio_negative": {
23         "premeasure": True,
24         "portfolio": True,
25         "negative_updates": True,
26     },
27     "portfolio_risk2": {"premeasure": True, "portfolio": True, "risk_weight": 2.0},
28     "portfolio_clear": {"premeasure": True, "portfolio": True, "approach_clear": True},
29     "all": {
30         "premeasure": True,
31         "portfolio": True,
32         "negative_updates": True,
33         "approach_clear": True,
34     },
35 }

```

```

36
37
38 def run_field(seed, problem, config, scenario="random", mode="smooth"):
39     env = LocalSimulator(seed, problem, mode, scenario)
40     settings = {
41         "premeasure": False,
42         "negative_updates": False,
43         "portfolio": False,
44         "approach_clear": False,
45     }
46     planner = (
47         Planner(env, problem, probe_scale=0.22)
48         if config is None
49         else FieldPlanner(env, problem, **(settings | config))
50     )
51     result = planner.run()
52     actual = env.summary()
53     assert (
54         result["certificate_complete"] and actual["n_cleared"] == actual["n_sources"]
55     ), (seed, problem, result, actual)
56     pos = (0.0, 0.0)
57     radio = 1
58     us = 0
59     successes = set()
60     for event in env.log:
61         request, response = event["request"], event["response"]
62         assert response["accepted"]
63         if request["path"] in ("/measure", "/clear"):
64             p = request["position"]["x"], request["position"]["y"]
65             us += round(math.dist(pos, p) / 5 * 1000000)
66             pos = p
67             if request["path"] == "/measure":
68                 us += (5 + int(radio != request["channel"])) * 1000000
69                 radio = request["channel"]
70             else:
71                 success = response["clear_result"] == "success"
72                 us += (5 if success else 3) * 1000000
73                 if success:
74                     assert request["channel"] not in successes
75                     successes.add(request["channel"])
76         assert abs(us / 1000000 - response["virtual_time_s"]) < 2e-6
77     assert len(successes) == actual["n_sources"]
78     return {
79         **actual,
80         "certificate_complete": True,
81         "timing_independently_verified": True,
82         "zero_travel_probes": result.get("zero_travel_probes", 0),
83         "negative_cuts": result.get("negative_cuts", 0),
84     }
85
86
87 def pair_stats(control, candidate):
88     assert [r["seed"] for r in control] == [r["seed"] for r in candidate]
89     diffs = [
90         a["average_time_s"] - b["average_time_s"]
91         for a, b in zip(control, candidate, strict=True)
92     ]
93     mean = statistics.mean(diffs)
94     se = statistics.stdev(diffs) / math.sqrt(len(diffs))
95     return {
96         "mean_saved_s": mean,
97         "saving_pct": 100
98         * mean
99         / statistics.mean(r["average_time_s"] for r in control),
100         "approx_95pct_ci": [mean - 1.96 * se, mean + 1.96 * se],
101         "faster_cases": sum(d > 0 for d in diffs),

```



```

102     }
103
104
105 def summarize(group):
106     values = sorted(r["average_time_s"] for r in group)
107     return {
108         "cases": len(group),
109         "all_clear": sum(r["fraction_cleared"] == 1 for r in group),
110         "mean": statistics.mean(values),
111         "p90": values[math.ceil(0.9 * len(values)) - 1],
112         "worst": max(values),
113         "distance": statistics.mean(r["distance_m"] for r in group),
114         "measures": statistics.mean(r["measures"] for r in group),
115         "mean_failed_clear_attempts_per_case": statistics.mean(
116             r["failed_clears"] for r in group
117         ),
118         "max_real_s": max(r["real_duration_s"] for r in group),
119     }
120
121
122 def main():
123     parser = argparse.ArgumentParser()
124     parser.add_argument("--cases", type=int, default=150)
125     parser.add_argument("--seed", type=int, default=6100000)
126     parser.add_argument(
127         "--output", type=Path, default=Path("results/field_development.json")
128     )
129     args = parser.parse_args()
130     summaries = {}
131     rows = []
132     for p in (3, 4):
133         names = (
134             (
135                 "control",
136                 "premeasure",
137                 "negative",
138                 "pre_negative",
139                 "portfolio",
140                 "portfolio_negative",
141                 "all",
142             )
143             if p == 3
144             else (
145                 "control",
146                 "premeasure",
147                 "risk2",
148                 "risk4",
149                 "risk8",
150                 "portfolio",
151                 "portfolio_risk2",
152                 "portfolio_clear",
153             )
154         )
155         control = None
156         for name in names:
157             group = [
158                 run_field(seed, p, CONFIGS[name])
159                 for seed in range(args.seed, args.seed + args.cases)
160             ]
161             for row in group:
162                 row["configuration"] = name
163             rows.extend(group)
164             if control is None:
165                 control = group
166             summaries[f"Q{p}_{name}"] = {
167                 **summarize(group),

```

```

168         **pair_stats(control, group),
169     }
170     print(f"Q{p}_{name}", json.dumps(summaries[f"Q{p}_{name}"]), flush=True)
171     args.output.write_text(
172         json.dumps(
173             {
174                 "evidence": "field_informed_local_development_not_official",
175                 "seed_range": [args.seed, args.seed + args.cases - 1],
176                 "configs": CONFIGS,
177                 "summary": summaries,
178                 "total_runs": len(rows),
179                 "runs": rows,
180             },
181             ensure_ascii=False,
182             indent=2,
183         )
184         + "\n"
185     )
186
187
188 if __name__ == "__main__":
189     main()

```

Listing 14: 问题一交会计算

```

1
2
3 from __future__ import annotations
4
5 import itertools
6 import json
7 import math
8 from pathlib import Path
9
10 import numpy as np
11 from scipy.optimize import linprog
12
13 from submission.支撑材料.geometry import diameter, enclosing_circle
14
15
16 def solve_intersection(stations, bearings_deg, error_deg=1.0):
17     if len(stations) != len(bearings_deg) or not 0 <= error_deg < 90:
18         raise ValueError("Match stations/readings and use an error in [0,90) degrees")
19     if not stations:
20         return {"status": "unbounded", "vertices": [], "diameter_m": None}
21     rows, rhs = [], []
22     alpha = math.radians(error_deg)
23     for (x, y), angle in zip(stations, bearings_deg):
24         if not all(math.isfinite(value) for value in (x, y, angle)):
25             raise ValueError("Finite coordinates and bearings are required")
26         theta = math.radians(angle % 360)
27         for normal in (
28             (math.sin(theta - alpha), -math.cos(theta - alpha)),
29             (-math.sin(theta + alpha), math.cos(theta + alpha)),
30             (-math.cos(theta), -math.sin(theta)),
31         ):
32             rows.append(normal)
33             rhs.append(normal[0] * x + normal[1] * y)
34     a, b = np.asarray(rows), np.asarray(rhs)
35     feasible = linprog(
36         np.zeros(2), A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
37     )
38     if feasible.status == 2:
39         return {"status": "infeasible", "vertices": [], "diameter_m": None}
40     if not feasible.success:

```

```

41     raise RuntimeError(f"Halfplane feasibility LP failed: {feasible.message}")
42 for objective in ((1.0, 0.0), (-1.0, 0.0), (0.0, 1.0), (0.0, -1.0)):
43     bound = linprog(
44         objective, A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
45     )
46     if bound.status == 3:
47         return {"status": "unbounded", "vertices": [], "diameter_m": None}
48     if not bound.success:
49         raise RuntimeError(f"Halfplane boundedness LP failed: {bound.message}")
50 vertices = []
51 for i, j in itertools.combinations(range(len(a)), 2):
52     determinant = a[i, 0] * a[j, 1] - a[i, 1] * a[j, 0]
53     if determinant == 0:
54         continue
55     point = (
56         (b[i] * a[j, 1] - a[i, 1] * b[j]) / determinant,
57         (a[i, 0] * b[j] - b[i] * a[j, 0]) / determinant,
58     )
59     if np.max(a @ point - b) <= 1e-7 and all(
60         math.dist(point, old) > 1e-7 for old in vertices
61     ):
62         vertices.append(point)
63 if not vertices:
64     raise ArithmeticError(
65         "Bounded nonempty intersection had no numerical vertex; inspect conditioning"
66     )
67 center = tuple(np.mean(vertices, axis=0))
68 vertices.sort(
69     key=lambda point: math.atan2(point[1] - center[1], point[0] - center[0])
70 )
71 circle = enclosing_circle(vertices)
72 return {
73     "status": "bounded",
74     "vertices": vertices,
75     "diameter_m": diameter(vertices),
76     "mec_center": circle[0],
77     "mec_radius_m": circle[1],
78     "max_halfplane_residual_m": max(float(np.max(a @ p - b)) for p in vertices),
79 }
80
81
82 def main():
83     triangle = [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))]
84     length = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
85     stations = []
86     for first, second in zip(triangle, triangle[1:] + triangle[:1]):
87         stations.append(
88             (
89                 first[0] - length * (second[0] - first[0]) / 36,
90                 first[1] - length * (second[1] - first[1]) / 36,
91             )
92         )
93     bounded = solve_intersection(stations, [1, 121, 241])
94     unbounded = solve_intersection([(0.0, 0.0)], [0.0])
95     infeasible = solve_intersection([(0.0, 0.0), (-10.0, 0.0)], [0.0, 180.0])
96     assert bounded["status"] == "bounded" and abs(bounded["diameter_m"] - 36) < 1e-6
97     assert unbounded["status"] == "unbounded" and infeasible["status"] == "infeasible"
98     result = {
99         "evidence": "Q1_direct_halfplane_algorithm_examples",
100         "bounded": bounded,
101         "unbounded": unbounded,
102         "infeasible": infeasible,
103     }
104     Path("results").mkdir(exist_ok=True)
105     Path("results/intersection_verification.json").write_text(
106         json.dumps(result, indent=2) + "\n"

```

```

107     )
108     print(json.dumps(result, indent=2))
109
110
111 if __name__ == "__main__":
112     main()

```

Listing 15: 问题二测点示例

```

1
2
3 from __future__ import annotations
4
5 import json
6 import math
7 from pathlib import Path
8
9 from submission.支撑材料.geometry import clip_bearing, enclosing_circle, initial_polygon
10
11
12 def safe_candidates(station=(0.0, 0.0), bearing_deg=0.0, alpha_deg=1.005):
13     alpha, angle = math.radians(alpha_deg), math.radians(bearing_deg)
14     u, n = (math.cos(angle), math.sin(angle)), (-math.sin(angle), math.cos(angle))
15     choices = []
16     for t in (125.0, 250.0, 375.0, 500.0, 625.0, 750.0, 875.0):
17         bound = min(
18             math.sqrt(1000**2 - t * t),
19             math.sqrt(1000**2 - (t - 1000 * math.cos(alpha)) ** 2)
20             - 1000 * math.sin(alpha),
21         )
22         for proportion in (0.25, 0.5, 0.75):
23             for sign in (-1, 1):
24                 b = sign * proportion * bound
25                 point = (
26                     station[0] + t * u[0] + b * n[0],
27                     station[1] + t * u[1] + b * n[1],
28                 )
29                 choices.append({"point": point, "t_m": t, "b_m": b})
30     return choices
31
32
33 def example():
34
35     first = clip_bearing(initial_polygon(), (0.0, 0.0), 0.0)
36     source_samples = [
37         (r * math.cos(math.radians(phi)), r * math.sin(math.radians(phi)))
38         for r in range(25, 1501, 25)
39         for phi in (-1.0, 0.0, 1.0)
40     ]
41     output = []
42     choices = safe_candidates() + [{"point": (750.0, 0.0), "t_m": 750.0, "b_m": 0.0}]
43     for candidate in choices:
44         q = candidate["point"]
45         worst_radius = 0.0
46         for source in source_samples:
47             assert math.dist(q, source) <= max(1000, math.dist((0, 0), source)) + 1e-7
48             if math.dist(q, source) <= 5:
49                 radius = 5.0
50             else:
51                 truth = math.degrees(math.atan2(source[1] - q[1], source[0] - q[0]))
52                 radius = 0.0
53                 for error in (-1.0, 0.0, 1.0):
54                     reading = round((truth + error) % 360, 2) % 360
55                     poly = clip_bearing(first, q, reading)
56                     radius = max(radius, enclosing_circle(poly)[1])

```

```

57         worst_radius = max(worst_radius, radius)
58     output.append(
59         {
60             **candidate,
61             "sampled_worst_mec_radius_m": worst_radius,
62             "first_move_and_measure_s": math.hypot(*q) / 5 + 5,
63         }
64     )
65     best = min(
66         output,
67         key=lambda row: (
68             row["sampled_worst_mec_radius_m"],
69             row["first_move_and_measure_s"],
70         ),
71     )
72     result = {
73         "evidence": "illustrative_Q2_grid_score_not_global_optimum",
74         "first_station": [0, 0],
75         "first_reading_deg": 0,
76         "source_samples": len(source_samples),
77         "second_error_values_deg": [-1, 0, 1],
78         "best_sampled_geometry": best,
79         "collinear_control": output[-1],
80         "candidates": output,
81     }
82     Path("results").mkdir(exist_ok=True)
83     Path("results/second_point_example.json").write_text(
84         json.dumps(result, indent=2) + "\n"
85     )
86     print(json.dumps({k: v for k, v in result.items() if k != "candidates"}, indent=2))
87
88
89 if __name__ == "__main__":
90     example()

```

Listing 16: 覆盖点生成

```

1
2
3 from __future__ import annotations
4
5 import math
6
7 Point = tuple[float, float]
8
9 ARENA_RADIUS = 1800.0
10 MIN_RADIO_RADIUS = 1000.0
11 OMNI_COVER_RADIUS = 900.0
12 DEFAULT_SPACING = 990.0
13
14
15 def omni_waypoints(ring_radius: float | None = None) -> list[Point]:
16
17     ring_radius = OMNI_COVER_RADIUS * math.sqrt(3.0) if ring_radius is None else float(ring_radius)
18     if not math.isfinite(ring_radius) or not 1200.0 <= ring_radius <= OMNI_COVER_RADIUS * math.sqrt(3.0):
19         raise ValueError("Supported ring radius is [1200, 900*sqrt(3)] m")
20     return [(0.0, 0.0)] + [
21         (
22             ring_radius * math.cos(k * math.pi / 3.0),
23             ring_radius * math.sin(k * math.pi / 3.0),
24         )
25         for k in range(6)
26     ]
27
28

```

```

29 def ring_cover_radius(ring_radius: float) -> float:
30
31     omni_waypoints(ring_radius)
32     return max(ring_radius / math.sqrt(3.0),
33                math.sqrt(ARENA_RADIUS**2 + ring_radius**2
34                        - math.sqrt(3.0) * ARENA_RADIUS * ring_radius))
35
36
37 def directional_waypoints(spacing: float = DEFAULT_SPACING) -> list[Point]:
38
39     spacing = float(spacing)
40     if not math.isfinite(spacing) or not 900.0 <= spacing <= MIN_RADIO_RADIUS - 1.0:
41         raise ValueError("spacing must be finite and in [900, 999] m")
42
43
44
45
46     numerator, denominator = spacing.as_integer_ratio()
47     guard_numerator = 1800 * denominator + numerator
48     numerator_squared = numerator * numerator
49     guard_squared = guard_numerator * guard_numerator
50
51
52     bound = (2 * guard_numerator + numerator - 1) // numerator
53     row_height = spacing * math.sqrt(3.0) / 2.0
54     points: list[Point] = [(0.0, 0.0)]
55     for j in range(-bound, bound + 1):
56         for i in range(-bound, bound + 1):
57             if i == 0 and j == 0:
58                 continue
59             norm_squared_units = i * i + i * j + j * j
60             if norm_squared_units * numerator_squared <= guard_squared:
61                 points.append((spacing * (i + j / 2.0), row_height * j))
62     return points
63
64
65 def coverage_certificate() -> dict:
66
67     return {
68         "certificate_kind": "analytic_geometric_cover",
69         "arena_radius_m": ARENA_RADIUS,
70         "minimum_radio_radius_m": MIN_RADIO_RADIUS,
71         "channels": 20,
72         "omni_legacy": {
73             "waypoint_count": 7,
74             "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
75             "cover_radius_m": OMNI_COVER_RADIUS,
76             "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
77             "proof": (
78                 "r<=900 is covered by the origin. For 900<=r<=1800, "
79                 "choose a ring center with angular gap <=pi/6. Then "
80                 "distance^2<=r^2-2700*r+2430000<=810000 by convexity "
81                 "and equality of the two endpoint values."
82             ),
83         },
84         "omni_enhanced": {
85             "waypoint_count": 7,
86             "ring_radius_m": 1200.0,
87             "cover_radius_m": ring_cover_radius(1200.0),
88             "radio_slack_m": MIN_RADIO_RADIUS - ring_cover_radius(1200.0),
89             "proof": "Exact envelope formula ring_cover_radius(1200); no point or channel obligation is removed
90         },
91         "omni": {
92             "waypoint_count": 7,
93             "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),

```

```

94         "cover_radius_m": OMNI_COVER_RADIUS,
95         "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
96         "proof": "Legacy 900m certificate; enhanced 1200m certificate is in omni_enhanced.",
97     },
98     "directional": {
99         "spacing_m": DEFAULT_SPACING,
100         "basis_m": [
101             [DEFAULT_SPACING, 0.0],
102             [DEFAULT_SPACING / 2.0, DEFAULT_SPACING * math.sqrt(3.0) / 2.0],
103         ],
104         "selection_radius_m": ARENA_RADIUS + DEFAULT_SPACING,
105         "maximum_source_vertex_distance_m": DEFAULT_SPACING,
106         "radio_slack_m": MIN_RADIO_RADIUS - DEFAULT_SPACING,
107         "waypoint_count": 31,
108         "inside_or_on_arena_count": 13,
109         "outside_arena_count": 18,
110         "norm_squared_units_counts": {0: 1, 1: 6, 3: 6, 4: 6, 7: 12},
111         "maximum_waypoint_radius_m": DEFAULT_SPACING * math.sqrt(7.0),
112         "closed_halfplane_required": True,
113         "boundary_guard_required": True,
114         "proof": (
115             "An elementary triangular-lattice triangle containing g has "
116             "three vertices within spacing of g. Their norms are at most "
117             "1800+spacing, hence all are selected. Write "
118             "g=sum(lambda_i*v_i), lambda_i>=0, sum(lambda_i)=1. "
119             "For any emission normal n, sum(lambda_i*n dot (v_i-g))=0, "
120             "so at least one vertex is in the closed emission halfplane. "
121             "If g is a lattice vertex inside the source arena, all six "
122             "nearest neighbors have norm <=1800+spacing and are selected; "
123             "one makes angle <=pi/6 with any emission normal."
124         ),
125         "count_proof": (
126             "At spacing 990 the cutoff is  $i^2+i*j+j^2 \leq 961/121$ . "
127             "The possible shells are 0,1,3,4,7 with multiplicities "
128             "1,6,6,6,12. The arena cutoff is 400/121, leaving shells "
129             "0,1,3 inside and shells 4,7 outside."
130         ),
131     },
132     "termination": (
133         "For each of 20 channels, retain either a confirmed successful "
134         "clear or a complete all-no_signal covering scan for that channel. "
135         "A detected but uncleared channel remains unresolved. With at "
136         "most one source per channel, all 20 certificates imply no "
137         "remaining source. Independently, 16 distinct confirmed clears "
138         "reach the stated global upper bound. Fewer clears alone do not."
139     ),
140     "no_signal": {
141         "q3": "Excludes every source at distance <=1000 from that detector.",
142         "q4": (
143             "Does not exclude that disk: a source may emit away from the "
144             "detector. Only the completed per-channel halfplane cover "
145             "excludes every source position and orientation."
146         ),
147     },
148     "assumptions": [
149         "Sources are stationary, persist until cleared, and use distinct channels.",
150         "Directional emission is a closed 180-degree halfplane with fixed orientation.",
151         "Reception reaches at least 1000 m throughout the emission halfplane.",
152         "All designated measurements actually complete on the indicated channel.",
153         "Near-source responses count as detections, not as no_signal.",
154         "Robot coordinates outside the radius-1800 source arena are permitted.",
155         "Geometric proofs use exact real coordinates; default float coordinates have 10 m range slack.",
156     ],
157 }

```

Listing 17: Q1、Q2 图片重绘

```

1
2 from __future__ import annotations
3
4 import math
5 from pathlib import Path
6
7 import matplotlib as mpl
8 mpl.use("pdf")
9 import matplotlib.pyplot as plt
10 from matplotlib.patches import Circle, Polygon, Wedge
11 import numpy as np
12
13 ROOT = Path(__file__).resolve().parent
14 OUT = ROOT / "report" / "figures"
15 NAVY, ORANGE, TEAL, GREY = "#12355B", "#E67E22", "#17807E", "#68737D"
16 mpl.rcParams.update({
17     "font.family": "sans-serif", "font.sans-serif": ["Arial Unicode MS"],
18     "axes.unicode_minus": False, "pdf.fonttype": 42, "font.size": 9,
19 })
20
21
22 def save(fig, name):
23     fig.savefig(OUT / name, bbox_inches="tight", pad_inches=0.08)
24     plt.close(fig)
25
26
27 def geometry_figure():
28     D = 36.0
29     tri = np.array([(0, 0), (D, 0), (D / 2, math.sqrt(3) * D / 2)])
30     mec_r = D / math.sqrt(3)
31     fig, ax = plt.subplots(1, 2, figsize=(10.8, 4.6), gridspec_kw={"width_ratios": [1, 1.18]})
32
33     a = ax[0]
34     a.add_patch(Polygon(tri, closed=True, facecolor=ORANGE, alpha=.18, edgecolor=NAVY, lw=2.1, label="可行域 (等边三角形)"))
35     stations = []
36     L = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
37     for p, q, bearing in zip(tri, np.roll(tri, -1, axis=0), (1, 121, 241)):
38         e = (q - p) / D
39         s = p - L * e
40         stations.append(s)
41         a.plot([s[0], p[0]], [s[1], p[1]], color=GREY, lw=1, ls="--")
42         a.scatter(*s, s=24, color=TEAL, zorder=4)
43         a.annotate(f"测站 {len(stations)}\n{bearing}°", s, xytext=(0, -17 if len(stations) == 1 else 10),
44                 textcoords="offset points", ha="center", fontsize=7, color=TEAL)
45     for label, p, offset in zip("ABC", tri, ((-8, -10), (7, -10), (0, 8))):
46         a.scatter(*p, color=NAVY, s=20, zorder=5); a.annotate(label, p, xytext=offset, textcoords="offset points", color=NAVY, weight="bold")
47     a.annotate("D=36 m", (18, 0), xytext=(0, -17), textcoords="offset points", ha="center", color=NAVY)
48     a.set_title("(a) 可实现的测向可行域", color=NAVY, weight="bold")
49     a.set_xlabel("x / m"); a.set_ylabel("y / m"); a.set_aspect("equal"); a.grid(alpha=.18)
50     a.legend(loc="upper right", fontsize=7, frameon=False)
51     a.set_xlim(-L*.08, D+L*.08); a.set_ylim(-L*.03, tri[2,1]+12)
52
53     b = ax[1]
54     b.add_patch(Polygon(tri, closed=True, facecolor=ORANGE, alpha=.15, edgecolor=NAVY, lw=2, label="三角形可行域"))
55     b.add_patch(Circle((18, tri[2,1]/3), D/2, fill=False, color=GREY, lw=1.8, ls="--", label="直径圆 r=18 m"))
56     b.add_patch(Circle((18, tri[2,1]/3), mec_r, fill=False, color=ORANGE, lw=2.2, label=f"最小包围圆 r={mec_r:.3f} m"))
57     b.scatter([18], [tri[2,1]/3], color=ORANGE, s=22, zorder=4)
58     b.annotate("MEC 圆心", (18, tri[2,1]/3), xytext=(8, 10), textcoords="offset points", color=ORANGE, fontsize=8)
59     for label, p in zip("ABC", tri): b.scatter(*p, color=NAVY, s=18, zorder=4); b.annotate(label, p, xytext=(5,

```



```

    4), textcoords="offset points", color=NAVY, weight="bold")
60 b.set_title("(b) 直径圆不足, 需用最小包围圆", color=NAVY, weight="bold")
61 b.set_xlabel("x / m"); b.set_ylabel("y / m"); b.set_aspect("equal"); b.grid(alpha=.18)
62 b.legend(loc="upper right", fontsize=7, frameon=False)
63 b.set_xlim(-5, 41); b.set_ylim(-7, 40)
64 fig.tight_layout(); save(fig, "geometry.pdf")
65
66
67 def lens_figure():
68     alpha = math.radians(1.005); ts = np.linspace(0, 1000, 500)
69
70     centers = np.array([(0., 0.), (1000*math.cos(alpha), 1000*math.sin(alpha)), (1000*math.cos(alpha), -1000*
71         math.sin(alpha))])
72     lim = 1030; gx = np.linspace(-30, lim, 700); gy = np.linspace(-530, 530, 500); X,Y=np.meshgrid(gx,gy)
73     mask = np.ones_like(X, dtype=bool)
74     for c in centers: mask &= (X-c[0])**2+(Y-c[1])**2 <= 1000**2
75     fig, ax = plt.subplots(1, 2, figsize=(10.8, 4.8), gridspec_kw={"width_ratios":[1.12,1]})
76     a=ax[0]; a.contourf(X,Y,mask,levels=[.5,1],colors=[ORANGE],alpha=.28)
77     for c in centers: a.add_patch(Circle(c,1000,fill=False,color=NAVY,lw=1.2,alpha=.6))
78     a.add_patch(Wedge((0,0),1000,-math.degrees(alpha),math.degrees(alpha),facecolor=TEAL,alpha=.10,edgecolor=
79         TEAL,lw=1.2))
80     a.plot([0,1000],[0,0],color=TEAL,lw=1.4,label="首次测向方向 theta_1=0 deg")
81     a.scatter(0,0,color=NAVY,s=28,zorder=4); a.annotate("S1=(0,0)",(0,0),xytext=(7,8),textcoords="offset points
82         ",color=NAVY)
83     a.set_title("(a) 全局: 三圆交的安全候选域",color=NAVY,weight="bold"); a.set_xlabel("沿首次方向 / m"); a.
84         set_ylabel("横向 / m"); a.set_aspect("equal"); a.grid(alpha=.18); a.legend(fontsize=7,frameon=False,loc="
85         upper right"); a.set_xlim(-30,1030); a.set_ylim(-530,530)
86     b=ax[1]; b.contourf(X,Y,mask,levels=[.5,1],colors=[ORANGE],alpha=.3)
87
88     def bound(t): return min(math.sqrt(max(0,1000**2-t**2)), math.sqrt(max(0,1000**2-(t-1000*math.cos(alpha))
89         **2))-1000*math.sin(alpha))
90     valid=[t for t in ts if bound(t)>=0]
91     b.plot(valid,[bound(t) for t in valid],color=ORANGE,lw=1.8,label="|b| <= b_max(t)"); b.plot(valid,[-bound(t)
92         ] for t in valid],color=ORANGE,lw=1.8)
93     for t in (125,250,375,500,625,750,875):
94         bb=bound(t)
95         for frac in (.25,.5,.75): b.scatter([t,t],[frac*bb,-frac*bb],s=10,color=TEAL,alpha=.7)
96     b.scatter(750,-496.078,s=35,color=NAVY,zorder=5,label="示例候选 (750,-496.078)")
97     b.scatter(750,0,s=28,color=GREY,zorder=5,label="共线对照 (750,0)")
98     b.annotate("横向正/负候选", (500,250),xytext=(0,12),textcoords="offset points",ha="center",color=ORANGE)
99     b.set_title("(b) 局部: (t,b) 横向候选与共线对照",color=NAVY,weight="bold"); b.set_xlabel("t / m (沿首次方向)
100         "); b.set_ylabel("b / m (横向)"); b.set_aspect("equal"); b.grid(alpha=.18); b.legend(fontsize=7,frameon=
101         False,loc="lower left"); b.set_xlim(-30,1030); b.set_ylim(-530,530)
102     fig.tight_layout(); save(fig,"q2_lens.pdf")
103
104 if __name__ == "__main__":
105     OUT.mkdir(parents=True, exist_ok=True)
106     geometry_figure(); lens_figure()
107     print("wrote", OUT / "geometry.pdf", OUT / "q2_lens.pdf")

```

Listing 18: Q3、Q4 图片重绘

```

1
2 from pathlib import Path
3 import math
4 import matplotlib
5 matplotlib.use("Agg")
6 import matplotlib.pyplot as plt
7 from matplotlib.patches import Circle, Polygon, Wedge, FancyArrowPatch, Rectangle
8 import numpy as np
9
10 ROOT = Path(__file__).resolve().parent
11 OUT = ROOT / "report" / "figures"
12 BLUE, ORANGE, GREEN, GREY, RED = "#123B66", "#E67E22", "#1B8A70", "#687480", "#B23A48"
13 plt.rcParams.update({"font.family":"sans-serif", "font.sans-serif":["Arial Unicode MS","Hiragino Sans GB","

```

```

    Heiti SC","DejaVu Sans"], "font.size":9, "pdf.fonttype":42, "axes.unicode_minus":False})
14
15 def finish(fig, name):
16     OUT.mkdir(parents=True, exist_ok=True)
17     fig.savefig(OUT/name, format="pdf", metadata={"Creator":"CUMCM Q3/Q4 redraw_q34.py"})
18     plt.close(fig)
19
20 def ring():
21     fig, ax = plt.subplots(figsize=(7.2,6.0), layout="constrained")
22     ax.set_aspect("equal"); ax.set_xlim(-2150,2150); ax.set_ylim(-2150,2150)
23     ax.add_patch(Circle((0,0),1800, facecolor="#F4F7FA", edgecolor=BLUE, lw=1.8, ls="--", label="源域 R=1800 m"
24     ))
25     a=np.arange(6)*np.pi/3; pts=np.c_[1200*np.cos(a),1200*np.sin(a)]
26     for p in pts: ax.add_patch(Circle(p,1000,fill=False,edgecolor=ORANGE,lw=1.0,alpha=.62))
27     ax.add_patch(Circle((0,0),1000,fill=False,edgecolor=ORANGE,lw=1.0,alpha=.62))
28     ax.plot(*np.vstack([pts,pts[0]]).T,color=ORANGE,lw=1.4,alpha=.8)
29     ax.scatter(pts[:,0],pts[:,1],s=38,color=ORANGE,zorder=4); ax.scatter(0,0,s=42,color=BLUE,zorder=4)
30     for i,p in enumerate(pts): ax.text(p[0]*1.08,p[1]*1.08,f"P{i+1}",ha="center",va="center",color=BLUE,
31     fontsize=8)
32     ax.text(70,-120,"0",color=BLUE,fontsize=9)
33     g=1800*np.array([math.cos(math.pi/6),math.sin(math.pi/6)]); near=pts[0]
34     ax.scatter(*g,marker="x",s=70,color=RED,lw=2,zorder=6)
35     ax.plot([g[0],near[0]],[g[1],near[1]],color=GREEN,lw=2)
36     ax.annotate("最坏源点 g=(1800cos30°, 1800sin30°)",xy=g,xytext=(870,1780),arrowprops=dict(arrowstyle="→",
37     color=RED,lw=1),color=RED,fontsize=8)
38     ax.annotate("最近检测点 P1",xy=near,xytext=(1220,-210),arrowprops=dict(arrowstyle="→",color=GREEN,lw=1),
39     color=GREEN,fontsize=8)
40     mid=(g+near)/2; ax.text(mid[0]+30,mid[1]+25,"968.90 m",color=GREEN,fontsize=9,weight="bold")
41     ax.text(-2050,-2020,"1000 m 接收圆; 最坏距离余量 1000 - 968.90 = 31.10 m",color=BLUE,fontsize=8)
42     ax.set_xlabel("x / m"); ax.set_ylabel("y / m"); ax.set_title("Q3 七点环覆盖 (原理示意)",color=BLUE,weight="
43     bold")
44     ax.grid(alpha=.15); finish(fig,"competition_ring.pdf")
45
46 def localize():
47     fig,axs=plt.subplots(1,2,figsize=(8.0,4.1),layout="constrained")
48     ax=axs[0]; ax.set_aspect("equal"); ax.set_xlim(-25,310); ax.set_ylim(-115,125)
49     s=np.array([0.,0.]); z=np.array([185.,0.]); delta=25.
50     ax.add_patch(Polygon([[0,0],[295,75],[295,-75]],facecolor="#FBE9D7",edgecolor=ORANGE,lw=1.2))
51     ax.plot([0,295],[0,0],"--",color=BLUE,lw=1.2)
52     poly=np.array([[135,-25],[215,-35],[240,-8],[225,32],[145,28]])
53     ax.add_patch(Polygon(poly,facecolor="#DFF8F1",edgecolor=BLUE,lw=1.6))
54     ax.scatter(*s,color=BLUE,s=30); ax.text(-8,-18,r"$s^+$",color=BLUE)
55     ax.scatter(*z,color=BLUE,s=24); ax.text(z[0]-17,-12,"z",color=BLUE)
56     ax.scatter([z[0],z[0]],[delta,-delta],color=ORANGE,s=32,zorder=5)
57     ax.plot([z[0],z[0]],[delta,delta],color=ORANGE,lw=1.2)
58     ax.add_patch(FancyArrowPatch((185,40),(185,83),arrowstyle="|>",mutation_scale=12,color=BLUE,lw=1.4))
59     ax.text(194,76,"n",color=BLUE); ax.text(195,36,r"$z+\delta n$",color=ORANGE); ax.text(195,-55,r"$z-\delta
60     n$",color=ORANGE)
61     ax.text(255,10,r"$P_c$",color=BLUE); ax.text(12,45,"正示向楔形",color=ORANGE,fontsize=8)
62     ax.set_xlabel("局部 x / m"); ax.set_ylabel("局部 y / m")
63     ax.set_title("(a) 横向候选点",color=BLUE,fontsize=10)
64     ax=axs[1]; ax.set_aspect("equal"); ax.set_xlim(-15,115); ax.set_ylim(-20,95)
65
66     ax.add_patch(Rectangle((0,0),100,75,facecolor="#F4F7FA",edgecolor=BLUE,lw=1.5))
67     for v in (25,50,75): ax.plot([v,v],[0,75],color=GREY,alpha=.4,lw=.6)
68     for v in (25,50): ax.plot([0,100],[v,v],color=GREY,alpha=.4,lw=.6)
69     for y in (12.5,37.5,62.5):
70         for x in (12.5,37.5,62.5,87.5):
71             ax.add_patch(Circle((x,y),19.9,fill=False,edgecolor=ORANGE,lw=.9,alpha=.8)); ax.scatter(x,y,s=12,
72             color=ORANGE,zorder=3)
73     ax.plot([87.5,107.4],[62.5,62.5],color=BLUE,lw=1.3)
74     ax.annotate("19.9 m",xy=(99,62.5),xytext=(63,85),arrowprops=dict(arrowstyle="→",color=BLUE,lw=.8),color=
75     BLUE,fontsize=8)
76     ax.set_xlabel("矩形坐标 x / m"); ax.set_ylabel("矩形坐标 y / m")
77     ax.set_title("(b) 整个矩形的圆盘覆盖",color=BLUE,fontsize=10)
78     fig.suptitle("局部定位与光学兜底 (原理示意, 非观测轨迹)",color=BLUE,fontsize=10)

```

```

71     finish(fig,"competition_localize.pdf")
72
73 def direction():
74     fig,axs=plt.subplots(1,2,figsize=(8.5,4.0),layout="constrained")
75     ax=axs[0]; ax.set_aspect("equal"); ax.set_xlim(-2150,2150); ax.set_ylim(-2150,2150)
76     ax.add_patch(Circle((0,0),1800,fill=False,color=BLUE,lw=1.3,ls="--"));
77     ai=np.arange(12)*np.pi/6; ae=ai+np.pi/12
78     I=np.c_[940*np.cos(ai),940*np.sin(ai)]; E=np.c_[1870*np.cos(ae),1870*np.sin(ae)]
79     ax.scatter(I[:,0],I[:,1],s=20,color=ORANGE,label="内环 I (12点)"); ax.scatter(E[:,0],E[:,1],s=20,color=
    GREEN,label="外环 E (12点)"); ax.scatter(0,0,s=28,color=BLUE,label="0")
80     g=1800*np.array([math.cos(np.pi/4),math.sin(np.pi/4)]); n=g/1800
81     ax.scatter(*g,color=RED,marker="x",s=58,lw=2); ax.add_patch(FancyArrowPatch(g,g+650*n,arrowstyle="-|>",
    mutation_scale=14,color=RED,lw=2))
82     ax.text(-2050,-2020,"25点: 0 + 内环12 + 外环12; 外环点位于源域外",fontsize=7.8,color=BLUE)
83     ax.text(730,1700,"前向",color=RED,fontsize=8); ax.text(-850,1350,"背面失效",color=GREY,fontsize=8)
84     ax.set_title("定向源: 距离条件还需满足  $n \cdot (p-g) \geq 0$ ",color=BLUE,fontsize=8.5); ax.legend(fontsize=7,loc=
    "lower left"); ax.set_xlabel("x / m"); ax.set_ylabel("y / m")
85     ax=axs[1]; ax.set_aspect("equal"); ax.set_xlim(-1.25,1.35); ax.set_ylim(-1.05,1.1)
86     tri=np.array([[-.9,-.62],[.88,-.58],[.05,.82]]); ax.add_patch(Polygon(tri,fill=False,color=BLUE,lw=1.7));
87     g=np.array([-0.25,.10]); n=np.array([.72,.22]); n=n/np.linalg.norm(n); h=g+0.42*n; c=tri.mean(0)
88     ax.scatter(*g,color=RED,s=36); ax.scatter(*h,color=GREEN,s=36); ax.scatter(*c,color=GREY,s=28)
89     ax.add_patch(FancyArrowPatch(g,h,arrowstyle="-|>",mutation_scale=12,color=GREEN,lw=1.8)); ax.plot([c[0],h
    [0]],[c[1],h[1]],"--",color=GREY,lw=1.2)
90     ax.text(g[0]-.18,g[1]-.17,"g",color=RED); ax.text(h[0]+.04,h[1]+.04,"h=g+5n",color=GREEN,fontsize=8); ax.
    text(c[0]+.05,c[1],"重心投影",color=GREY,fontsize=8)
91     ax.text(-1.15,-.92,"任意方向 -> 辅助点 h 属于三角形 -> 至少一个顶点前向",fontsize=7.8,color=BLUE)
92     ax.set_title("三角形重心投影证明",color=BLUE,weight="bold"); ax.axis("off")
93     fig.suptitle("Q4 方向覆盖 (原理示意, 非观测轨迹)",color=BLUE,weight="bold",fontsize=10)
94     finish(fig,"competition_direction.pdf")
95
96 if __name__ == "__main__":
97     ring(); localize(); direction()

```

Listing 19: 结果图片重绘

```

1
2 from pathlib import Path
3 import json
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
7
8 ROOT = Path(__file__).resolve().parent
9 OUT = ROOT / "report" / "figures"
10 NAVY, ORANGE, TEAL, GREY = "#12355B", "#E07A28", "#2A9D8F", "#607080"
11 plt.rcParams.update({"font.family":"sans-serif", "font.sans-serif":["Arial Unicode MS"], "font.size":10, "pdf.
    fonttype":42, "axes.unicode_minus":False})
12
13 def save(fig, name):
14     fig.savefig(OUT/name, format="pdf", bbox_inches="tight", metadata={"Creator":"redraw_results.py"}); plt.
    close(fig)
15
16 def taskmap():
17     fig, ax = plt.subplots(figsize=(10,3.2)); ax.axis("off"); ax.set_xlim(0,1); ax.set_ylim(0,1)
18     ax.text(.5,.92,"四问依赖流程 (原理示意)",ha="center",color=NAVY,weight="bold",fontsize=14)
19     labs=[("Q1","几何边界\n可行域与交会"),("Q2","覆盖建模\n接收盘与光学"),("Q3","全向清除\n扫描—证书"),("Q4","定向
    定位\n测向—覆盖")]
20     xs=[.08,.32,.56,.80]
21     for i,(q,t) in enumerate(labs):
22         p=FancyBboxPatch((xs[i],.37),.15,.29,boxstyle="round,pad=.02",fc="#F5F8FC",ec=NAVY,lw=1.8)
23         ax.add_patch(p); ax.text(xs[i]+.075,.59,q,ha="center",color=ORANGE,weight="bold",fontsize=12); ax.text(
    xs[i]+.075,.47,t,ha="center",va="center",color=NAVY)
24         if i<3: ax.add_patch(FancyArrowPatch((xs[i]+.16,.515),(xs[i+1]-.015,.515),arrowstyle="-|>",
    mutation_scale=15,lw=1.5,color=GREY))
25     ax.text(.5,.15,"前一问输出作为后一问的约束与初始证书",ha="center",color=GREY,style="italic")

```

```

26     save(fig,"competition_taskmap.pdf")
27
28 def dispatch():
29     fig,ax=plt.subplots(figsize=(8,5.6)); ax.axis("off"); ax.set_xlim(0,1); ax.set_ylim(0,1)
30     ax.text(.5,.96,"Q3/Q4 任务调度闭环 (原理示意)",ha="center",color=NAVY,weight="bold",fontsize=14)
31     nodes=[(.5,.83,"初始化\n候选骨架与频道"),(.5,.66,"骨架检测\n保留发现义务"),(.5,.49,"正观测建域\n更新可行多边形"),(.5,.32,"局部清除 / 光学宽底\n按证书执行"),(.5,.15,"更新频道证书\n记录清除或覆盖")]
32     for x,y,t in nodes:
33         ax.add_patch(FancyBboxPatch((x-.17,y-.045),.34,.09,boxstyle="round,pad=.015",fc="#F5F8FC",ec=NAVY,lw=1.5)); ax.text(x,y,t,ha="center",va="center",color=NAVY)
34     for (_,y1,_),(_,y2,_) in zip(nodes,nodes[1:]): ax.add_patch(FancyArrowPatch((.5,y1-.055),(.5,y2+.055),arrowstyle="-|>",mutation_scale=13,color=ORANGE,lw=1.5))
35     ax.add_patch(FancyArrowPatch((.32,.15),(.12,.83),connectionstyle="arc3,rad=-.28",arrowstyle="-|>",mutation_scale=14,color=TEAL,lw=1.8))
36     ax.text(.06,.49,"未停止: \n重新选择任务",rotation=90,ha="center",va="center",color=TEAL)
37     ax.text(.77,.57,"停止条件: 每个频道已清除或\n覆盖认证缺失; 或已清除16个不同频道",ha="center",va="center",color=GREY,fontsize=9,bbox=dict(boxstyle="round",fc="#FFF8F0",ec=ORANGE,alpha=.8))
38     save(fig,"competition_dispatch.pdf")
39
40 def load_field(): return json.loads((ROOT/"results/field_confirmation.json").read_text(encoding="utf-8"))
41 def costs():
42     runs=[r for r in load_field()["runs"] if 7100000<=r["seed"]<=7100999]; groups=[]
43     for p,cfg in [(3,"control"),(3,"field"),(4,"control"),(4,"field")]:
44         rr=[r for r in runs if r["problem"]==p and r["configuration"]==cfg]
45         vals=[]
46         for r in rr:
47             n=r["n_sources"]; parts=np.array([r["distance_m"]/5,5*r["measures"],r["switches"],3*r["failed_clears"],5*r["n_cleared"]],float)/n
48             assert abs(r["average_time_s"]-sum(parts)) < 2e-5
49             vals.append(parts)
50         groups.append((f"Q{p} {cfg}",np.mean(vals,axis=0)))
51     a=np.array([v for _,v in groups]); fig,ax=plt.subplots(figsize=(9,4.8)); x=np.arange(4); bottom=np.zeros(4)
52     cols=[NAVY,ORANGE,TEAL,"#C95D63","#8D99AE"]; labs=["移动 distance/5","测量 5×measures","换频 switches","失败清除 3×failed_clears","成功清除 5×n_cleared"]
53     for j in range(5): ax.bar(x,a[:,j],bottom=bottom,color=cols[j],label=labs[j]); bottom+=a[:,j]
54     ax.set_xticks(x,[g[0] for g in groups]); ax.set_ylabel("平均耗时 (s/source)"); ax.set_title("主组成本分解 (6000行, 按世界与源平均)",color=NAVY,weight="bold"); ax.legend(ncol=2,fontsize=8); ax.grid(axis="y",alpha=.2); save(fig,"competition_costs.pdf")
55
56 def field_performance():
57     runs=load_field()["runs"]
58     runs=[r for r in runs if 7100000<=r["seed"]<=7100999]
59     fig,axs=plt.subplots(1,2,figsize=(10,4.3),sharey=False)
60     for ax,p in zip(axs,[3,4]):
61         c=sorted([r for r in runs if r["problem"]==p and r["configuration"]=="control"],key=lambda r:r["seed"])
62         f=sorted([r for r in runs if r["problem"]==p and r["configuration"]=="field"],key=lambda r:r["seed"])
63         for rr,col,lab in [(c,NAVY,"control"),(f,ORANGE,"field")]:
64             v=np.sort([r["average_time_s"] for r in rr]); ax.plot(v,np.linspace(.001,1,len(v)),color=col,lw=2,label=lab)
65             d=np.array([x["average_time_s"]-y["average_time_s"] for x,y in zip(c,f)]); ax.text(.04,.08,f"1000对; field快 {np.sum(d>0):d}/1000\n均值差 {d.mean():.1f} s/source",transform=ax.transAxes,fontsize=9,color=TEAL)
66             ax.set_title(f"Q{p}",color=NAVY,weight="bold"); ax.set_xlabel("逐世界平均耗时 (s/source)"); ax.set_ylabel("ECDF"); ax.legend(); ax.grid(alpha=.2)
67     fig.suptitle("field_confirmation: 控制与field同世界配对",color=NAVY,weight="bold"); save(fig,"field_performance.pdf")
68
69 def research_performance():
70     d=json.loads((ROOT/"results/research_experiments.json").read_text(encoding="utf-8")); s=d["summary"]; fig,ax=plt.subplots(figsize=(9,4.8)); keys=["Q3_control","Q3_joint_all","Q3_refined","Q4_control","Q4_joint_all","Q4_refined"]; labels=["Q3 control","Q3 joint_all","Q3 refined","Q4 control","Q4 joint_all","Q4 refined"]; vals=[s[k]["mean_average_time_s"] for k in keys]; cols=[GREY,TEAL,ORANGE,GREY,TEAL,ORANGE]; bars=ax.bar(labels,vals,color=cols); ax.set_ylabel("平均耗时 (s/source)"); ax.set_title("历史 refined 研究实验 (非 field 数据)",color=NAVY,weight="bold"); ax.grid(axis="y",alpha=.2)
71     for b,v in zip(bars,vals): ax.text(b.get_x()+b.get_width()/2,v+max(vals)*.015,f"{v:.1f}",ha="center",fontsize=9)

```

```

71     ax.text(.02,.95,"输入: results/research_experiments.json 主组, 1000 cases/problem",transform=ax.transAxes,va
    ="top",color=GREY,fontsize=9); ax.tick_params(axis="x",rotation=18); save(fig,"research_performance.pdf")
72
73 def main():
74     OUT.mkdir(parents=True,exist_ok=True)
75     for f in (taskmap,dispatch,costs,field_performance,research_performance): f()
76 if __name__=="__main__": main()

```

Listing 20: 新增正文图表生成

```

1  """Draw five additional body figures without changing any strategy or result.
2
3  Run with the project's py314 interpreter. Geometry panels use stated analytic
4  examples; the trajectory and formal comparison use existing saved observations.
5  """
6  from __future__ import annotations
7
8  import argparse
9  import hashlib
10 import itertools
11 import json
12 import math
13 from pathlib import Path
14
15 import matplotlib
16 matplotlib.use("Agg")
17 import matplotlib.pyplot as plt
18 from matplotlib.patches import Arc, Circle, FancyArrowPatch, Polygon
19 from matplotlib.ticker import MaxNLocator
20 import numpy as np
21
22 from intersection import solve_intersection
23 from geometry import BEARING_ERROR_DEG
24
25 ROOT = Path(__file__).resolve().parent
26 FIGURES = ROOT / "report" / "figures"
27 BLUE, ORANGE, TEAL, GREY = "#12355B", "#E67E22", "#17807E", "#68737D"
28 plt.rcParams.update({
29     "font.family": "sans-serif",
30     "font.sans-serif": ["Arial Unicode MS", "DejaVu Sans"],
31     "font.size": 9,
32     "axes.titlesize": 10,
33     "axes.labelsize": 9,
34     "xtick.labelsize": 8,
35     "ytick.labelsize": 8,
36     "legend.fontsize": 8,
37     "axes.unicode_minus": False,
38     "pdf.fonttype": 42,
39     "ps.fonttype": 42,
40     "axes.spines.top": False,
41     "axes.spines.right": False,
42 })
43
44
45 def save(fig, directory, name):
46     directory.mkdir(parents=True, exist_ok=True)
47     path = directory / name
48     fig.savefig(path, format="pdf", metadata={
49         "Creator": "redraw_additional.py", "CreationDate": None, "ModDate": None,
50     })
51     plt.close(fig)
52     return path
53
54
55 def wedge_figure(directory):

```

```

56     """Two legal bearings and an exact, bounded half-plane intersection."""
57     stations = [(0.0, 0.0), (950.0, 0.0)]
58     source = (600.0, 650.0)
59     bearings = [round(math.degrees(math.atan2(source[1]-y, source[0]-x)), 2)
60                 for x, y in stations]
61     result = solve_intersection(stations, bearings, BEARING_ERROR_DEG)
62     assert result["status"] == "bounded"
63     vertices = np.asarray(result["vertices"])
64     for (x, y), bearing in zip(stations, bearings):
65         actual = math.degrees(math.atan2(source[1]-y, source[0]-x))
66         assert abs(actual-bearing) <= BEARING_ERROR_DEG
67         assert math.dist((x, y), source) < 1000
68     i, j = max(itertools.combinations(range(len(vertices)), 2),
69               key=lambda ij: np.linalg.norm(vertices[ij[0]]-vertices[ij[1]]))
70     diameter = float(np.linalg.norm(vertices[i]-vertices[j]))
71     assert math.isclose(diameter, result["diameter_m"], abs_tol=1e-8)
72
73     fig, axes = plt.subplots(1, 2, figsize=(6.3, 2.85), layout="constrained")
74     ax = axes[0]
75     for k, (station, bearing, color) in enumerate(zip(stations, bearings, (BLUE, ORANGE)), 1):
76         theta = math.radians(bearing)
77         alpha = math.radians(BEARING_ERROR_DEG)
78         endpoints = [np.asarray(station)+1050*np.array([math.cos(theta+d), math.sin(theta+d)])
79                     for d in (-alpha, alpha)]
80         ax.add_patch(Polygon([station, *endpoints], facecolor=color, alpha=.16, edgecolor="none"))
81         for endpoint in endpoints:
82             ax.plot([station[0], endpoint[0]], [station[1], endpoint[1]], color=color, lw=.85)
83             ax.plot([station[0], source[0]], [station[1], source[1]], color=color, lw=.8, ls="--")
84             ax.scatter(*station, color=color, s=25, zorder=5)
85             ax.annotate(rf"$s_{k}$", station, xytext=(4, -15), textcoords="offset points", color=color)
86             ax.scatter(*source, marker="*", color=TEAL, s=50, zorder=6)
87             ax.annotate(r"$g$", source, xytext=(6, 5), textcoords="offset points", color=TEAL)
88             ax.set(xlim=(-70, 1050), ylim=(-100, 920), xlabel="x / m", ylabel="y / m",
89                   title=r"(a) 两次示向, $\alpha=1.005^\circ$")
90             ax.set_aspect("equal")
91             ax.grid(alpha=.13)
92
93     ax = axes[1]
94     ax.add_patch(Polygon(vertices, facecolor=TEAL, edgecolor=TEAL, alpha=.24, lw=1.3))
95     ax.scatter(vertices[:, 0], vertices[:, 1], color=TEAL, s=15, zorder=4)
96     ax.plot(vertices[[i, j], 0], vertices[[i, j], 1], color=ORANGE, lw=1.8, label="最远顶点连线")
97     ax.scatter(*source, marker="*", color=BLUE, s=50, zorder=6, label="算例源位置")
98     ax.text(.05, .95, rf"$D(P)={diameter:.2f}$ m", transform=ax.transAxes,
99           va="top", color=BLUE, fontsize=9)
100     extent = np.max(np.ptp(vertices, axis=0)) * .80
101     center = (vertices.min(axis=0)+vertices.max(axis=0))/2
102     ax.set(xlim=(center[0]-extent, center[0]+extent),
103           ylim=(center[1]-extent, center[1]+extent),
104           xlabel="x / m", ylabel="y / m", title=r"(b) 交集 $P=W_1\cap W_2$ 放大")
105     ax.set_aspect("equal")
106     ax.xaxis.set_major_locator(MaxNLocator(4))
107     ax.yaxis.set_major_locator(MaxNLocator(4))
108     ax.legend(loc="lower left", frameon=False, fontsize=7.5)
109     ax.grid(alpha=.13)
110     path = save(fig, directory, "q1_wedge.pdf")
111     return path, {"kind": "analytic_example", "stations_m": stations,
112                  "source_m": source, "bearings_deg": bearings,
113                  "error_deg": BEARING_ERROR_DEG, "vertices_m": result["vertices"],
114                  "diameter_m": diameter}
115
116
117 def angle_figure(directory):
118     """Only compare the angle factor with ranges and angular error fixed."""
119     fig, axes = plt.subplots(1, 2, figsize=(6.3, 2.75), layout="constrained")
120     source = np.array([0., 0.])
121     s1 = np.array([-600., 0.])

```



```

122 s2a = -600*np.array([math.cos(math.radians(20)), math.sin(math.radians(20))])
123 s2b = np.array([0., -600.])
124 ax = axes[0]
125 for point, color, label, offset in (
126     (s1, BLUE, r"$S_1$", (-12, 8)),
127     (s2a, GREY, r"$S_{2a}$", (-12, -15)),
128     (s2b, ORANGE, r"$S_{2b}$", (7, -3)),
129 ):
130     ax.plot([0, point[0]], [0, point[1]], color=color, lw=1.2)
131     ax.scatter(*point, s=22, color=color)
132     ax.annotate(label, point, xytext=offset, textcoords="offset points", color=color)
133 ax.add_patch(Arc(source, 290, 290, theta1=180, theta2=200, color=GREY, lw=1.1))
134 ax.add_patch(Arc(source, 185, 185, theta1=180, theta2=270, color=ORANGE, lw=1.1))
135 ax.text(-365, -115, r"$20^\circ$", fontsize=9, color=GREY)
136 ax.text(-215, -245, r"$90^\circ$", fontsize=9, color=ORANGE)
137 ax.scatter(*source, marker="*", s=60, color=TEAL, zorder=5)
138 ax.annotate(r"$g$", source, xytext=(7, 6), textcoords="offset points", color=TEAL)
139 ax.set(xlim=(-760, 160), ylim=(-720, 160), xlabel="x / m", ylabel="y / m",
140       title=r"(a) 固定 $r_1=r_2=600$ m")
141 ax.set_aspect("equal")
142 ax.grid(alpha=.13)
143
144 ax = axes[1]
145 beta = np.linspace(10, 170, 481)
146 factor = 1 / np.sin(np.radians(beta))
147 ax.plot(beta, factor, color=BLUE, lw=1.8)
148 for angle, color in ((20, GREY), (90, ORANGE)):
149     value = 1 / math.sin(math.radians(angle))
150     ax.scatter(angle, value, color=color, s=24, zorder=4)
151     ax.annotate(f"({angle}$^\circ$, {value:.2f})", (angle, value), xytext=(9, 10),
152               textcoords="offset points", color=color, fontsize=8)
153 ax.axvline(90, color=ORANGE, lw=.8, ls="--")
154 ax.set(xlim=(5, 175), ylim=(.7, 6.3), xlabel=r"交会角 $\beta$ / ($^\circ$)",
155       ylabel=r"相对面积 $1/|\sin\beta|$", title="(b) 固定距离下的角度影响")
156 ax.set_xticks([20, 60, 90, 120, 160])
157 ax.grid(alpha=.17)
158 path = save(fig, directory, "q2_angle.pdf")
159 return path, {"kind": "analytic_approximation", "ranges_m": [600, 600],
160              "beta_range_deg": [10, 170], "factor_20_deg": 1/math.sin(math.radians(20)),
161              "factor_90_deg": 1.0, "varying_distance_effect_included": False}
162
163
164 def q3_tradeoff_figure(directory):
165     R = 1800.0
166     dmax = 900 * math.sqrt(3)
167     ds = np.linspace(1200, dmax, 500)
168     rho = np.maximum(ds / math.sqrt(3), np.sqrt(R**2 + ds**2 - math.sqrt(3)*R*ds))
169     travel = 6 * ds
170     assert np.all(rho < 1000) and np.all(np.diff(rho) <= 1e-9)
171     assert math.isclose(rho[-1], 900, abs_tol=1e-8)
172     fig, axes = plt.subplots(1, 2, figsize=(6.3, 2.55), layout="constrained")
173     axes[0].plot(ds, rho, color=BLUE, lw=1.8)
174     axes[0].axhline(1000, color=GREY, ls="--", lw=.8)
175     axes[0].text(1280, 1003, "接收下界 1000 m", fontsize=8, color=GREY)
176     axes[0].scatter([1200, dmax], [rho[0], rho[-1]], color=ORANGE, s=22, zorder=4)
177     axes[0].annotate(f"{rho[0]:.2f} m", (1200, rho[0]), xytext=(9, 8),
178                    textcoords="offset points", color=BLUE, fontsize=8)
179     axes[0].text(.97, .16, "900.00 m", transform=axes[0].transAxes,
180                ha="right", color=BLUE, fontsize=8)
181     axes[0].set(xlabel=r"环半径 $d$ / m", ylabel=r"覆盖半径 $\rho(d)$ / m",
182               ylim=(890, 1016), title="(a) 环增大, 最坏覆盖距离缩短")
183     axes[1].plot(ds, travel, color=ORANGE, lw=1.8)
184     axes[1].scatter([1200, dmax], [travel[0], travel[-1]], color=BLUE, s=22, zorder=4)
185     axes[1].annotate("7200 m", (1200, 7200), xytext=(12, 18),
186                    textcoords="offset points", color=BLUE, fontsize=8,
187                    bbox=dict(facecolor="white", edgecolor="none", pad=.15))

```

```

188 axes[1].annotate(f"{travel[-1]:.1f} m", (dmax, travel[-1]), xytext=(-10, -24),
189                 textcoords="offset points", ha="right", color=ORANGE, fontsize=8,
190                 bbox=dict(facecolor="white", edgecolor="none", pad=15))
191 axes[1].set(xlabel=r"环半径 $d$ / m", ylabel="纯骨架行程 / m",
192            ylim=(7000, 9690), title="(b) 开放骨架行程为 6d")
193 for ax in axes:
194     ax.set_xlim(1185, 1580)
195     ax.set_xticks([1200, 1350, 1500])
196     ax.grid(alpha=.17)
197 path = save(fig, directory, "q3_tradeoff.pdf")
198 return path, {"kind": "analytic_curves", "ring_radius_range_m": [1200, dmax],
199                "coverage_at_1200_m": float(rho[0]), "travel_at_1200_m": 7200,
200                "travel_at_900sqrt3_m": float(travel[-1]),
201                "localization_or_measurement_cost_included": False}
202
203
204 def q4_route_figure(directory):
205     a, b = 940.0, 1870.0
206     angle = np.arange(12)*np.pi/6
207     inner = np.column_stack([a*np.cos(angle), a*np.sin(angle)])
208     outer = np.column_stack([b*np.cos(angle+np.pi/12), b*np.sin(angle+np.pi/12)])
209     route = np.vstack([[0., 0.], inner, outer[:-1]])
210     length = sum(math.dist(p, q) for p, q in zip(route, route[1:]))
211     formula = a+22*(a+b)*math.sin(math.pi/12)+math.sqrt(a*a+b*b-2*a*b*math.cos(math.pi/12))
212     assert len(route) == 25 and math.isclose(length, formula, abs_tol=1e-7)
213
214     trace_path = ROOT / "report/generated/research_trajectory_q4.json"
215     trace = json.loads(trace_path.read_text(encoding="utf-8"))
216     assert trace["evidence"] == "observable_local_synthetic_trajectory_not_official"
217     assert trace["planner"]["strategy"] == "refined" and trace["seed"] == 20260910
218     points, cleared = [(0., 0.)], []
219     elapsed_us, radio, distance = 0, 1, 0.0
220     for event in trace["observable_log"]:
221         req, resp = event["request"], event["response"]
222         if resp.get("accepted") is not True:
223             continue
224         if req["path"] in ("/measure", "/clear"):
225             point = (req["position"]["x"], req["position"]["y"])
226             step = math.dist(points[-1], point)
227             distance += step
228             elapsed_us += round(step / 5 * 1_000_000)
229             if req["path"] == "/measure":
230                 elapsed_us += (5+int(radio != req["channel"]))*1_000_000
231                 radio = req["channel"]
232             else:
233                 success = resp["clear_result"] == "success"
234                 elapsed_us += (5 if success else 3)*1_000_000
235                 if success:
236                     cleared.append(point)
237                 if point != points[-1]:
238                     points.append(point)
239                 assert abs(elapsed_us / 1_000_000 - resp["virtual_time_s"]) <= 1e-6
240     actual = trace["post_exit_evaluation"]
241     assert trace["planner"]["certificate_complete"]
242     assert len(cleared) == actual["n_sources"] == actual["n_cleared"]
243     assert math.isclose(distance, actual["distance_m"], abs_tol=1e-6)
244     assert math.isclose(elapsed_us/1_000_000, actual["virtual_time_s"], abs_tol=1e-6)
245
246     fig, axes = plt.subplots(1, 2, figsize=(6.3, 3.05), layout="constrained")
247     for ax in axes:
248         ax.add_patch(Circle((0, 0), 1800, fill=False, color=GREY, ls="--", lw=.8))
249         ax.set(xlim=(-2140, 2140), ylim=(-2140, 2140), xlabel="x / m", ylabel="y / m")
250         ax.set_aspect("equal")
251         ax.set_xticks([-1800, 0, 1800])
252         ax.set_yticks([-1800, 0, 1800])
253         ax.grid(alpha=.13)

```



```

254 ax = axes[0]
255 ax.plot(route[:, 0], route[:, 1], lw=1.05, color=BLUE, zorder=2)
256 for p, q in zip(route[:-1:3], route[1::3]):
257     ax.add_patch(FancyArrowPatch(p+.3*(q-p), p+.7*(q-p), arrowstyle="->>",
258                                mutation_scale=9, color=BLUE, lw=.85))
259 ax.scatter(inner[:, 0], inner[:, 1], s=14, color=ORANGE, label="内环点")
260 ax.scatter(outer[:, 0], outer[:, 1], s=14, color=TEAL, label="外环点")
261 ax.scatter(0, 0, s=25, marker="D", color=BLUE)
262 ax.set_title(f"(a) 骨架可行路线: {length/1000:.2f} km")
263 ax.legend(loc="upper center", bbox_to_anchor=(.5, -.23), ncol=2, frameon=False)
264
265 ax = axes[1]
266 points, cleared = np.asarray(points), np.asarray(cleared)
267 ax.plot(points[:, 0], points[:, 1], color=BLUE, lw=.8, alpha=.8)
268 ax.scatter(cleared[:, 0], cleared[:, 1], color=ORANGE, s=17, label="成功清除位置", zorder=4)
269 ax.scatter(0, 0, color=TEAL, marker="D", s=25, label="起点", zorder=5)
270 ax.scatter(points[-1], color=BLUE, marker="s", s=20, label="终点", zorder=5)
271 ax.set_title(f"(b) refined 本地轨迹: {distance/1000:.2f} km")
272 ax.legend(loc="upper center", bbox_to_anchor=(.5, -.23), ncol=2, frameon=False, fontsize=7.5)
273 path = save(fig, directory, "q4_route.pdf")
274 return path, {"kind": "analytic_route_and_saved_local_simulation",
275              "source_file": trace_path.relative_to(ROOT).as_posix(),
276              "static_route_m": length, "seed": trace["seed"], "strategy": "refined",
277              "simulated_route_m": distance, "simulated_virtual_time_s": elapsed_us/1_000_000,
278              "cleared_count": len(cleared), "markers_are": "successful_clear_positions_not_source_truth",
279              "timing_independently_recomputed": True}
280
281
282 def formal_compare_figure(directory):
283     source = ROOT / "results/formal_results.json"
284     data = json.loads(source.read_text(encoding="utf-8"))
285     rows = sorted(data["rows"], key=lambda r: (r["problem"], r["attempt"]))
286     assert data["formal"] is True
287     assert [(r["problem"], r["attempt"]) for r in rows] == [(p, i) for p in (3, 4) for i in (1, 2, 3)]
288     for row in rows:
289         assert row["K"] > 0 and row["certificate_complete"] is True
290         assert math.isclose(row["T_over_K_s"], row["virtual_time_s"]/row["K"], abs_tol=1e-8)
291     values = np.asarray([r["virtual_time_s"]/r["K"] for r in rows])
292     x = np.arange(6)
293     fig, ax = plt.subplots(figsize=(6.3, 2.65), layout="constrained")
294     ax.bar(x, values, color=[BLUE]*3+[ORANGE]*3, width=.58, zorder=3)
295     for j, (value, row) in enumerate(zip(values, rows)):
296         ax.annotate(f"{value:.1f}", (j, value), xytext=(0, 7),
297                    textcoords="offset points", ha="center", va="bottom", fontsize=8,
298                    bbox=dict(facecolor="white", edgecolor="none", pad=.15), zorder=6)
299         ax.text(j, value-22, f"K={row['K']}", ha="center", va="top", fontsize=8,
300                color="white", zorder=6)
301     for left, right, group, color, label in ((-.45, 2.45, values[:3], BLUE, "Q3"), (2.55, 5.45, values[3:],
302                                             ORANGE, "Q4")):
303         mean = float(group.mean())
304         ax.hlines(mean, left, right, colors=color, lw=1.1, linestyle="--", zorder=4)
305         ax.text((left+right)/2, 775, f"{label} 等权均值 {mean:.3f}", ha="center", color=color, fontsize=9)
306     ax.axvline(2.5, color=GREY, alpha=.4, lw=.7)
307     ax.set_xticks(x, [f"Q{r['problem']}-{r['attempt']}" for r in rows])
308     ax.set_ylabel("单源时间 / (s/源)", ylim=(0, 820), xlabel="正式测试序号 (均采用 field)")
309     ax.grid(axis="y", alpha=.17, zorder=0)
310     path = save(fig, directory, "formal_compare.pdf")
311     return path, {"kind": "observed_formal_summary", "source_file": source.relative_to(ROOT).as_posix(),
312                  "T_over_K_s": values.tolist(), "K": [r["K"] for r in rows],
313                  "equal_run_means": {"Q3": float(values[:3].mean()), "Q4": float(values[3:].mean())},
314                  "cases_are_paired": False}
315
316 def main():
317     parser = argparse.ArgumentParser(description=__doc__)
318     parser.add_argument("--output-dir", type=Path, default=FIGURES)

```

```

319     args = parser.parse_args()
320     figures = {}
321     for fn in (wedge_figure, angle_figure, q3_tradeoff_figure, q4_route_figure, formal_compare_figure):
322         path, evidence = fn(args.output_dir)
323         figures[path.name] = evidence
324     inputs = [Path(__file__), ROOT/"geometry.py", ROOT/"intersection.py",
325              ROOT/"results/formal_results.json", ROOT/"report/generated/research_trajectory_q4.json"]
326     metadata = {"generator": "redraw_additional.py", "figures": figures,
327               "input_sha256": {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for
328                               p in inputs}}
329     (args.output_dir/"additional_metadata.json").write_text(json.dumps(metadata, ensure_ascii=False, indent=2)+
330                    "\n", encoding="utf-8")
331     print(json.dumps({name: data["kind"] for name, data in figures.items()}, ensure_ascii=False, indent=2))
332
333 if __name__ == "__main__":
334     main()

```